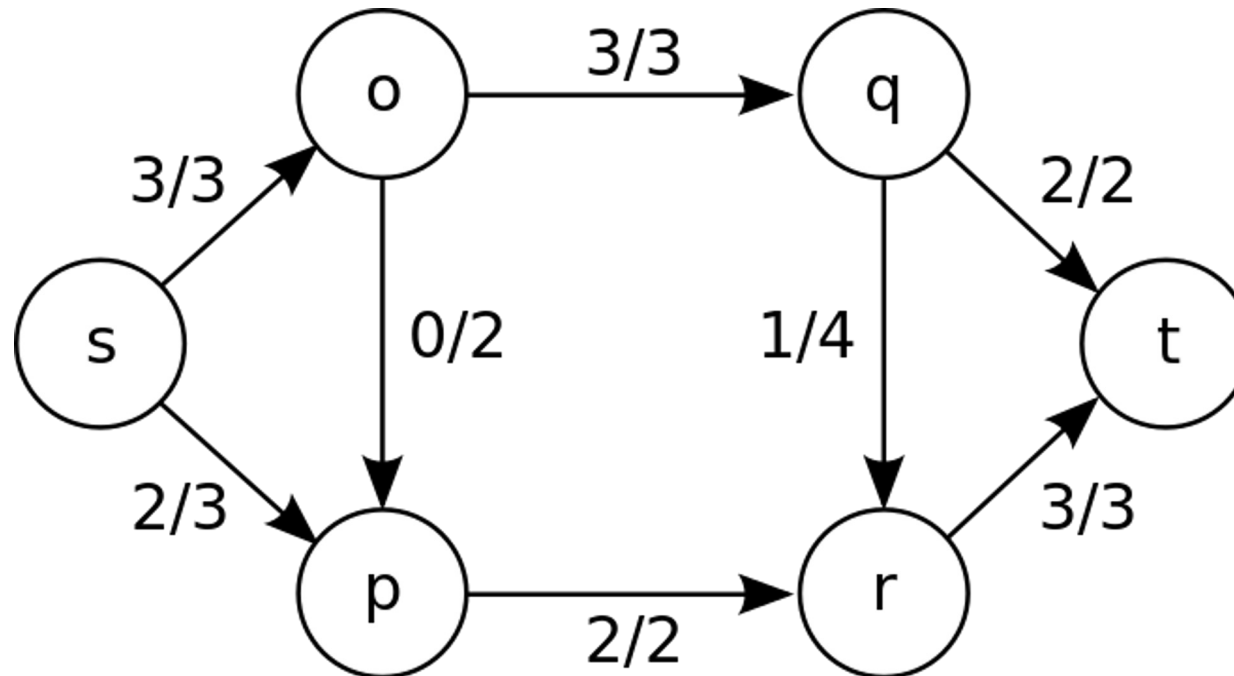


**Максимальный поток.
Максимальный поток
минимальной стоимости.
Многополюсная сеть.**

М.К. Горденко

Определения

Задача о максимальном потоке заключается в нахождении такого потока по транспортной сети, что сумма потоков из истока, или, что то же самое, сумма потоков в сток максимальна.



Определения

- **Остаточная пропускная способность**
- **Дополняющий путь**
- **Насыщенная дуга**
- **Остаточный граф (сеть)**

[http://encyclopaedia.bid/википедия/Поток в графе](http://encyclopaedia.bid/википедия/Поток_в_графе)

Определения

В теории графов **транспортная сеть** - ориентированный граф $G = (V, E)$, в котором каждое ребро $(u, v) \in E$ имеет неотрицательную **пропускную способность** $c(u, v) \in \mathbb{O}$ и **поток** $f(u, v)$. Выделяются две вершины: **источник** s и **сток** t такие, что любая другая вершина сети лежит на пути из s в t . Транспортная сеть может быть использована для моделирования, например, дорожного трафика.

Целочисленная транспортная сеть - транспортная сеть, все пропускные способности рёбер которой - целые числа.

Остаточная пропускная способность (residual capacity) ребра $c_f(u, v) = c(u, v) - f(u, v)$. Она всегда неотрицательна из-за условия на ограничение пропускной способности.

Остаточная сеть (residual network) - граф $G_f = (V, E_f)$, где E_f - множество рёбер с положительной остаточной пропускной способностью. В остаточной сети может быть путь из u в v , даже если его нет в исходной сети. Это выполняется, когда в исходной сети есть обратный путь (v, u) и поток по нему положителен.

Увеличивающий (остаточный, дополняющий) путь (augmenting path) - это путь $(u_1, u_2, \dots, u_k)$ в остаточной сети, где $u_1 = s$, $u_k = t$ и $c_f(u_i, u_{i+1}) > 0$. Ниже доказано, что поток максимален тогда и только тогда, когда нет увеличивающего пути в остаточной сети.

Определения

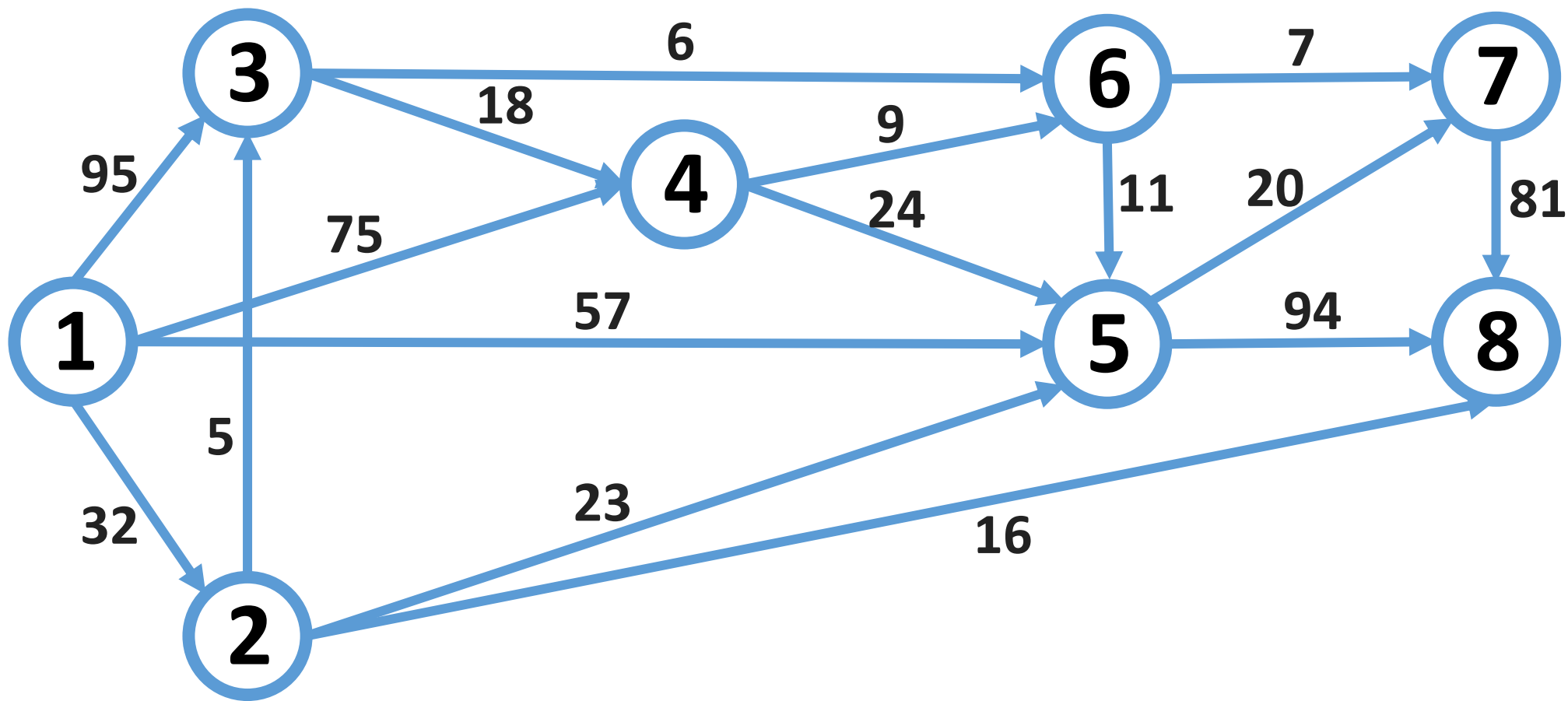
- Цикл
- Поток
- Исток
- Сток
- Пропускная способность

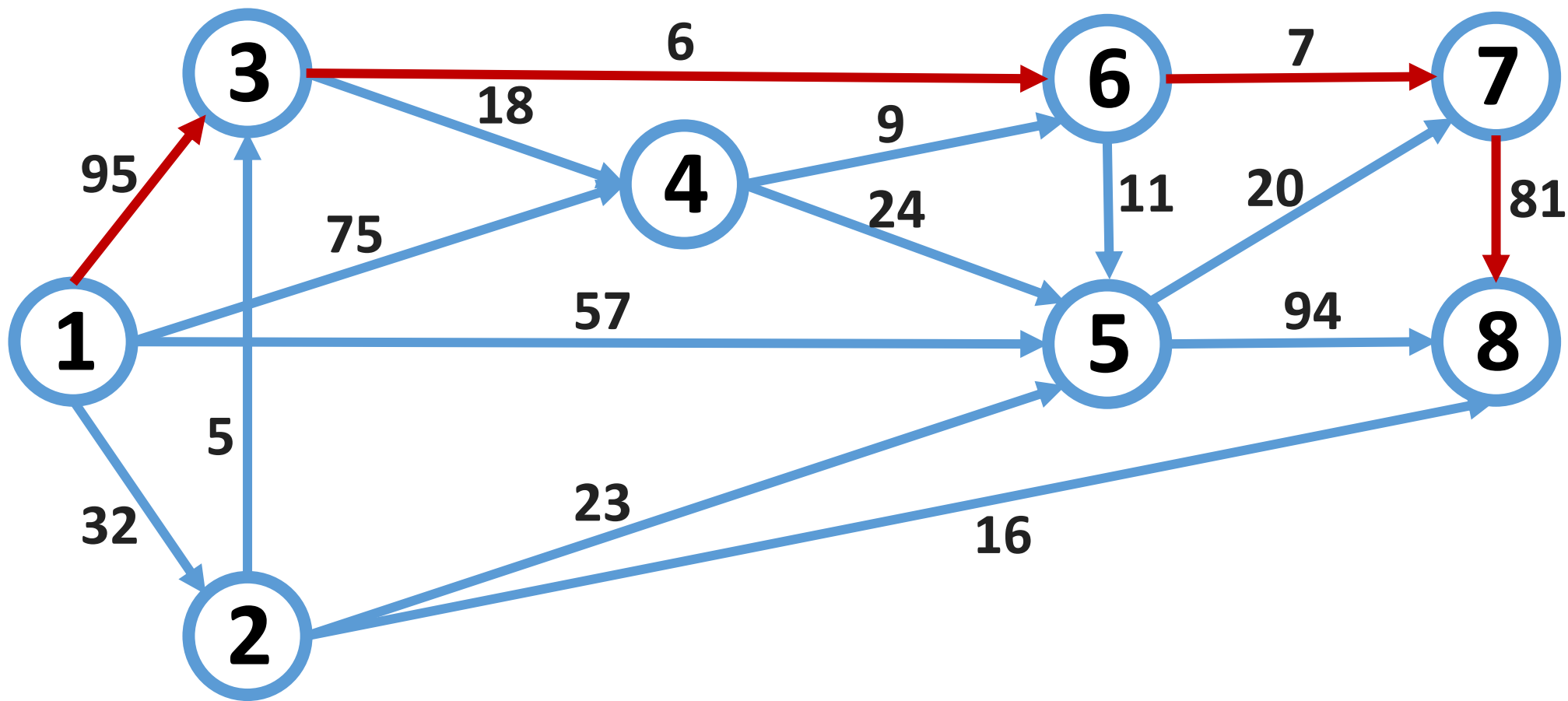
Алгоритм Форда — Фалкерсона

- **Алгоритм Форда — Фалкерсона** решает задачу нахождения максимального потока в транспортной сети.

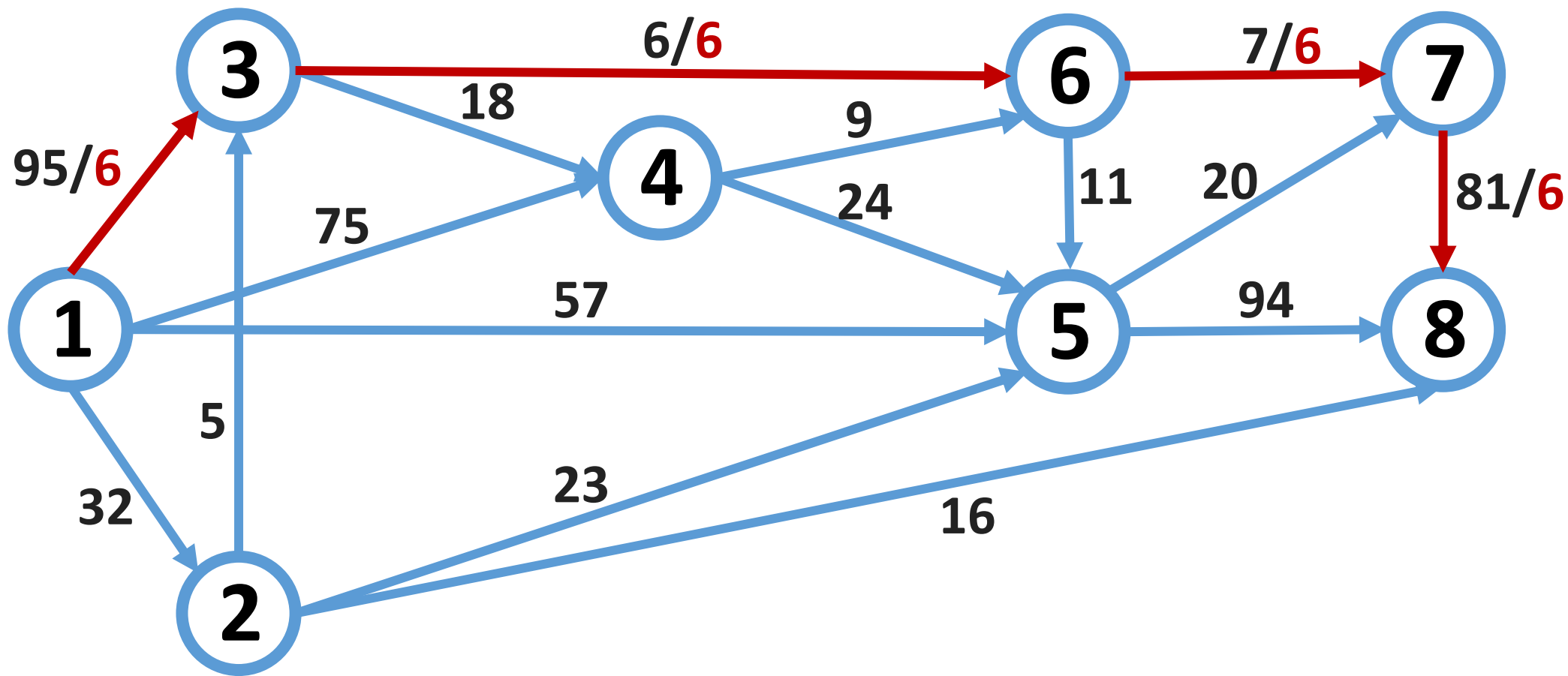
Алгоритм Форда — Фалкерсона

1. Обнуляем все потоки. Остаточная сеть изначально совпадает с исходной сетью.
2. В остаточной сети находим любой путь из источника в сток. Если такого пути нет, останавливаемся.
3. Пускаем через найденный путь (он называется *увеличивающим путём* или *увеличивающей цепью*) максимально возможный поток:
 1. На найденном пути в остаточной сети ищем ребро с минимальной пропускной способностью $C_{\min}$.
 2. Для каждого ребра на найденном пути увеличиваем поток на $C_{\min}$, а в противоположном ему — уменьшаем на $C_{\min}$.
 3. Модифицируем остаточную сеть. Для всех рёбер на найденном пути, а также для противоположных им рёбер, вычисляем новую пропускную способность. Если она стала ненулевой, добавляем ребро к остаточной сети, а если обнулилась, стираем его.
4. Возвращаемся на шаг 2.

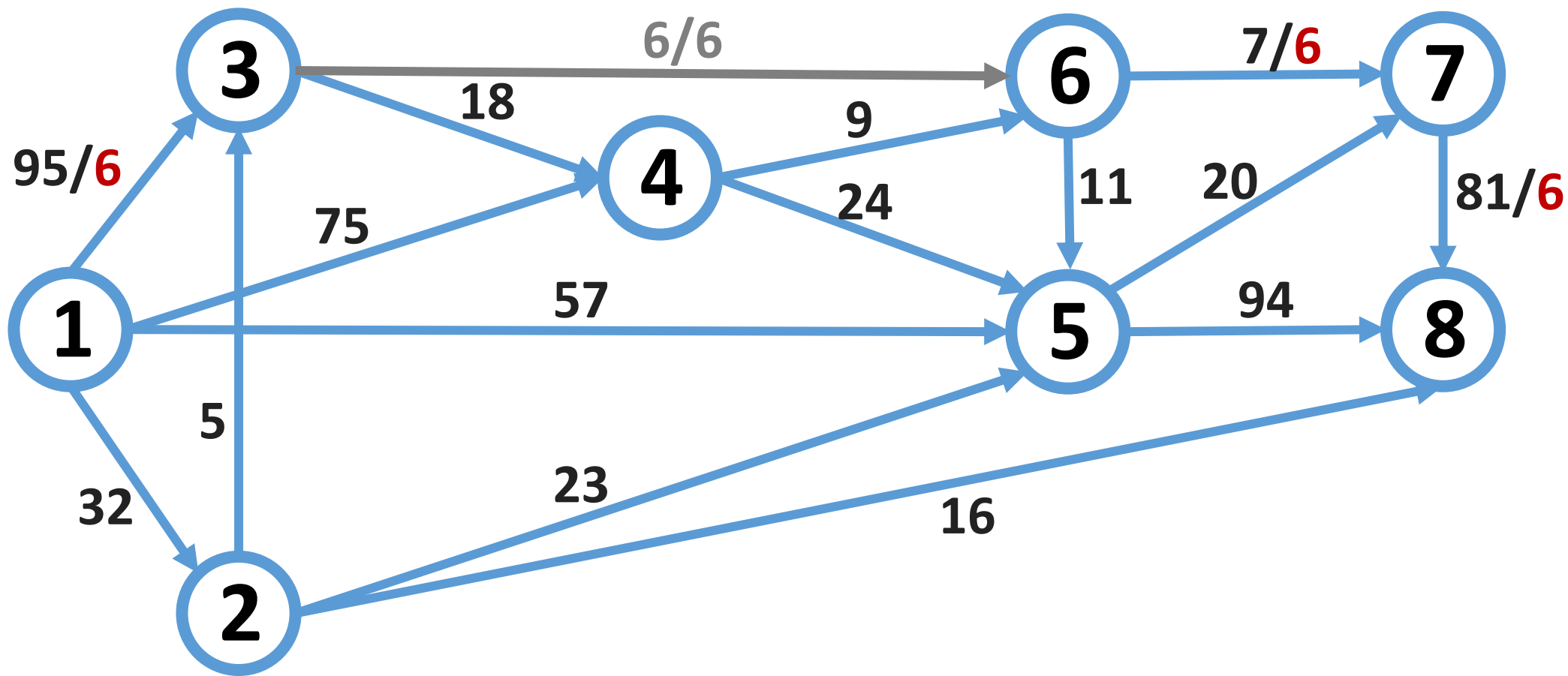




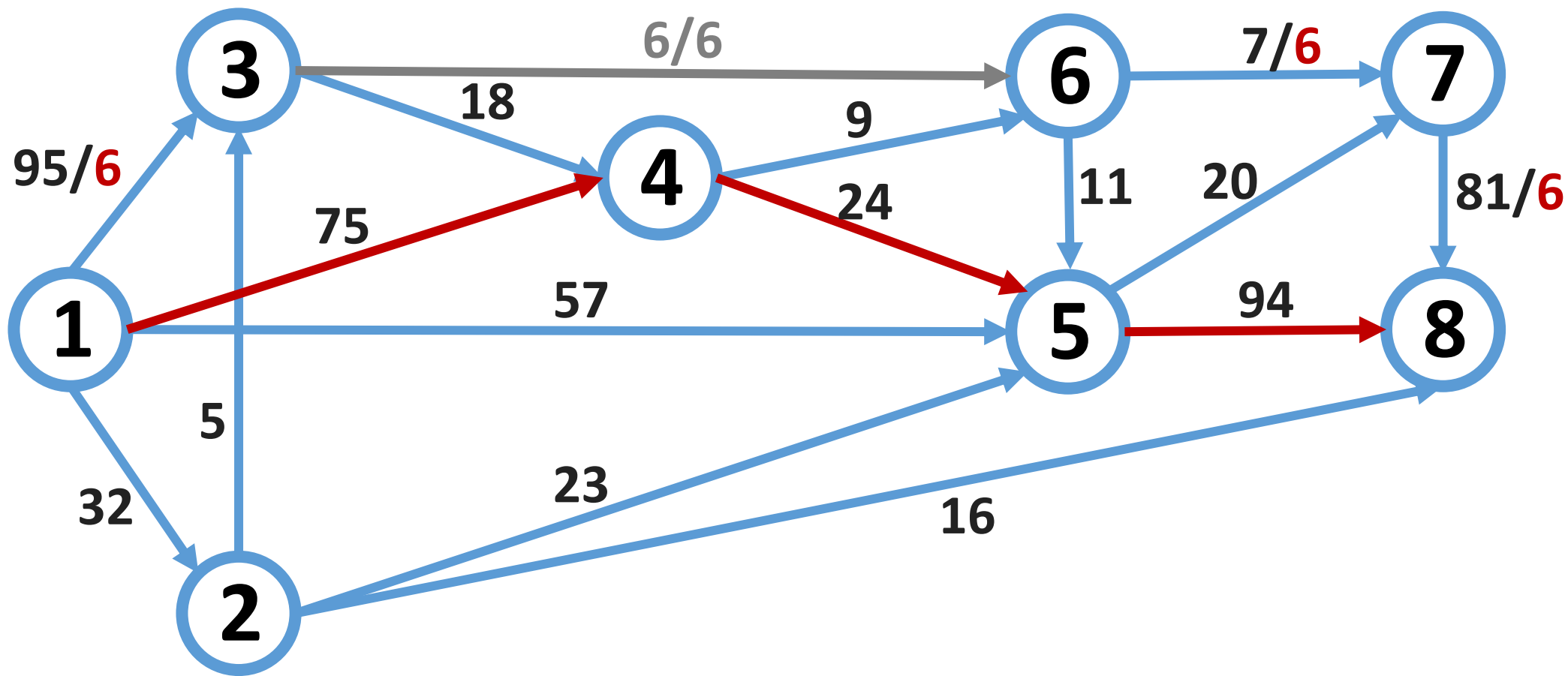
1-3-6-7-8



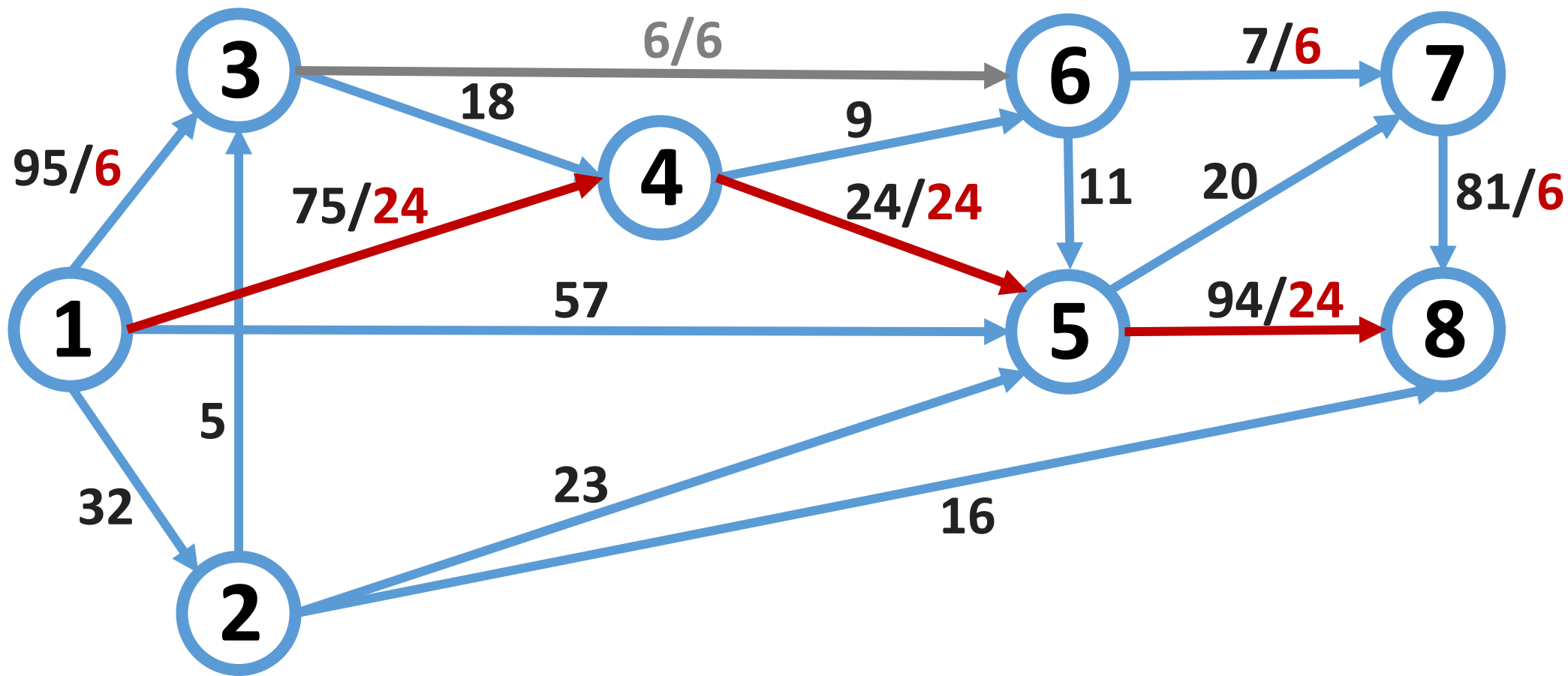
1-3-6-7-8



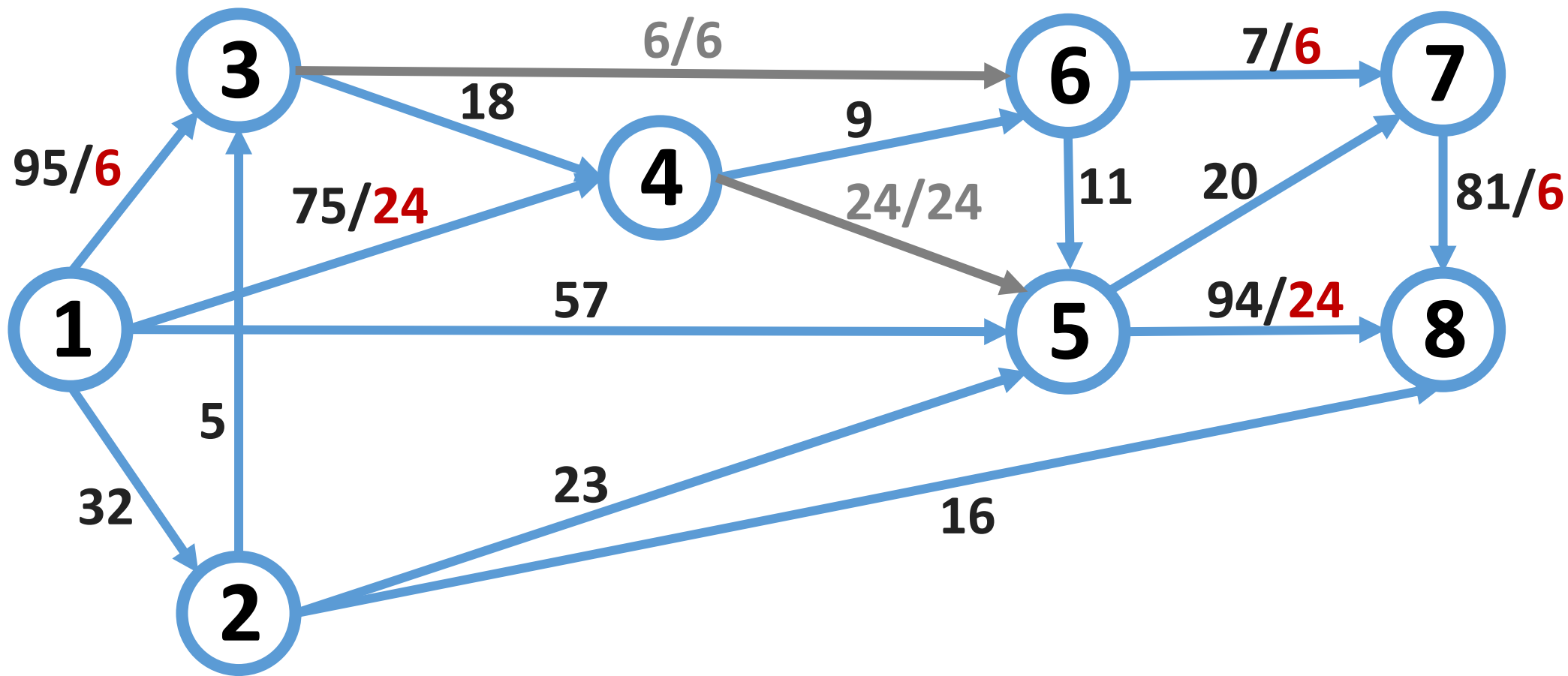
1-3-6-7-8



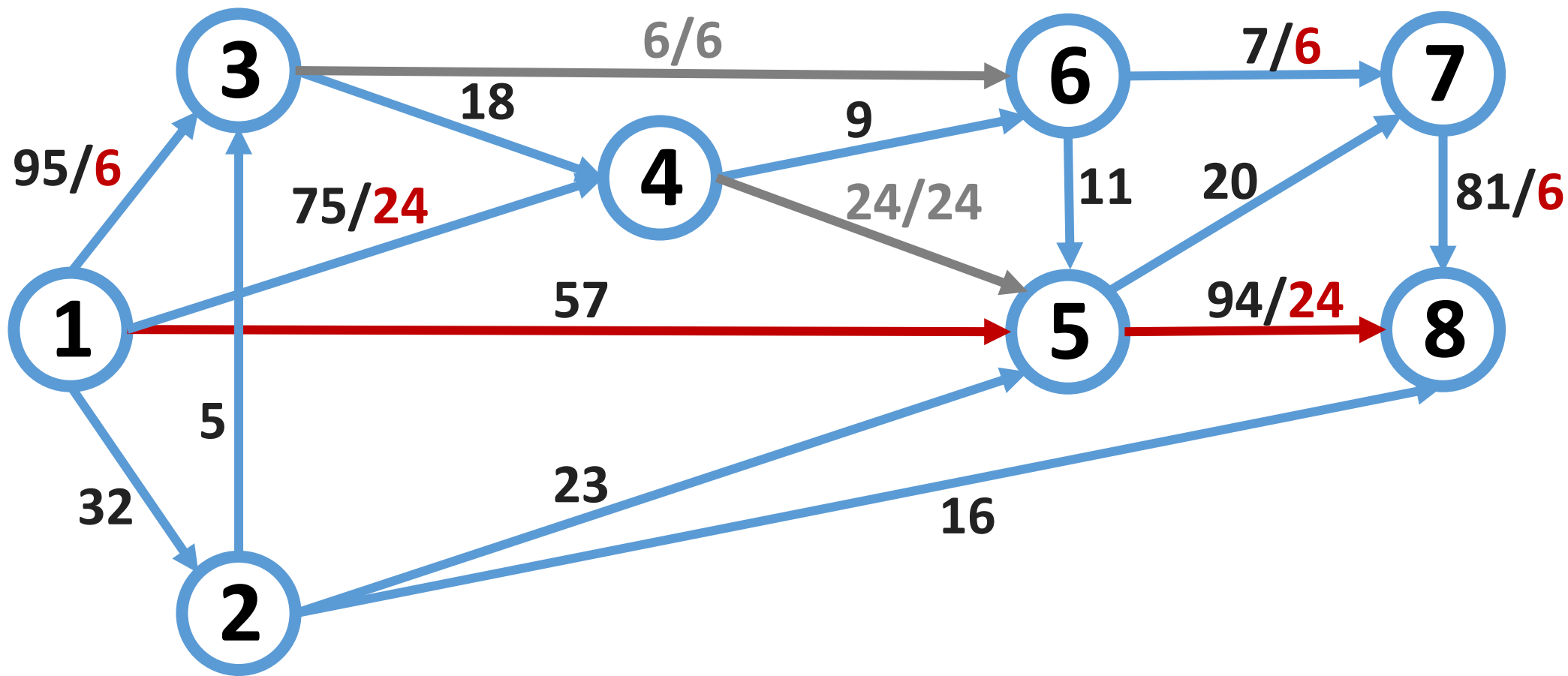
1-4-5-8



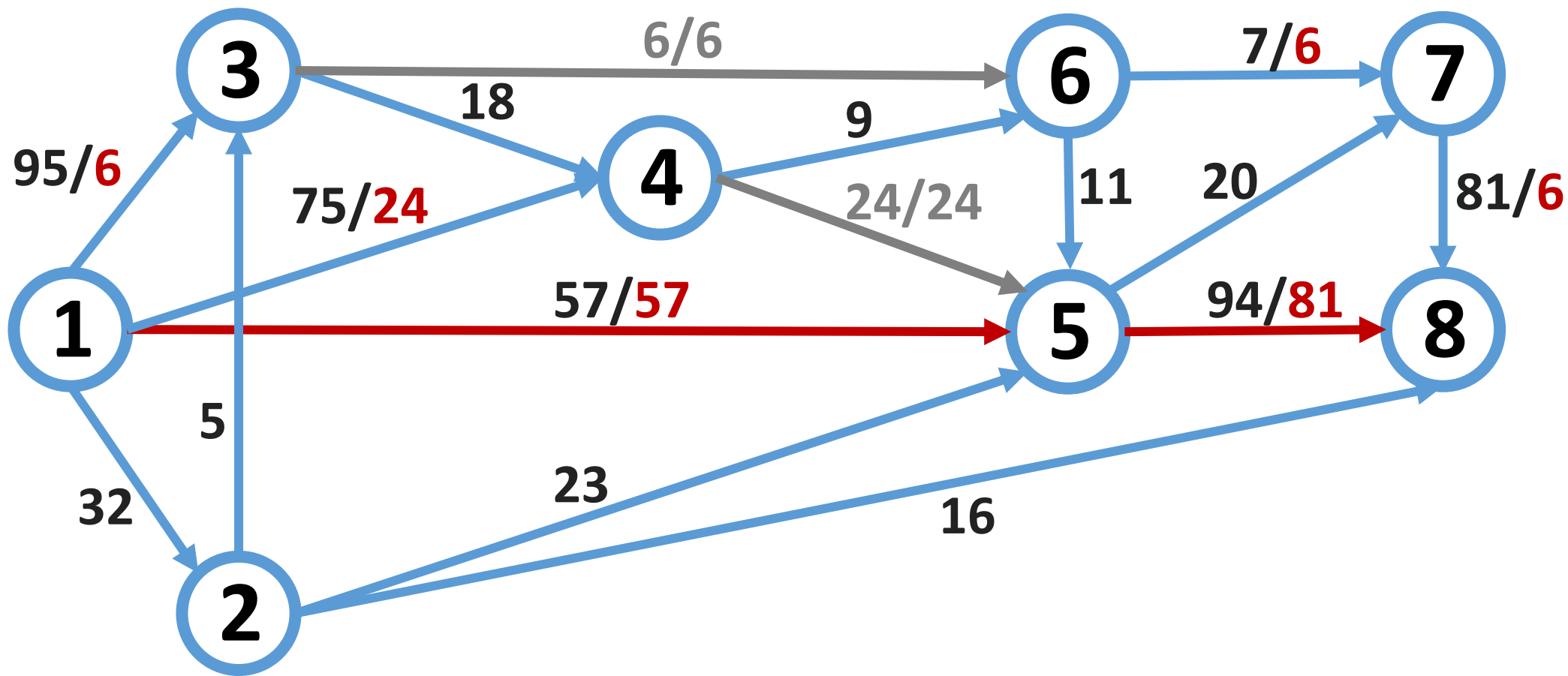
1-4-5-8



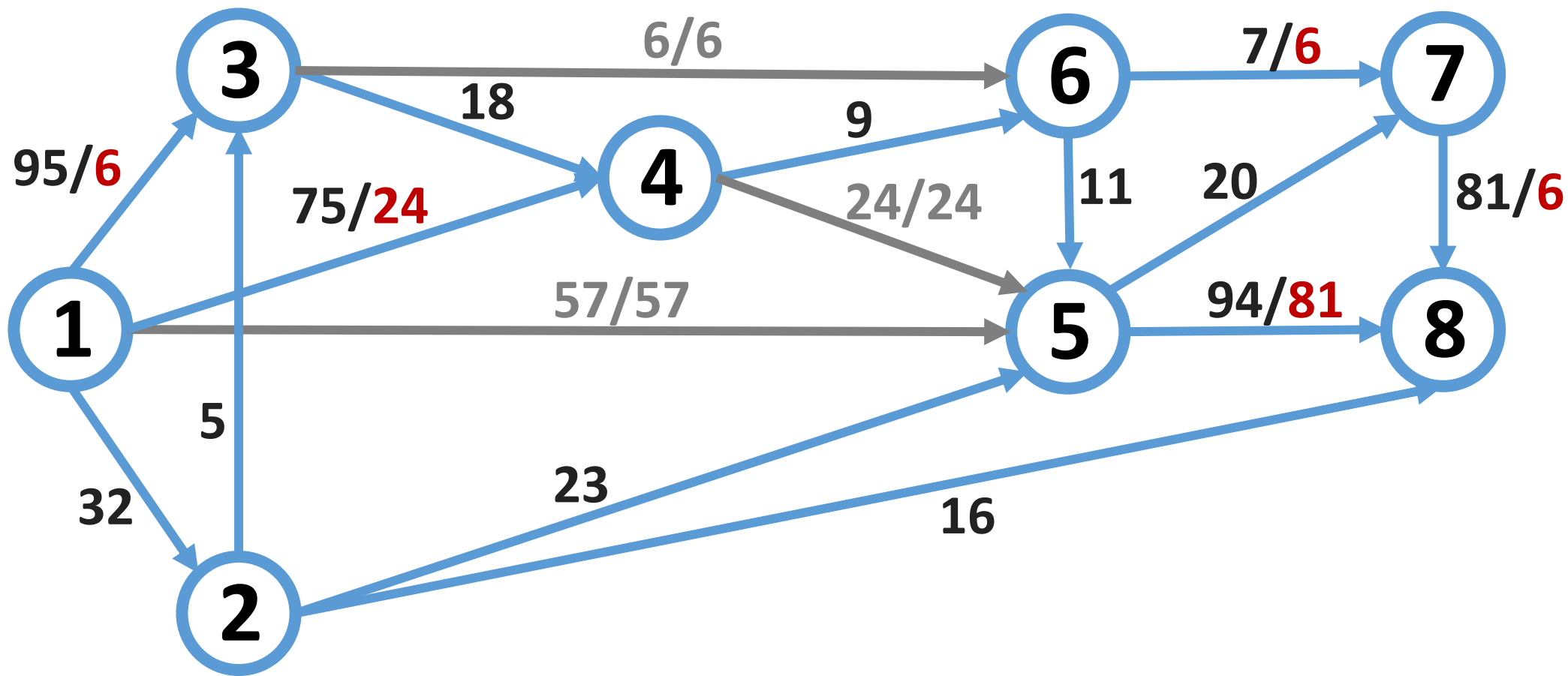
1-4-5-8



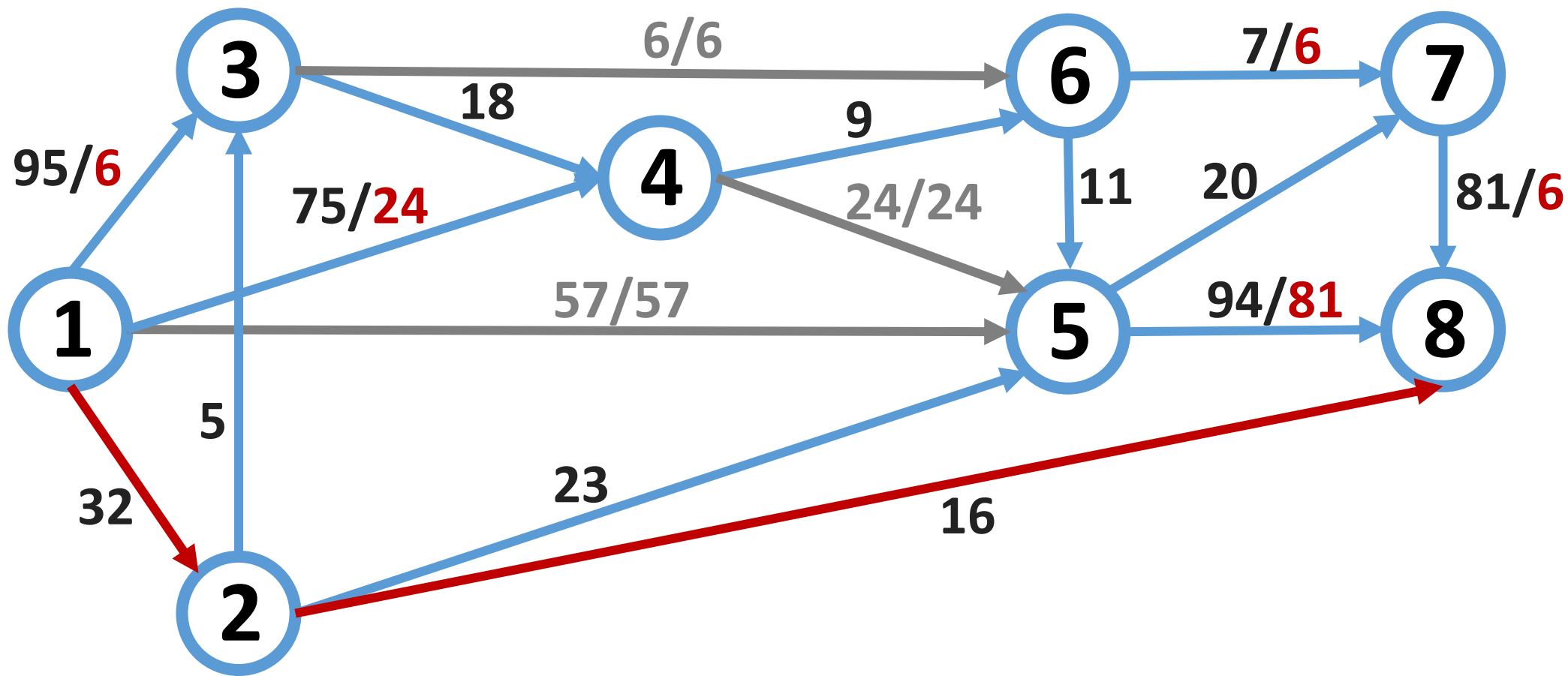
1-5-8



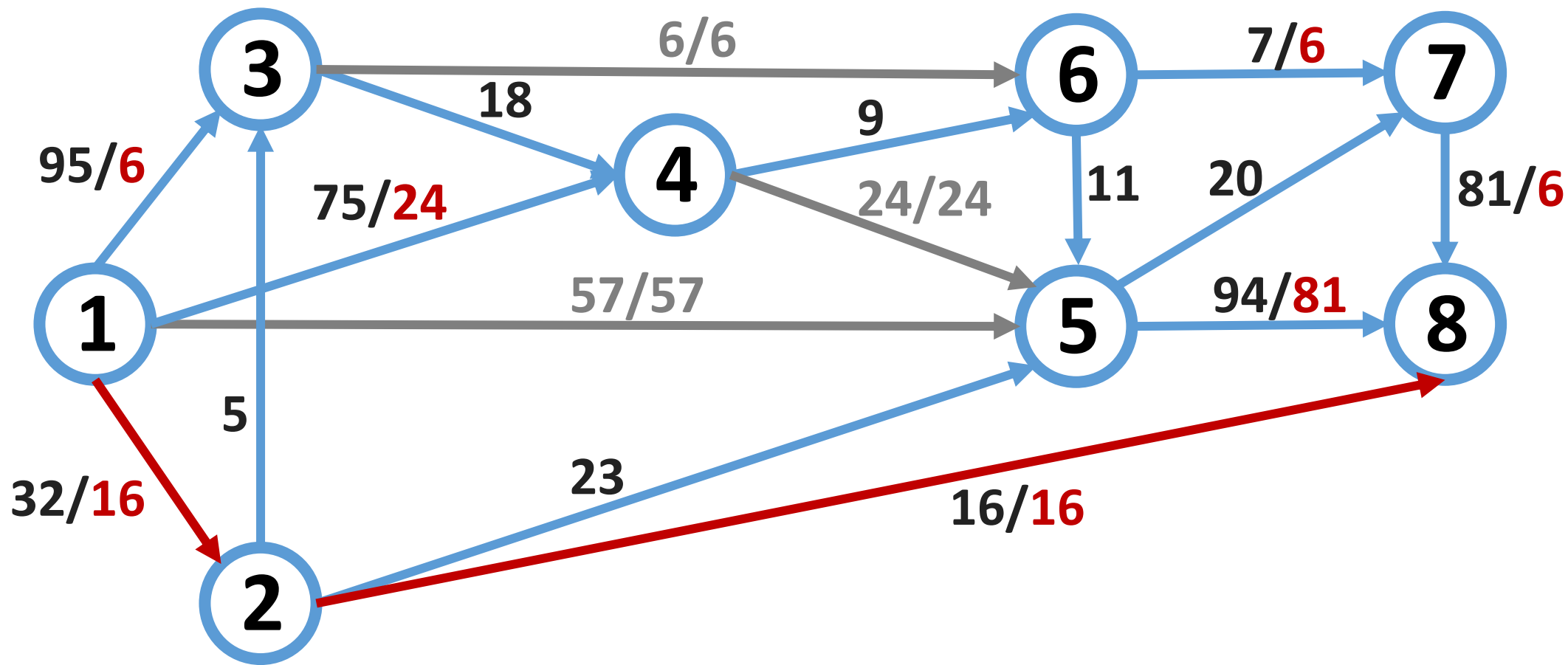
1-5-8



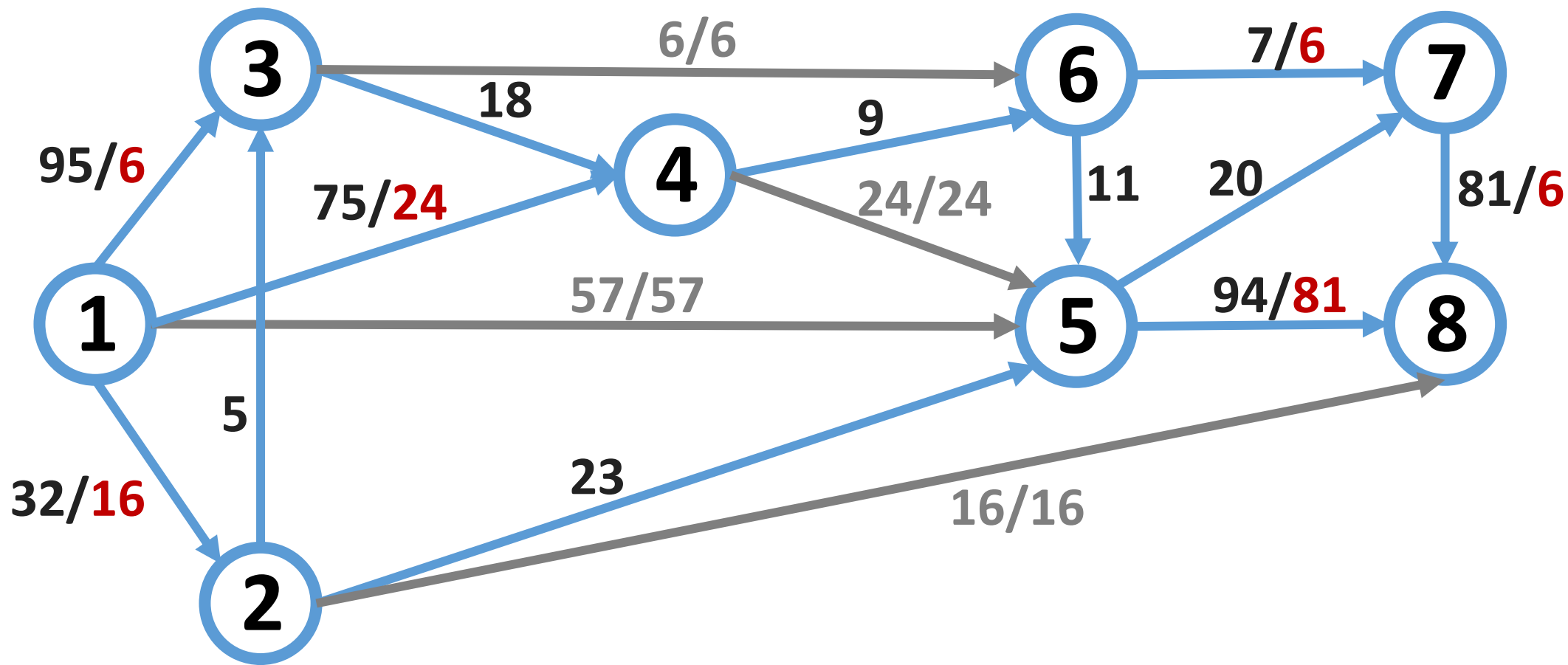
1-5-8



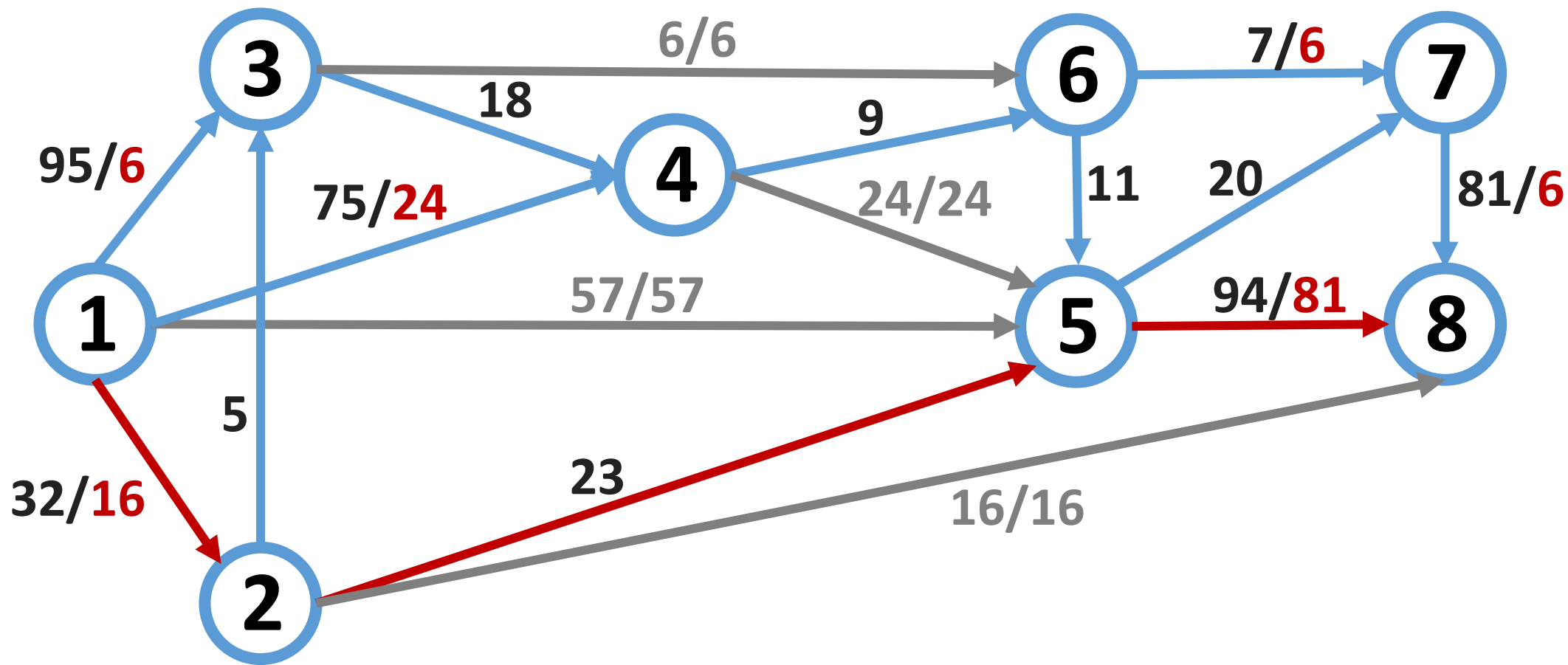
1-2-8



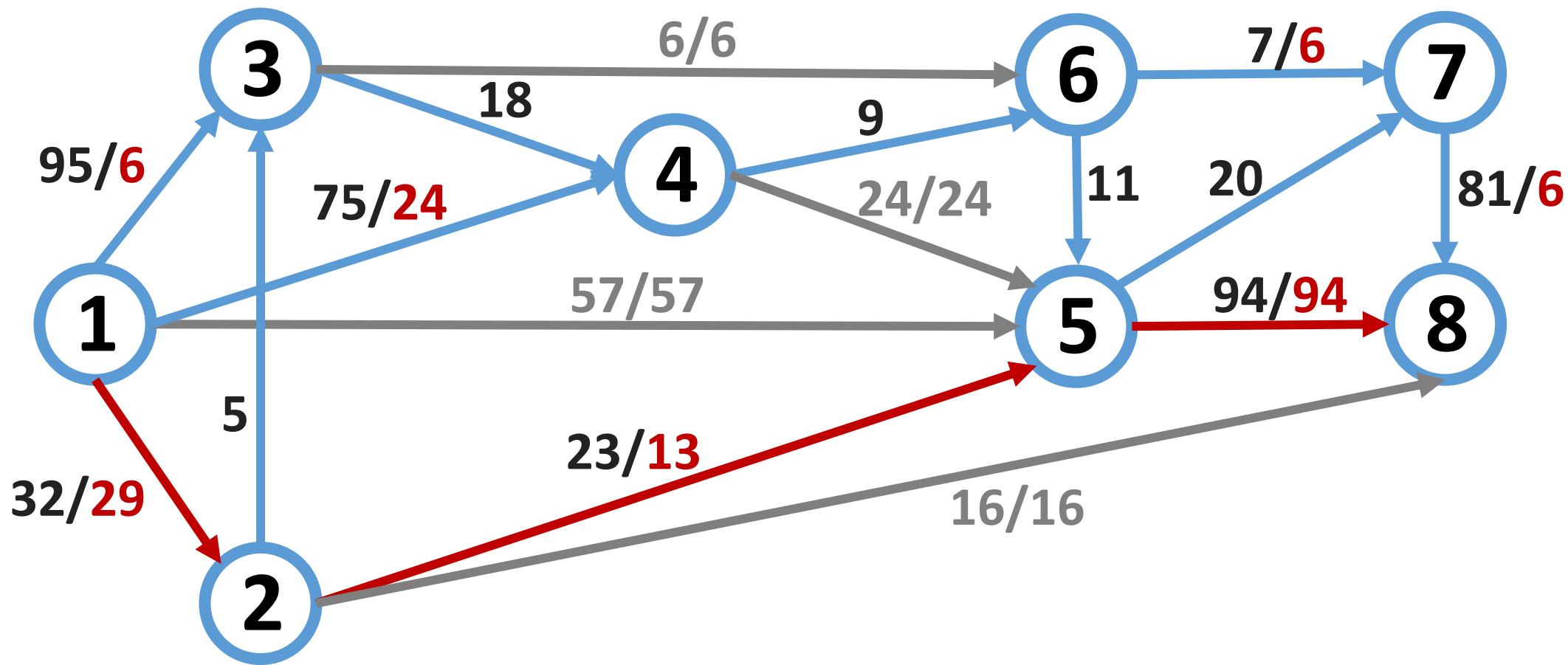
1-2-8



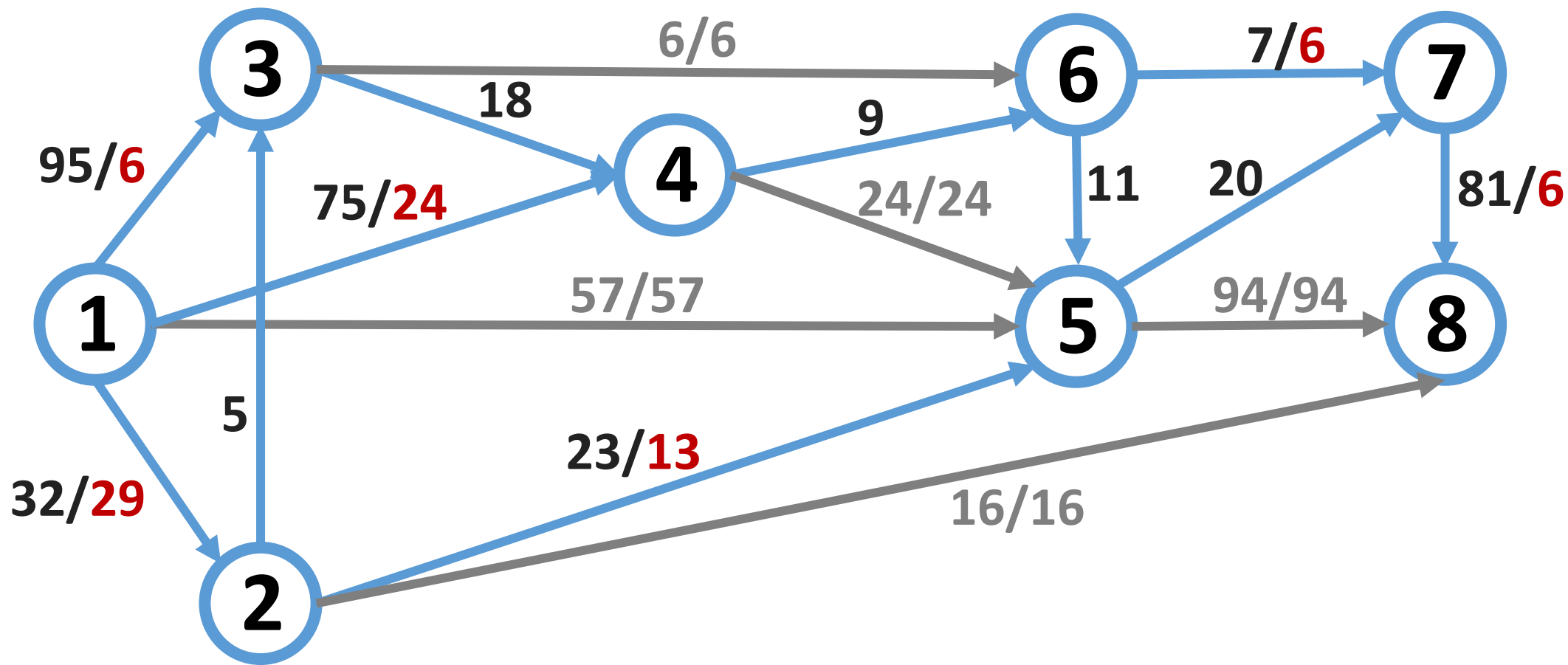
1-2-8



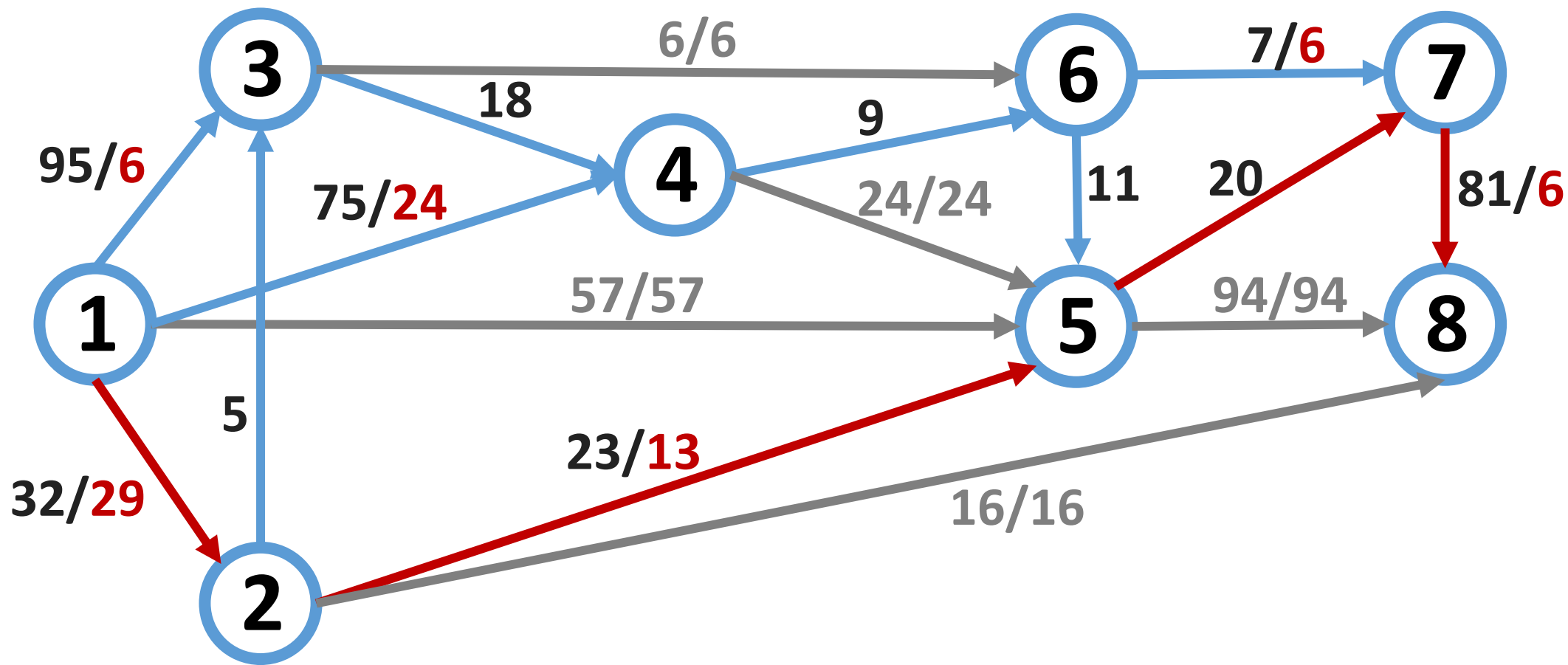
1-2-5-8



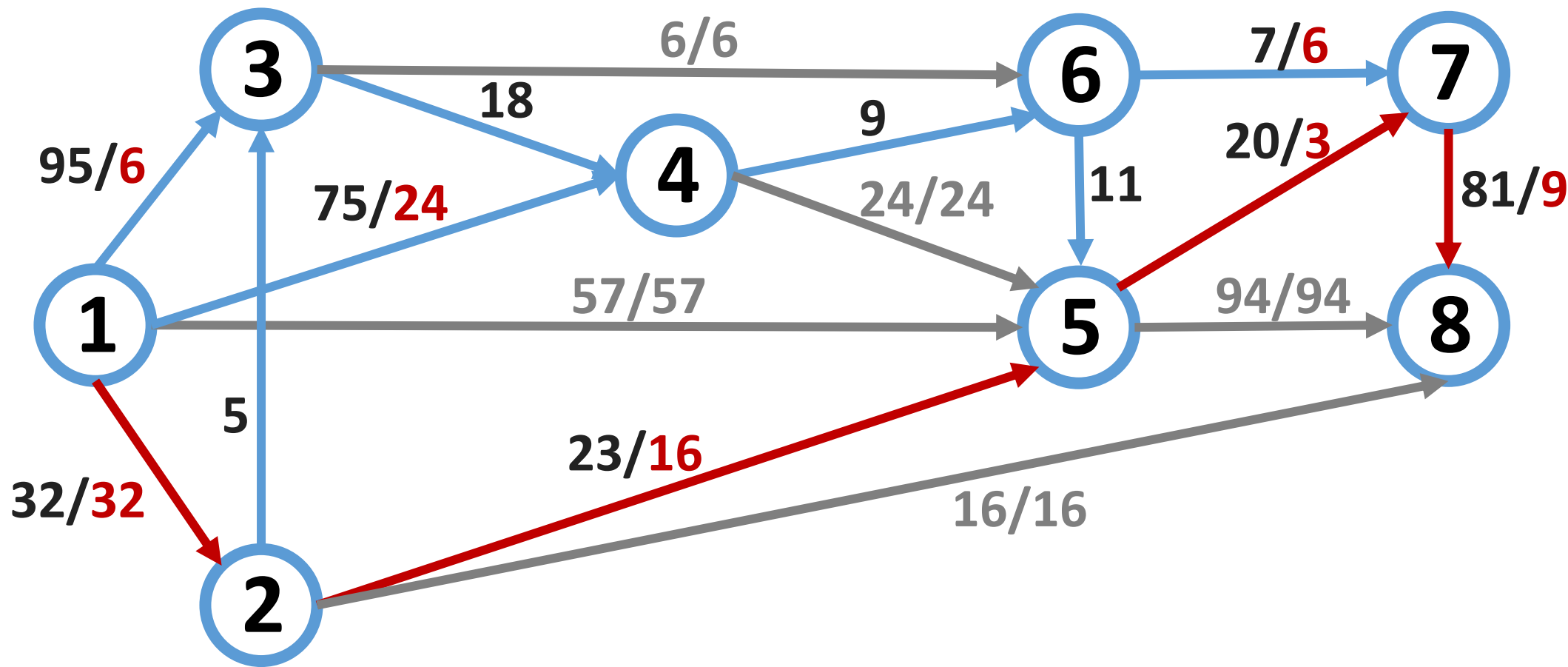
1-2-5-8



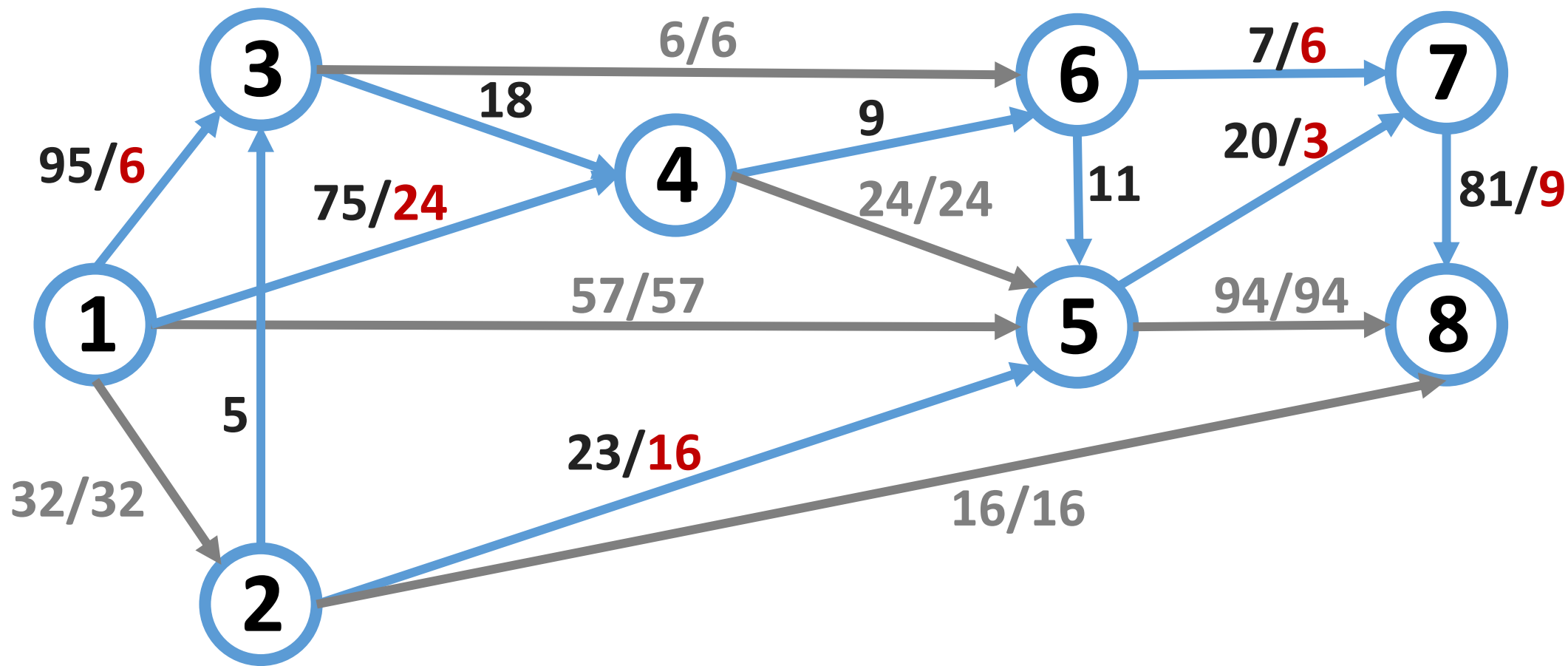
1-2-5-8



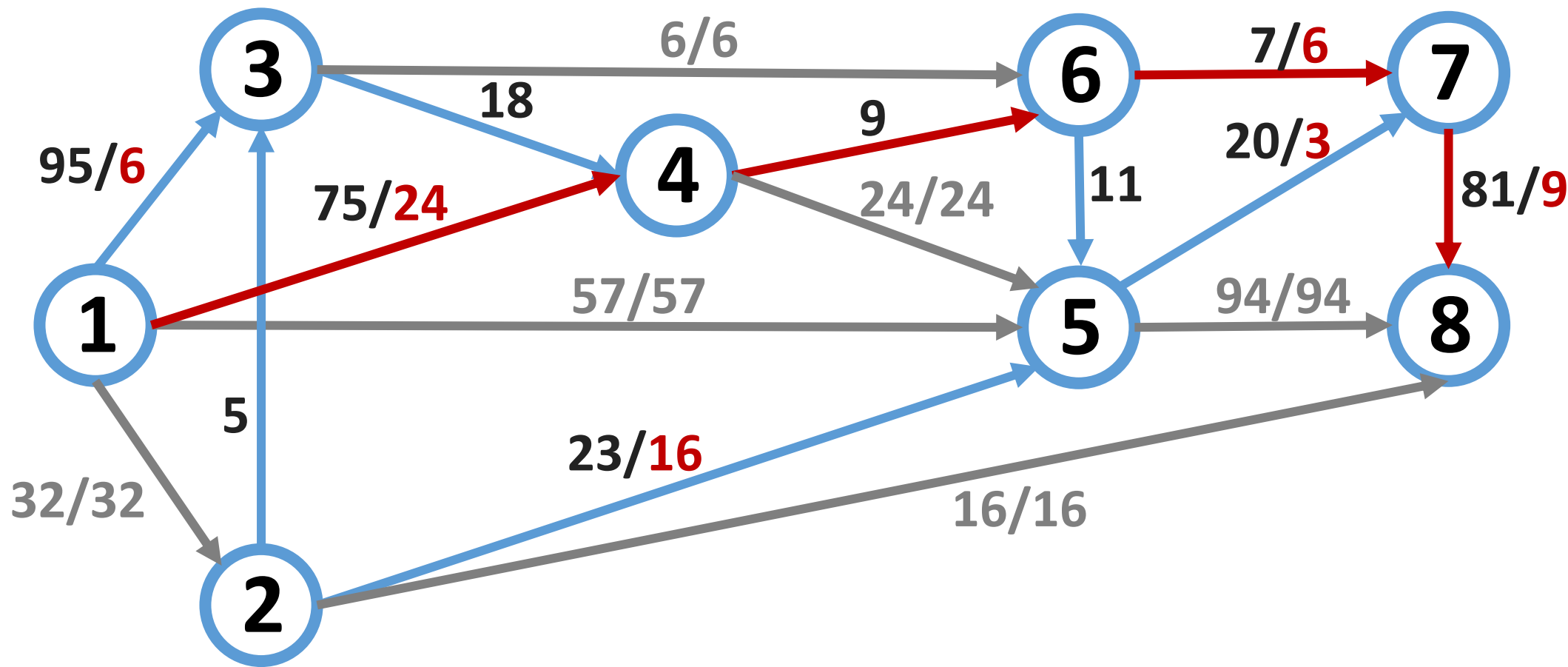
1-2-5-7-8



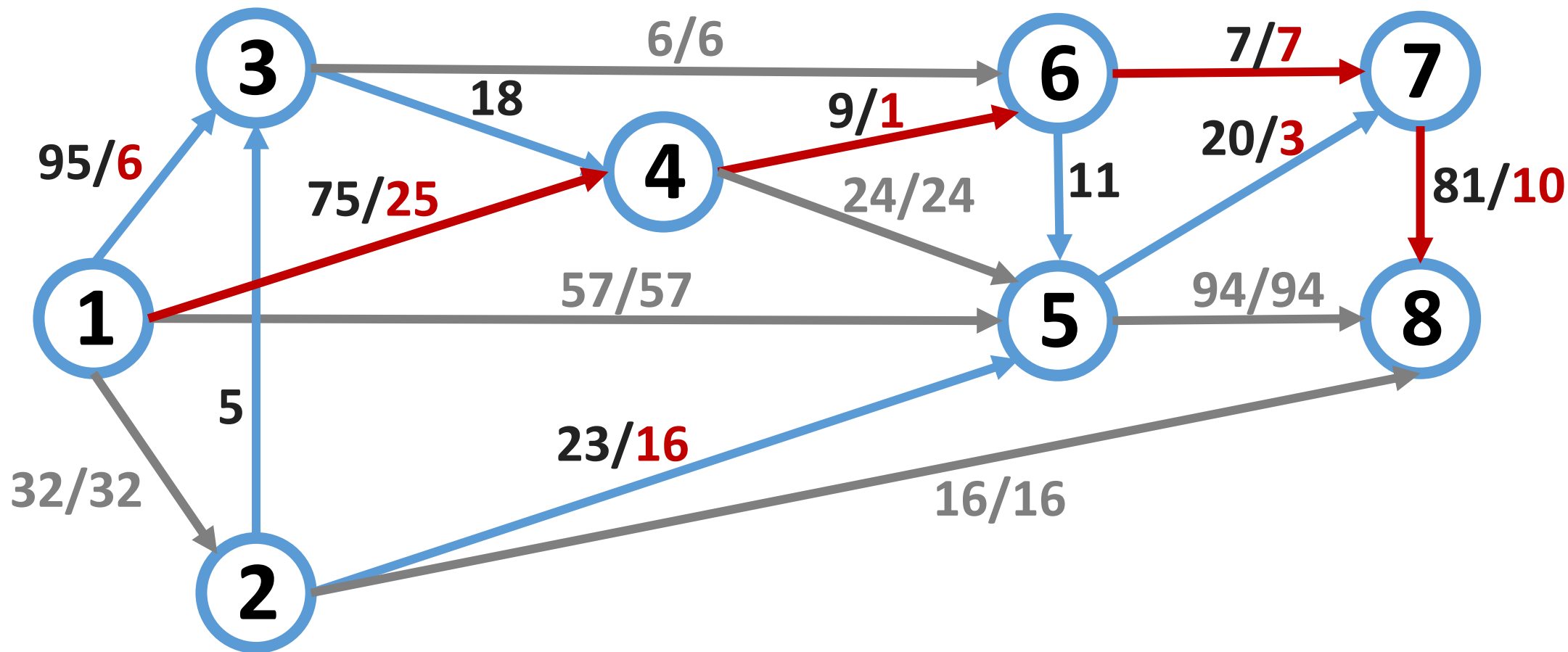
1-2-5-7-8



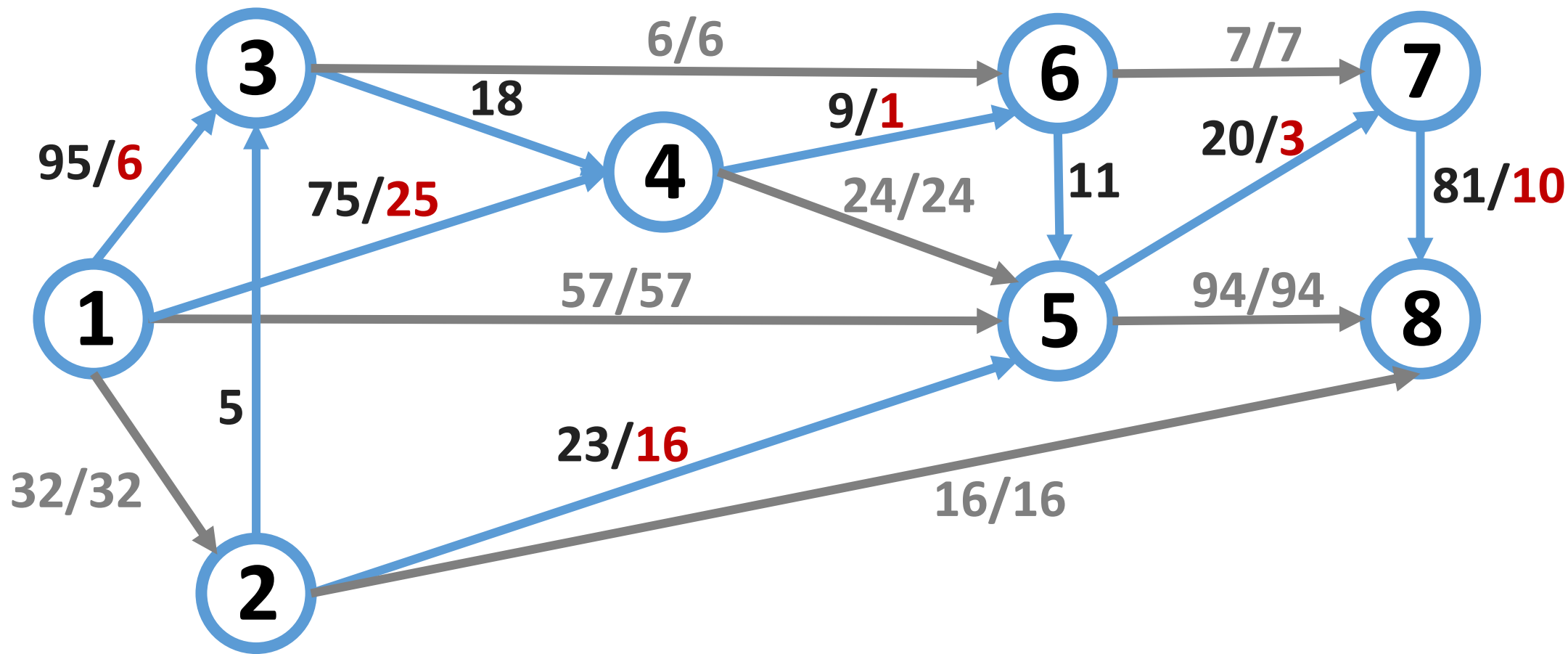
1-2-5-7-8



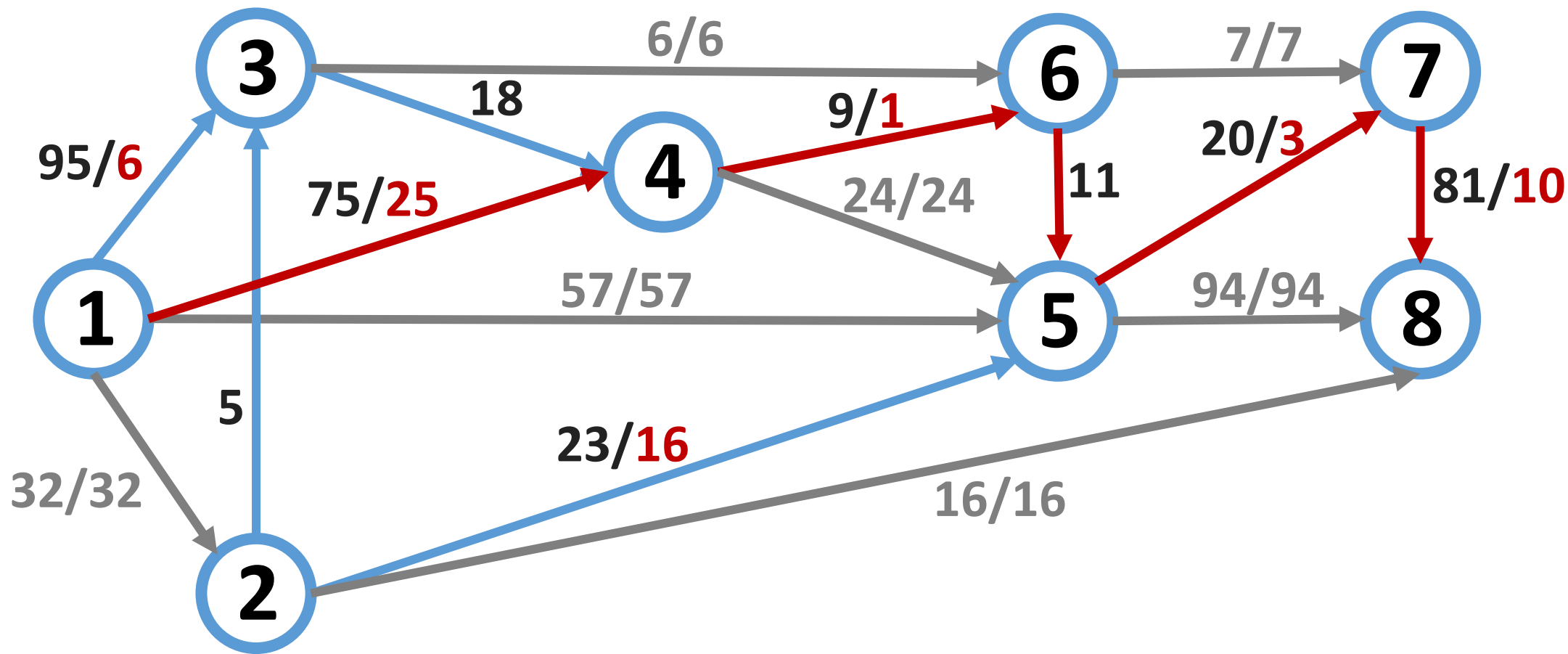
1-4-6-7-8



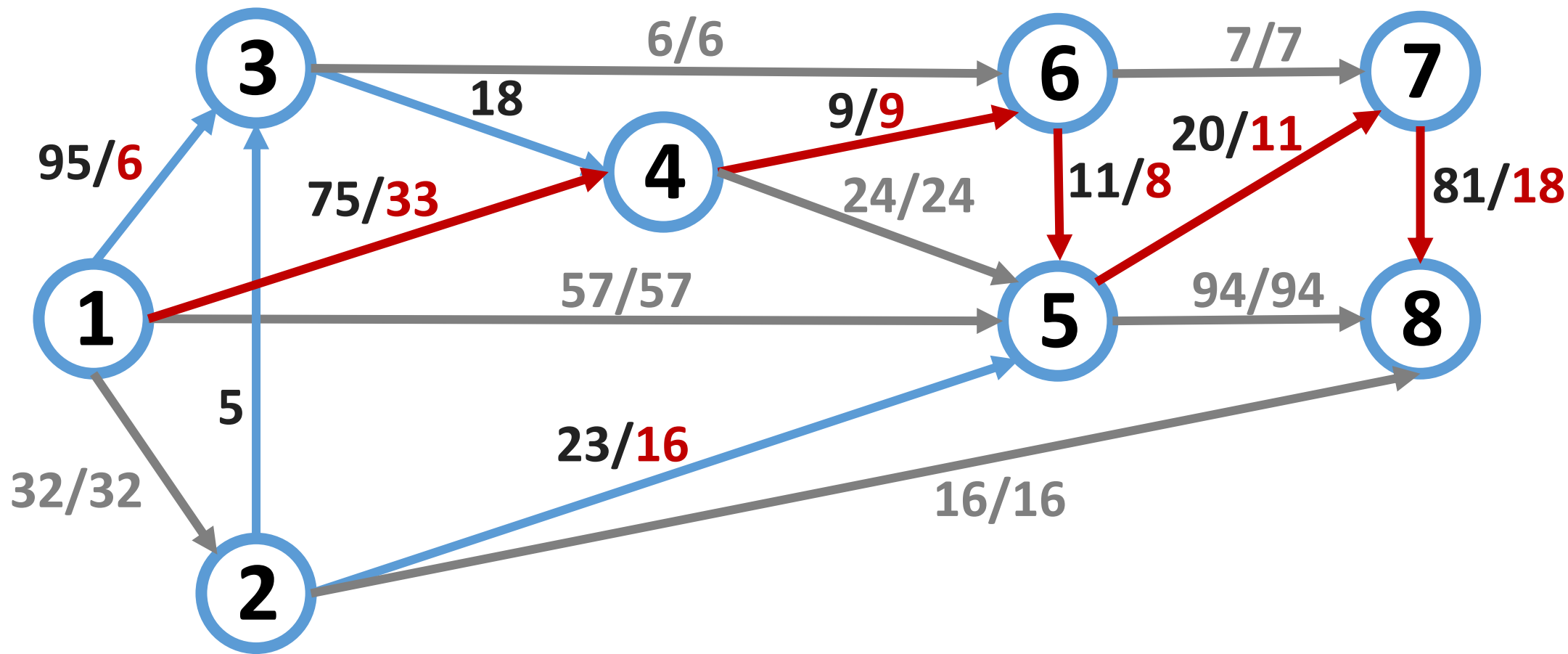
1-4-6-7-8



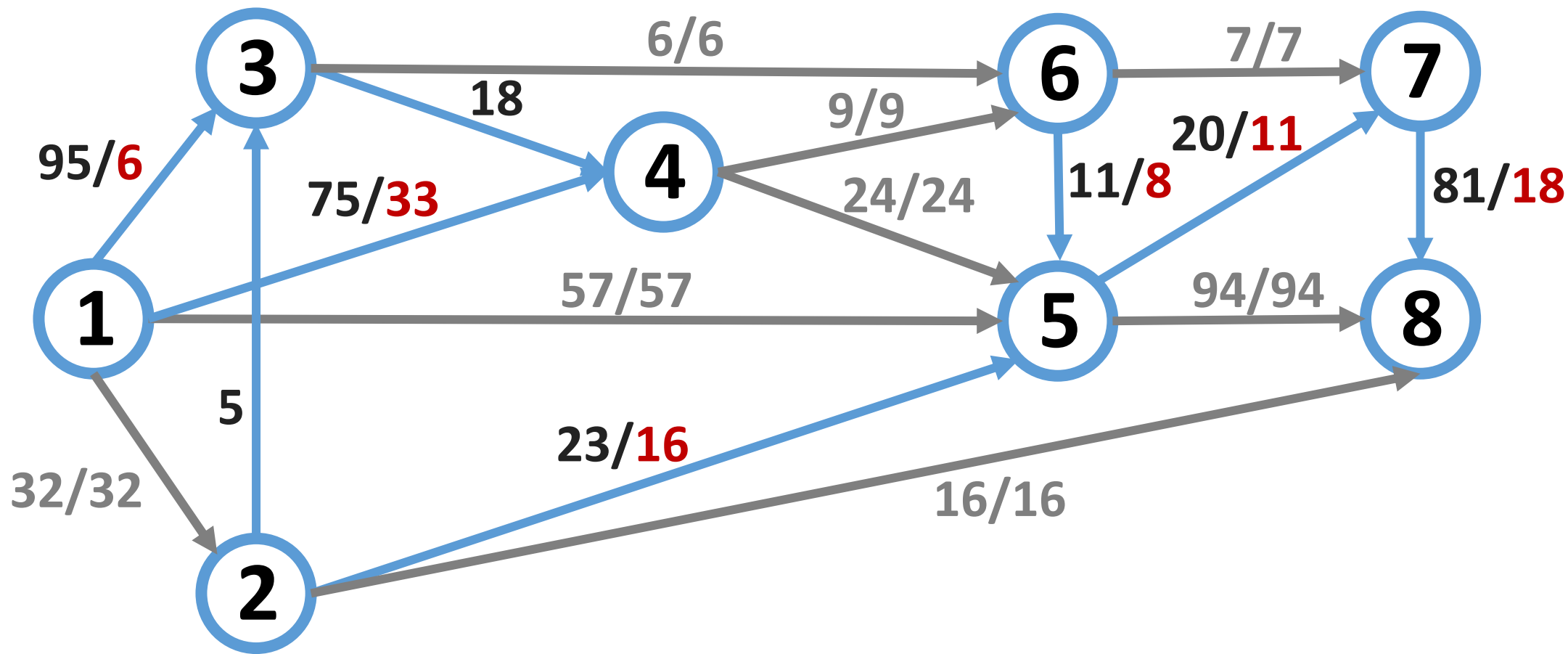
1-4-6-7-8



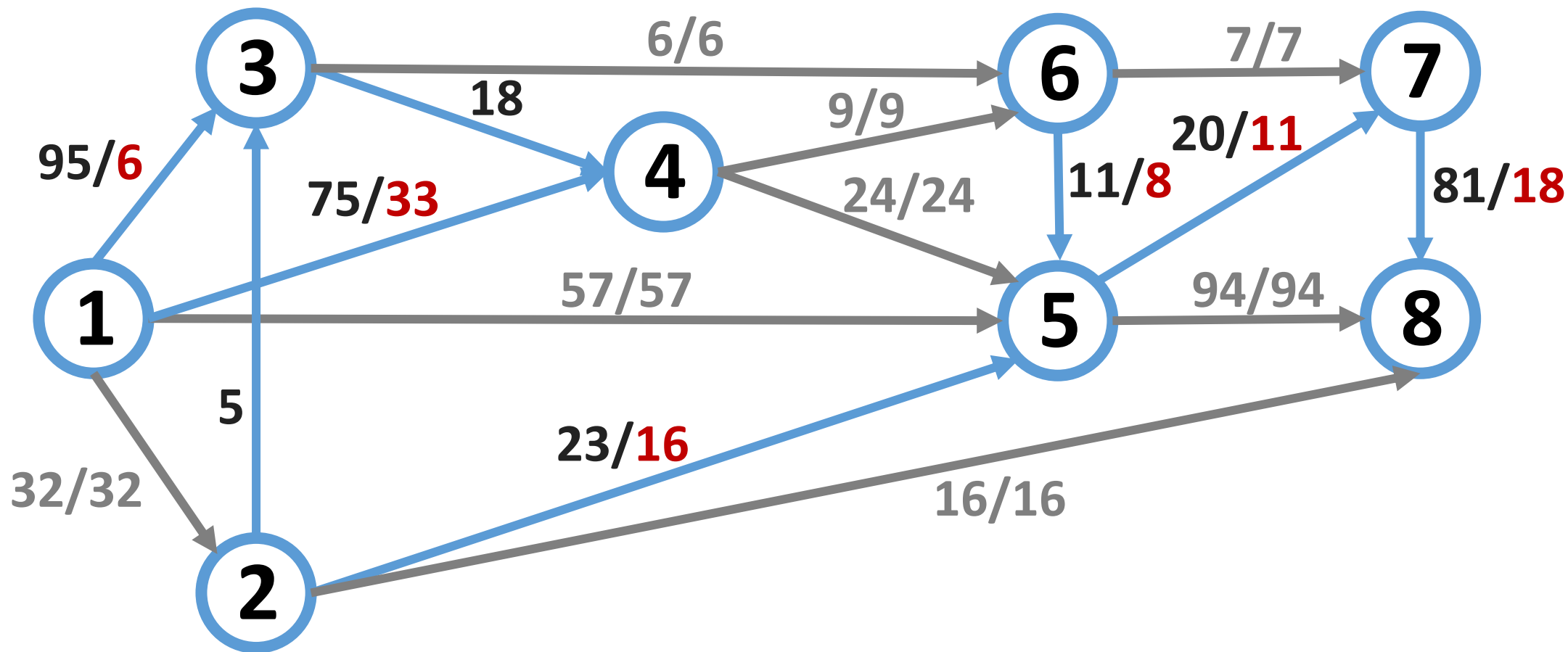
1-4-6-5-7-8



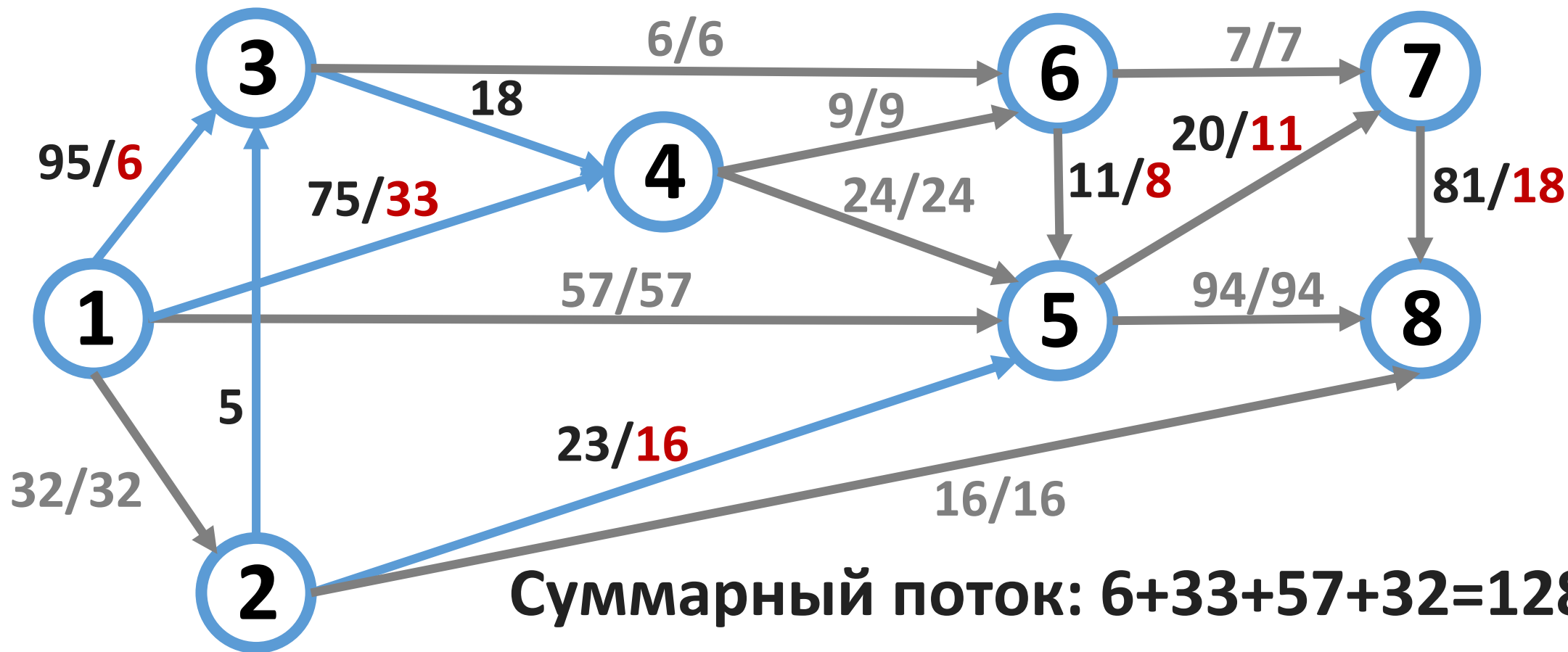
1-4-6-5-7-8

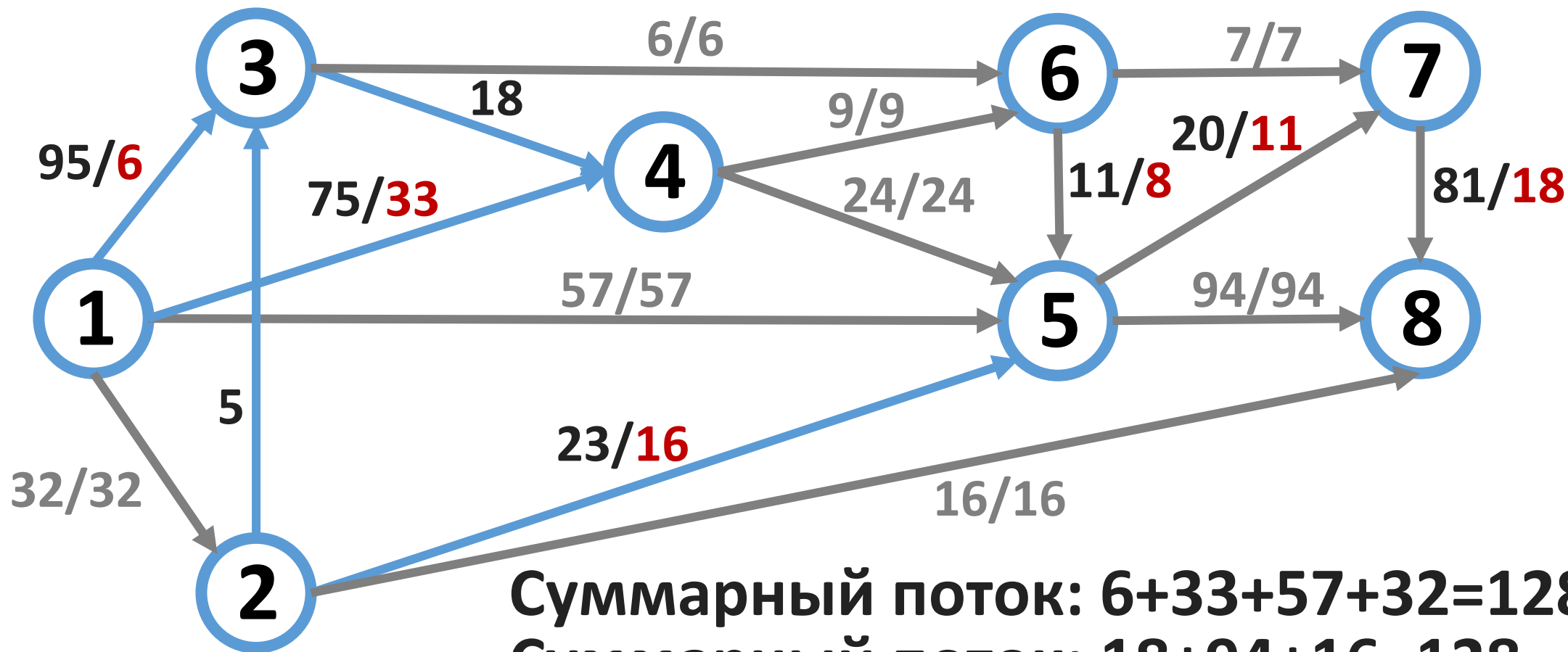


1-4-6-5-7-8

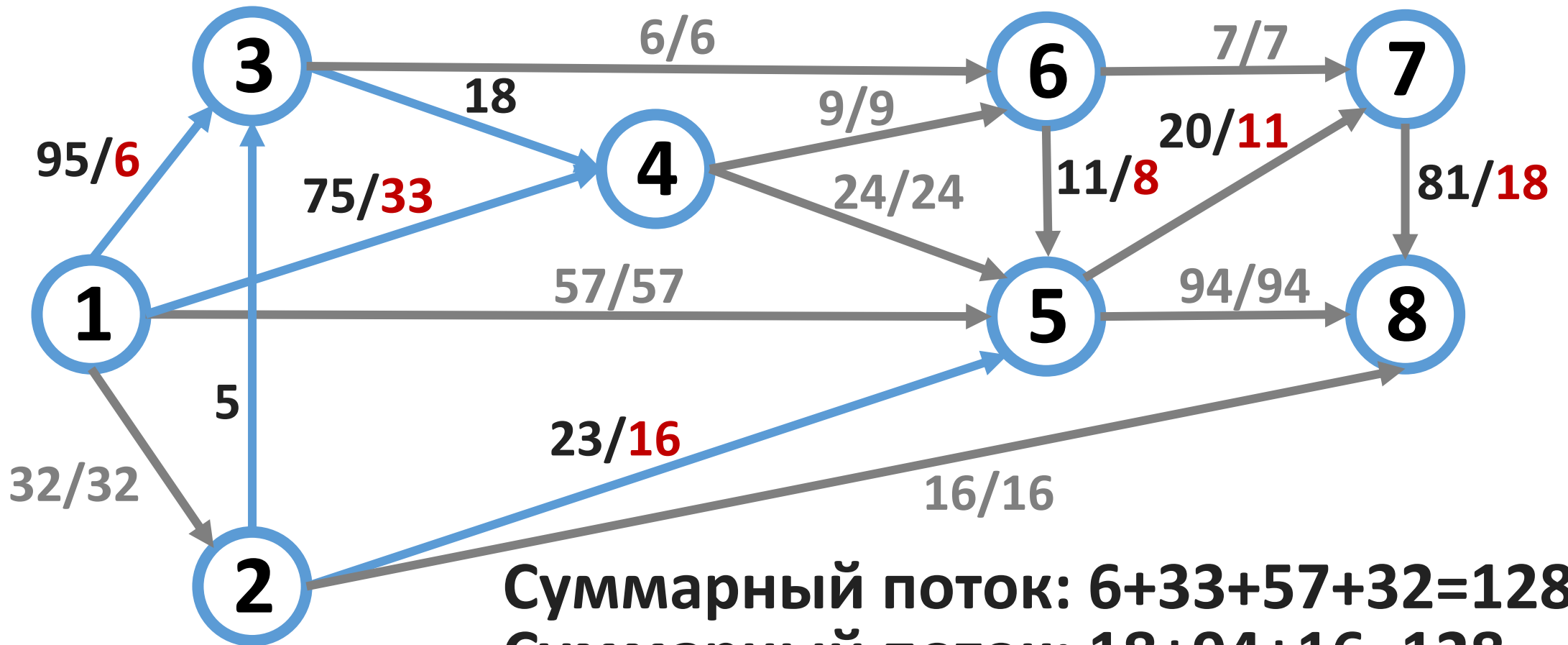


Путей нет!





Суммарный поток: $6+33+57+32=128$
Суммарный поток: $18+94+16=128$

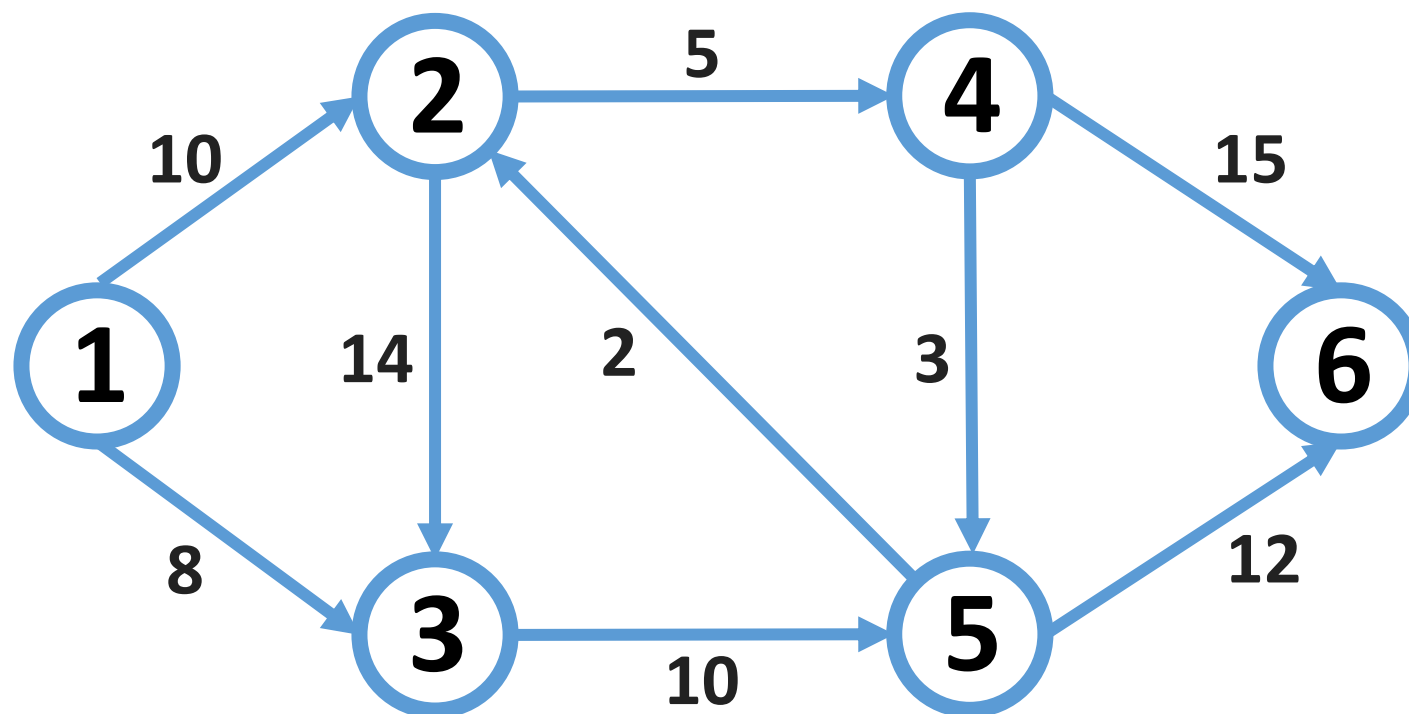


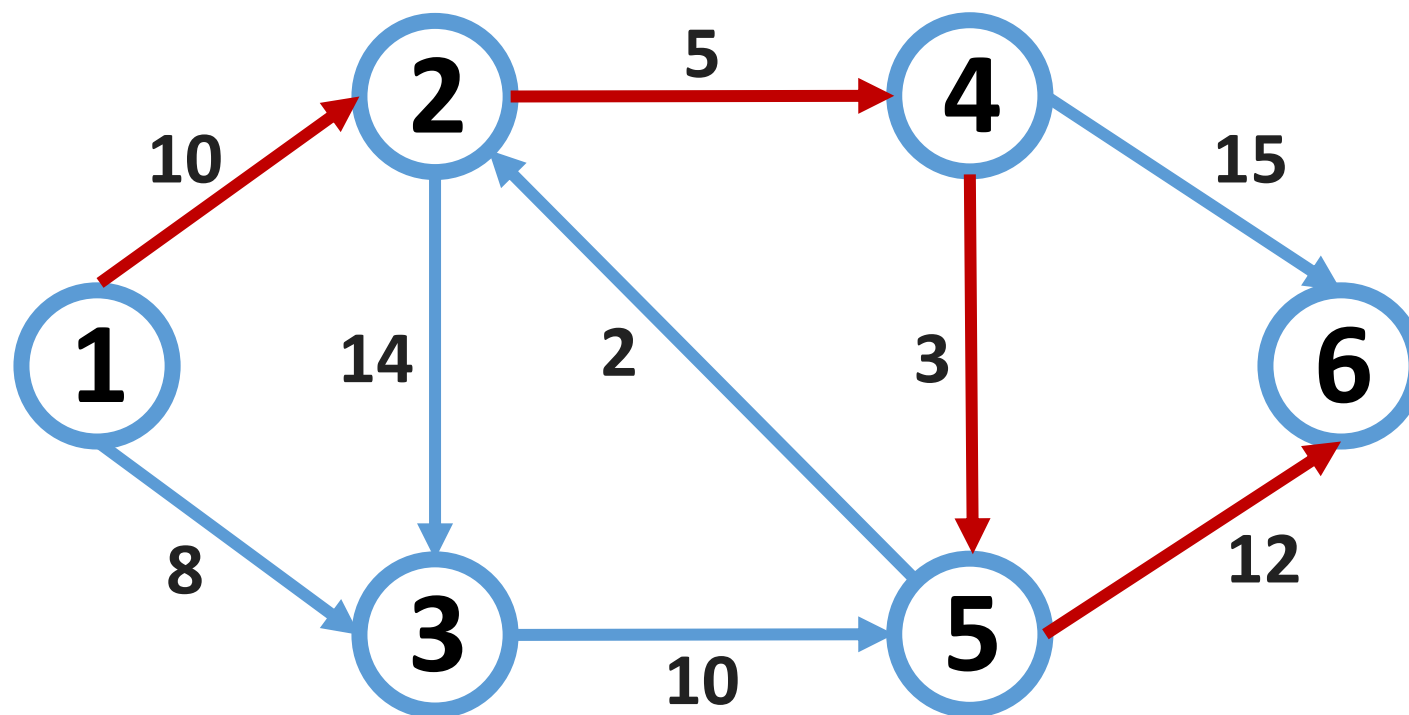
Суммарный поток: $6+33+57+32=128$

Суммарный поток: $18+94+16=128$

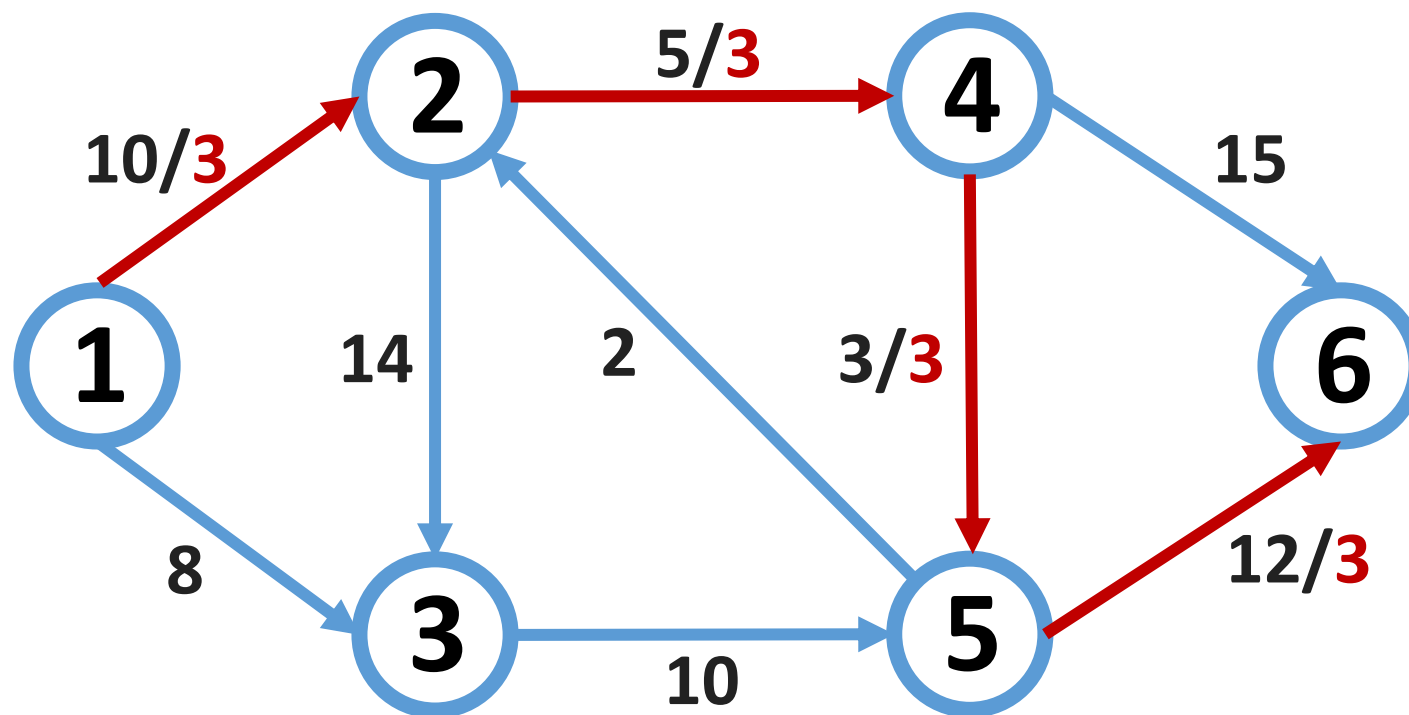
Суммарный поток: $6+24+57+16+13+3+1+8=128$

Задача 1. Найти максимальный поток

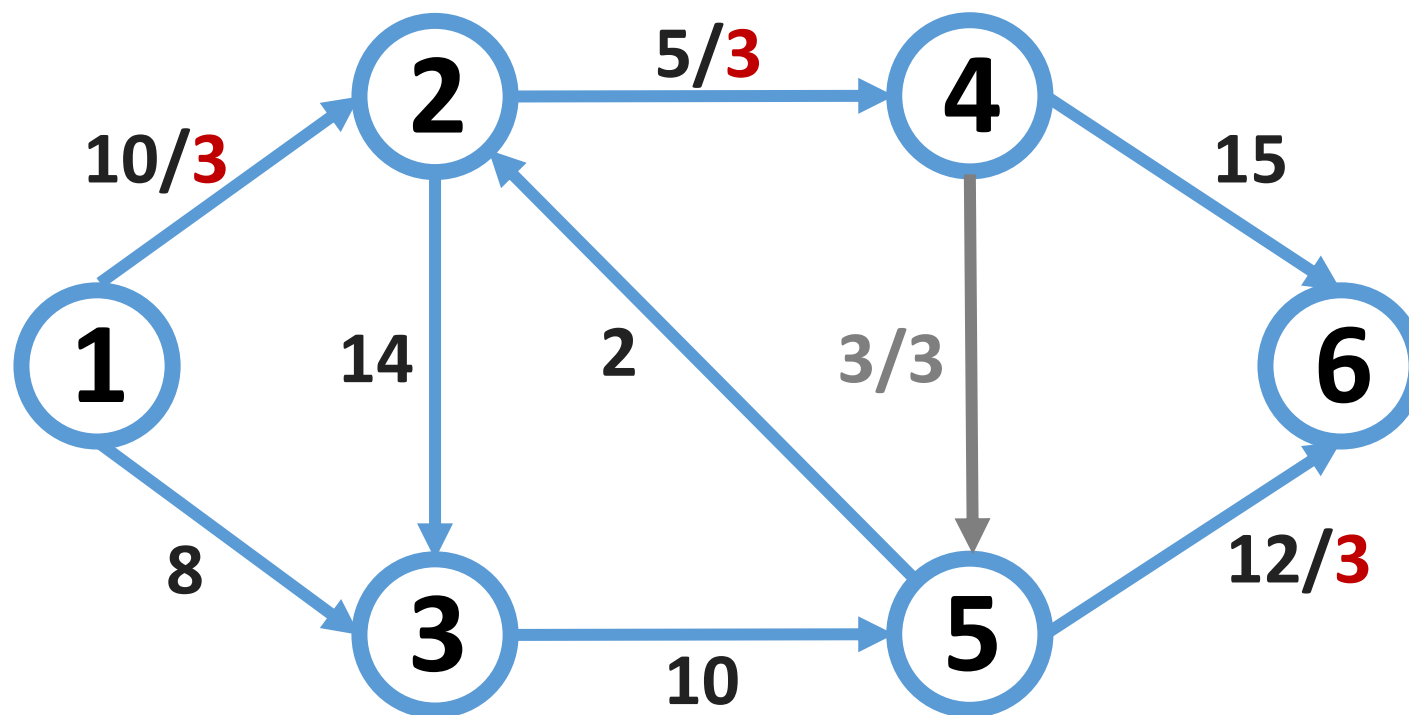




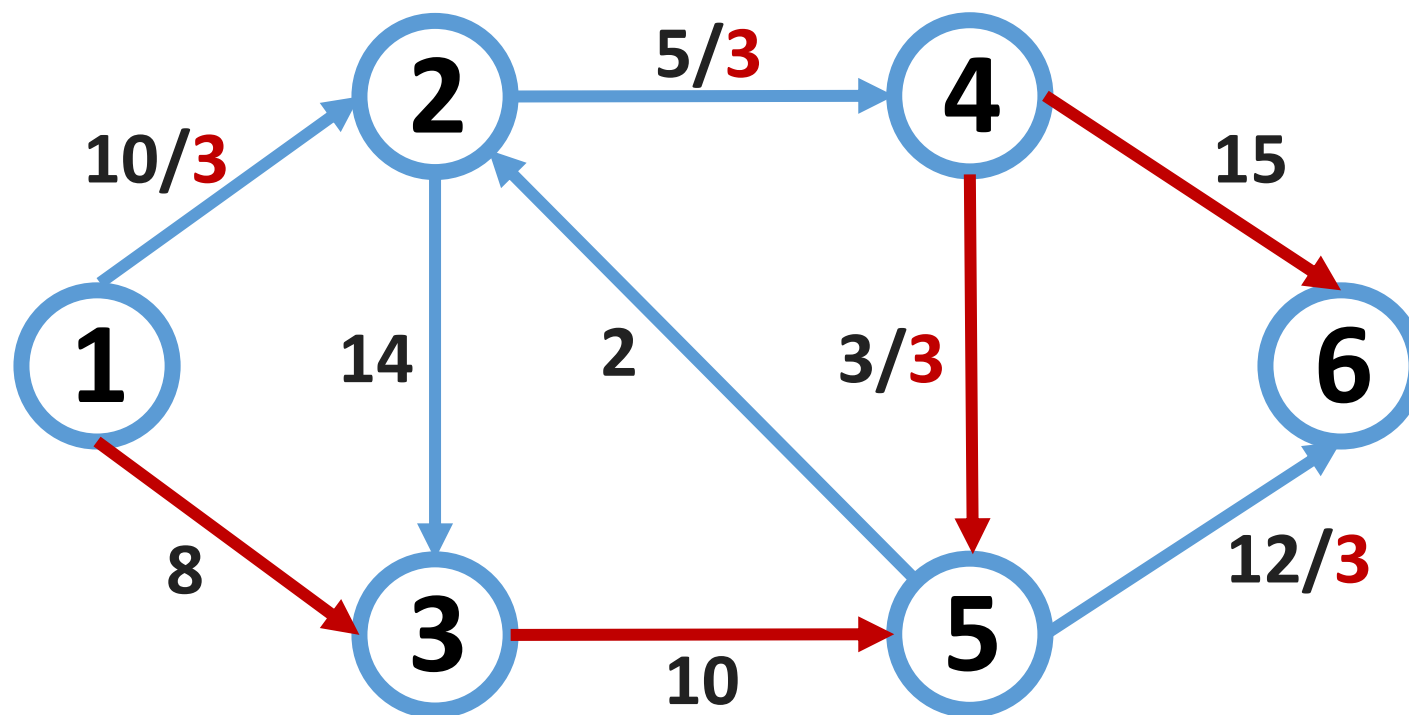
1-2-4-5-6



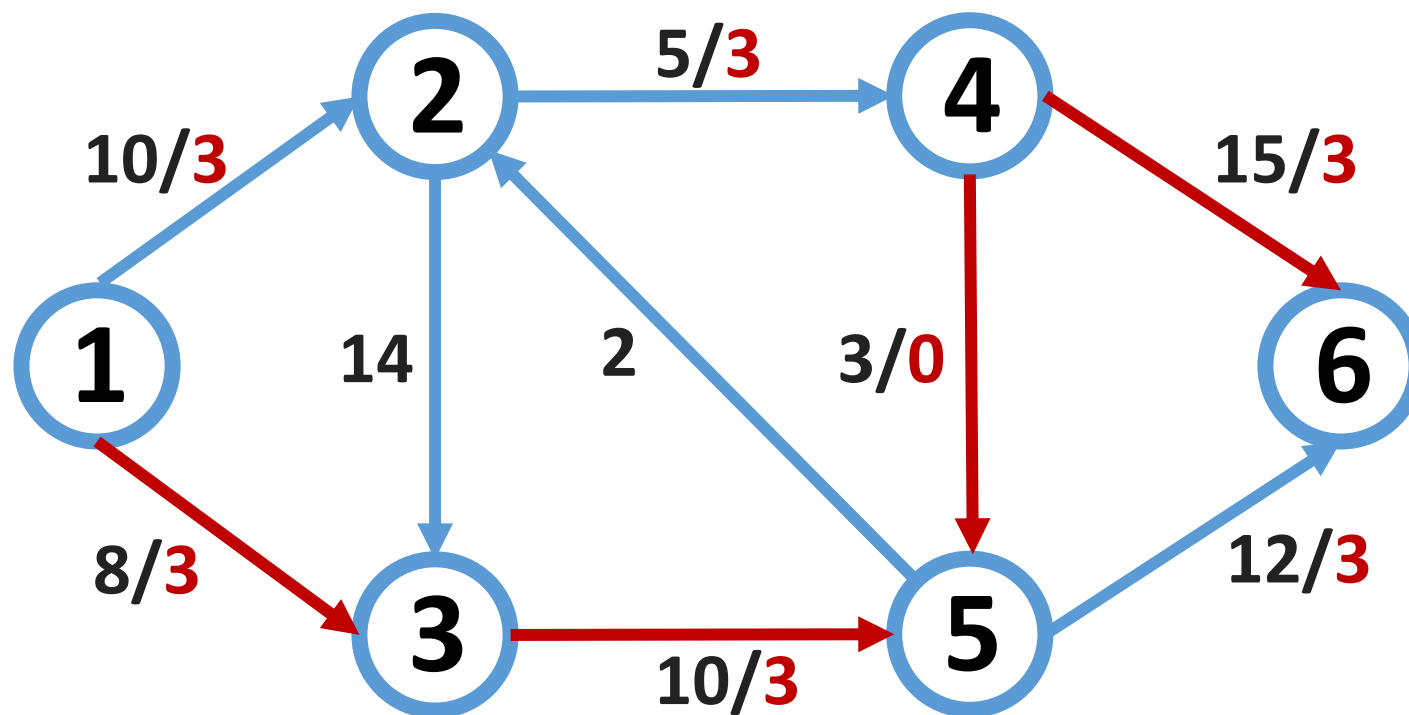
1-2-4-5-6



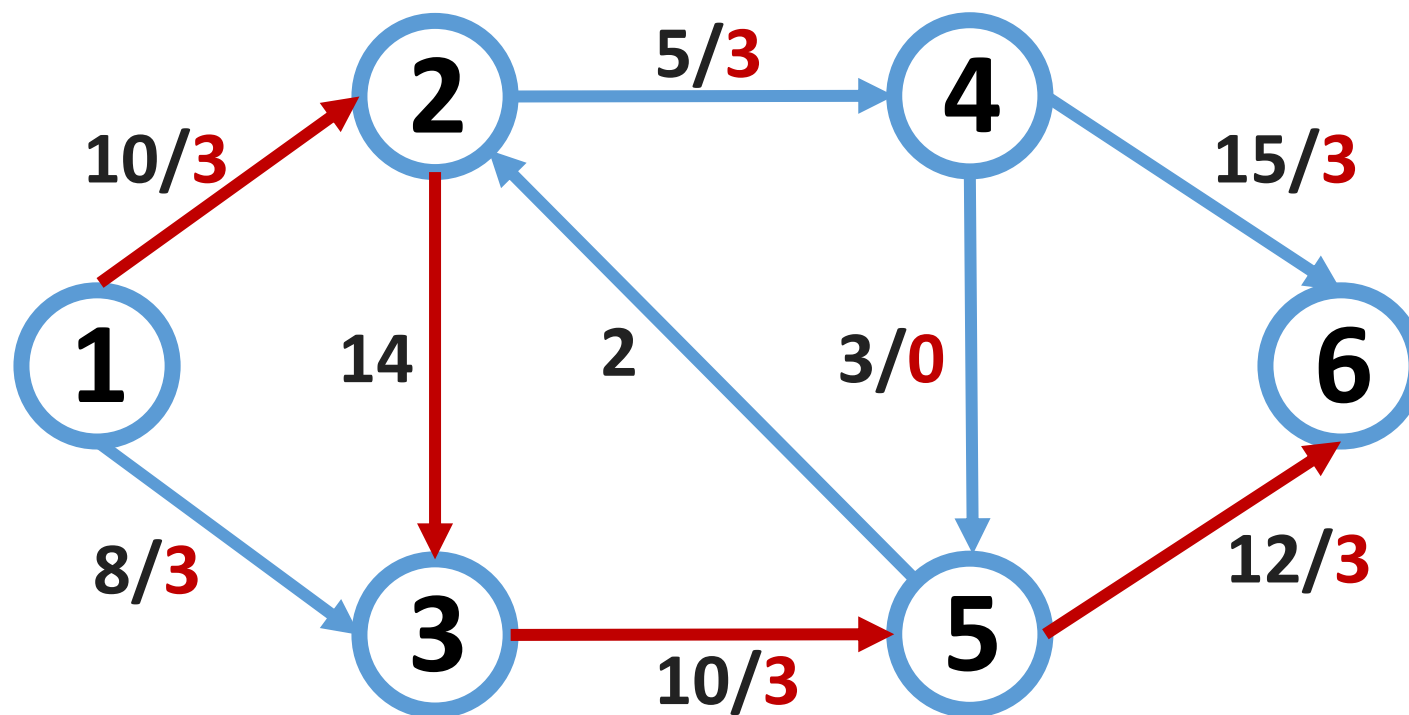
1-2-4-5-6



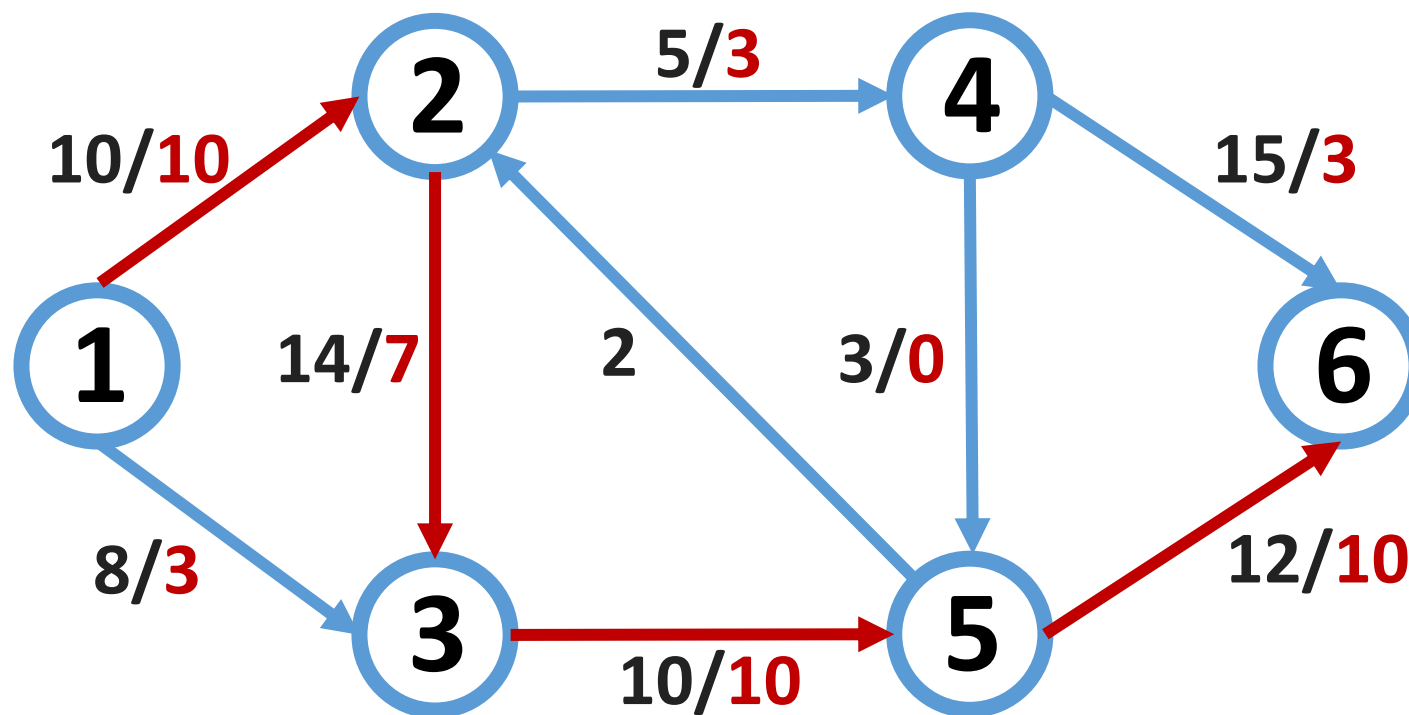
1-3-5-4-6



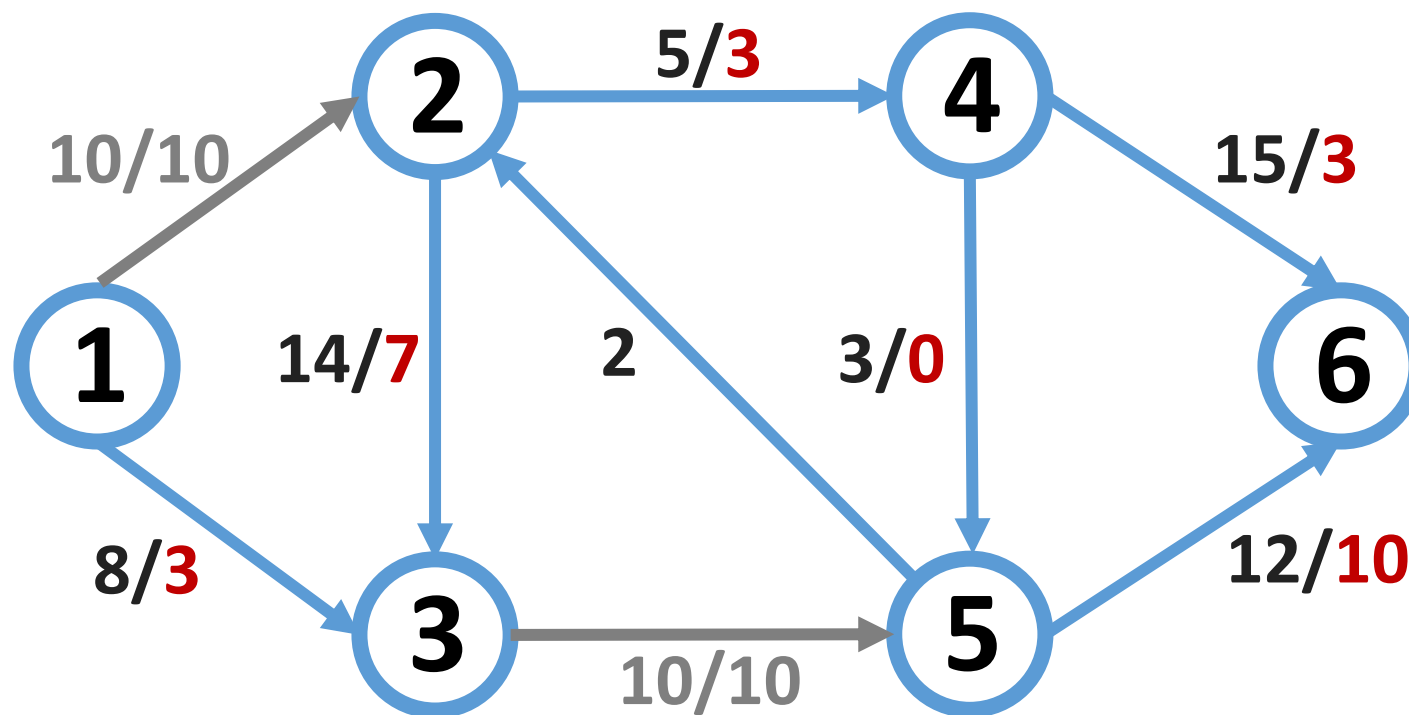
1-3-5-4-6



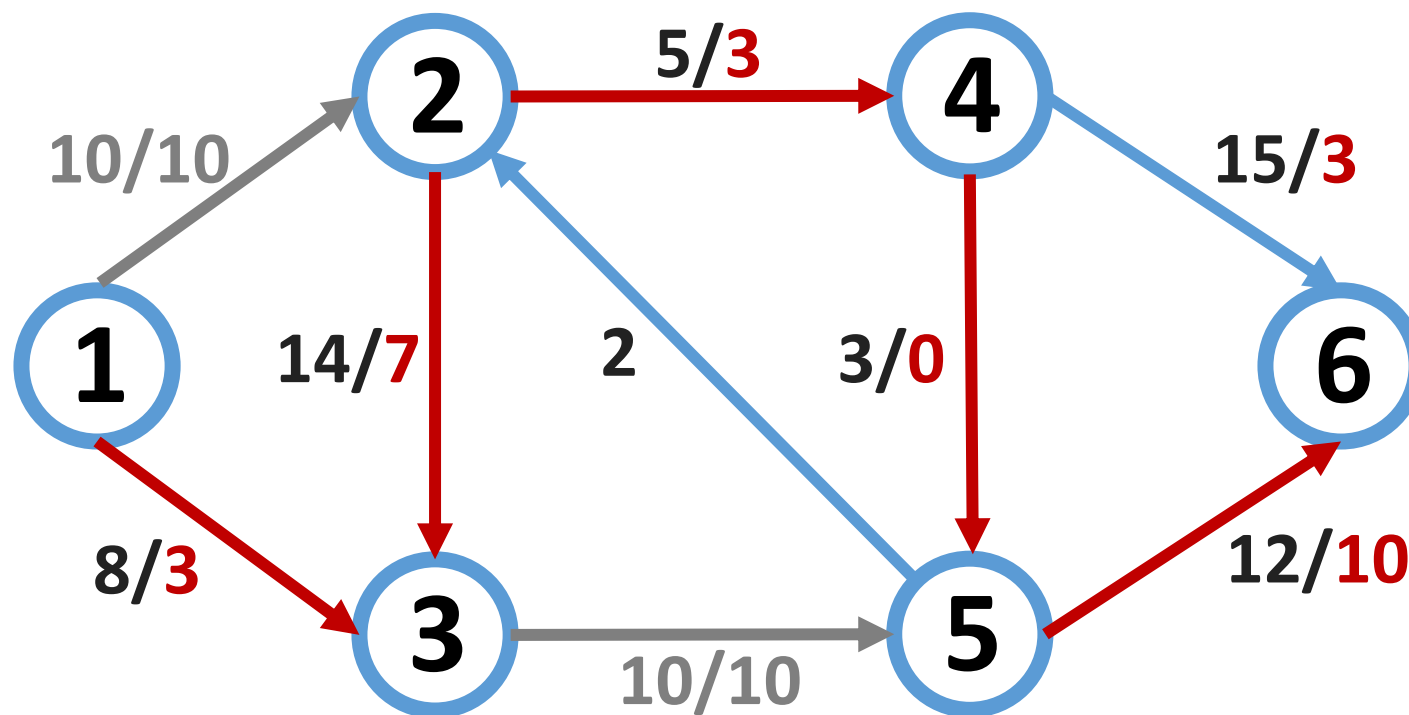
1-2-3-5-6



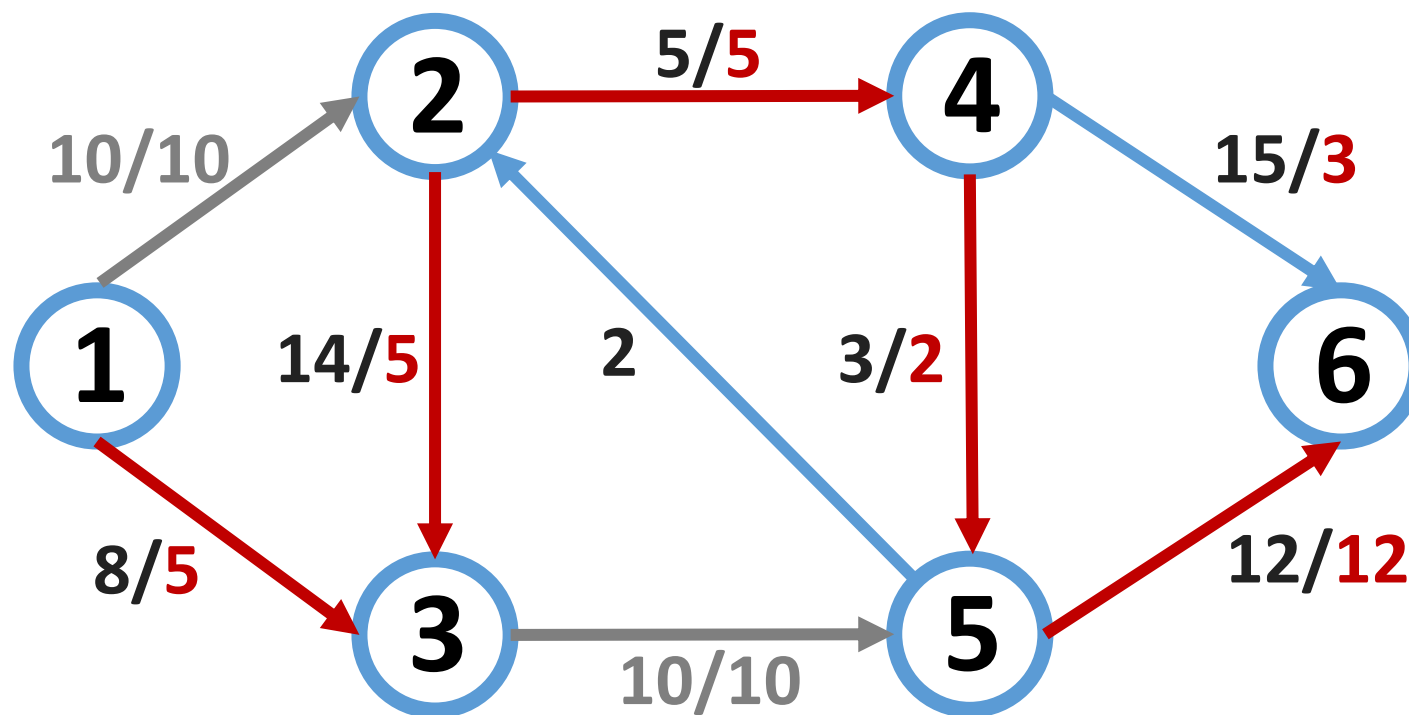
1-2-3-5-6



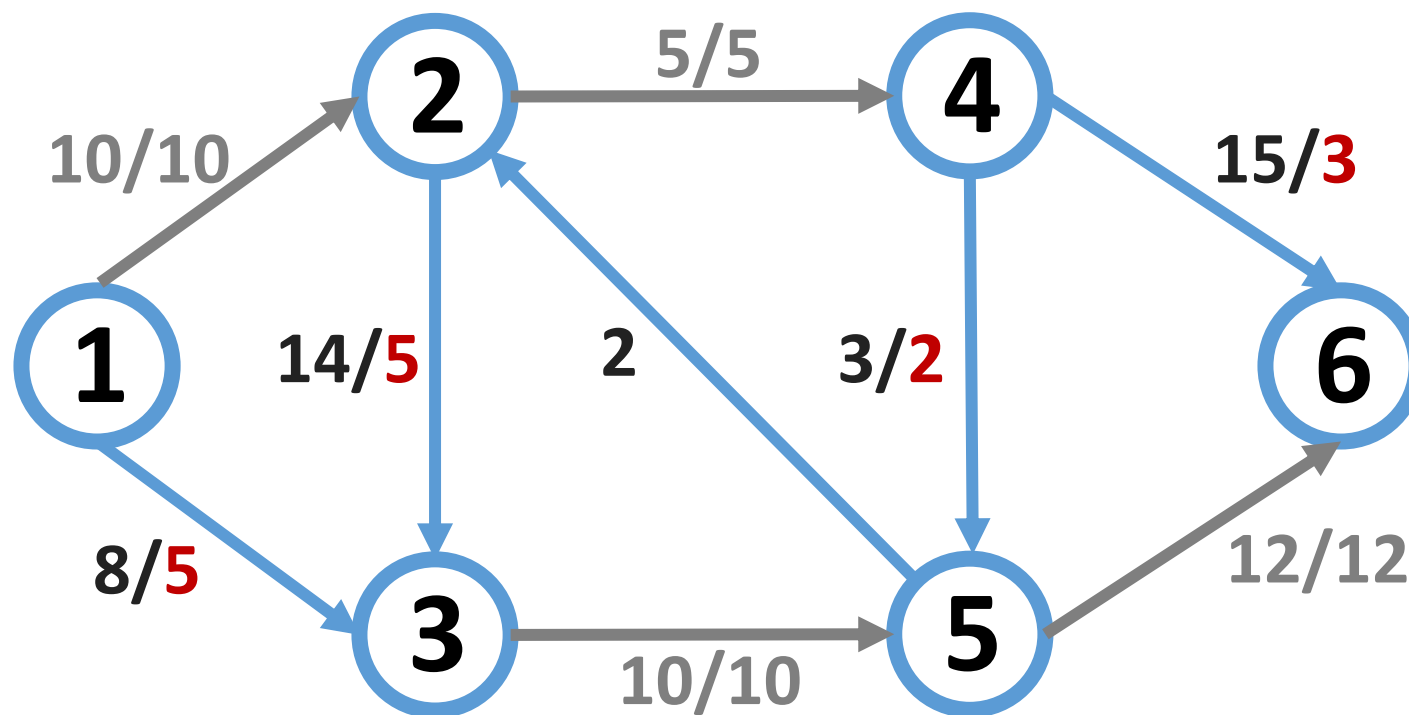
1-2-3-5-6



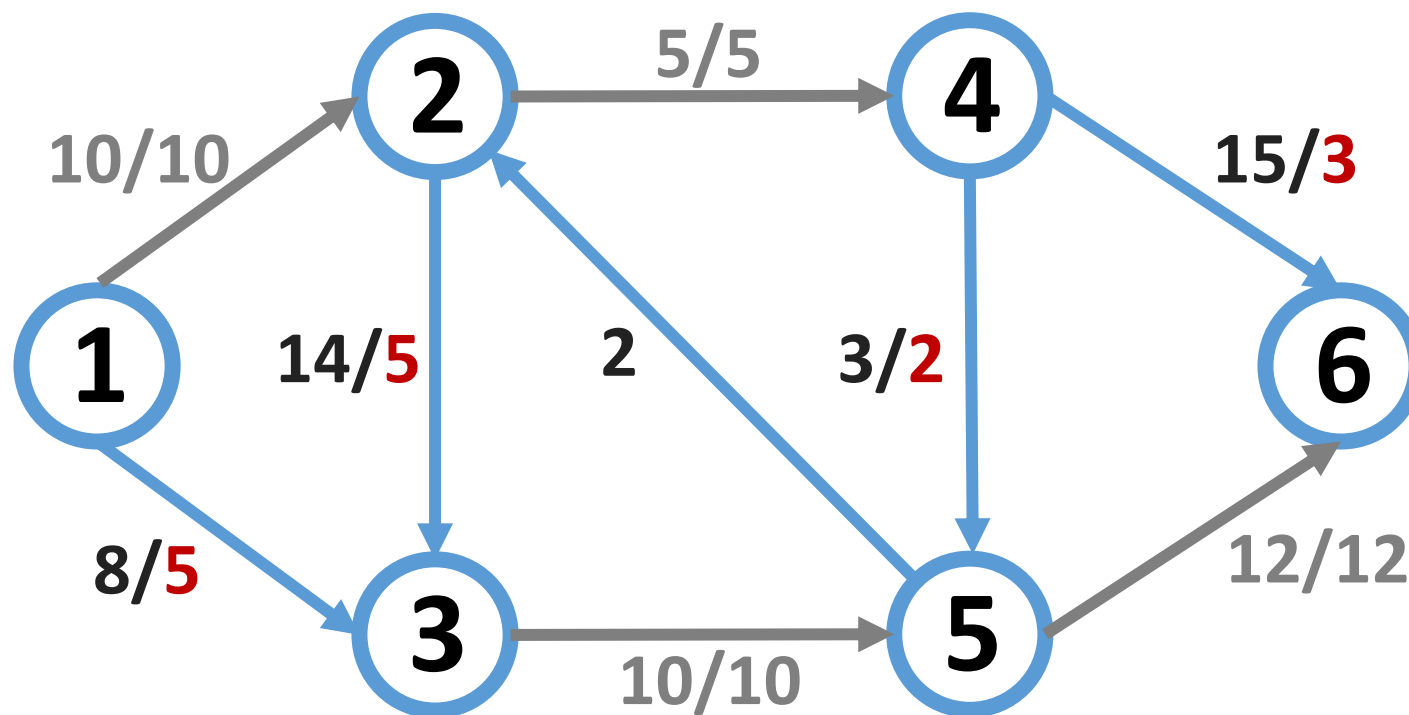
1-3-2-4-5-6



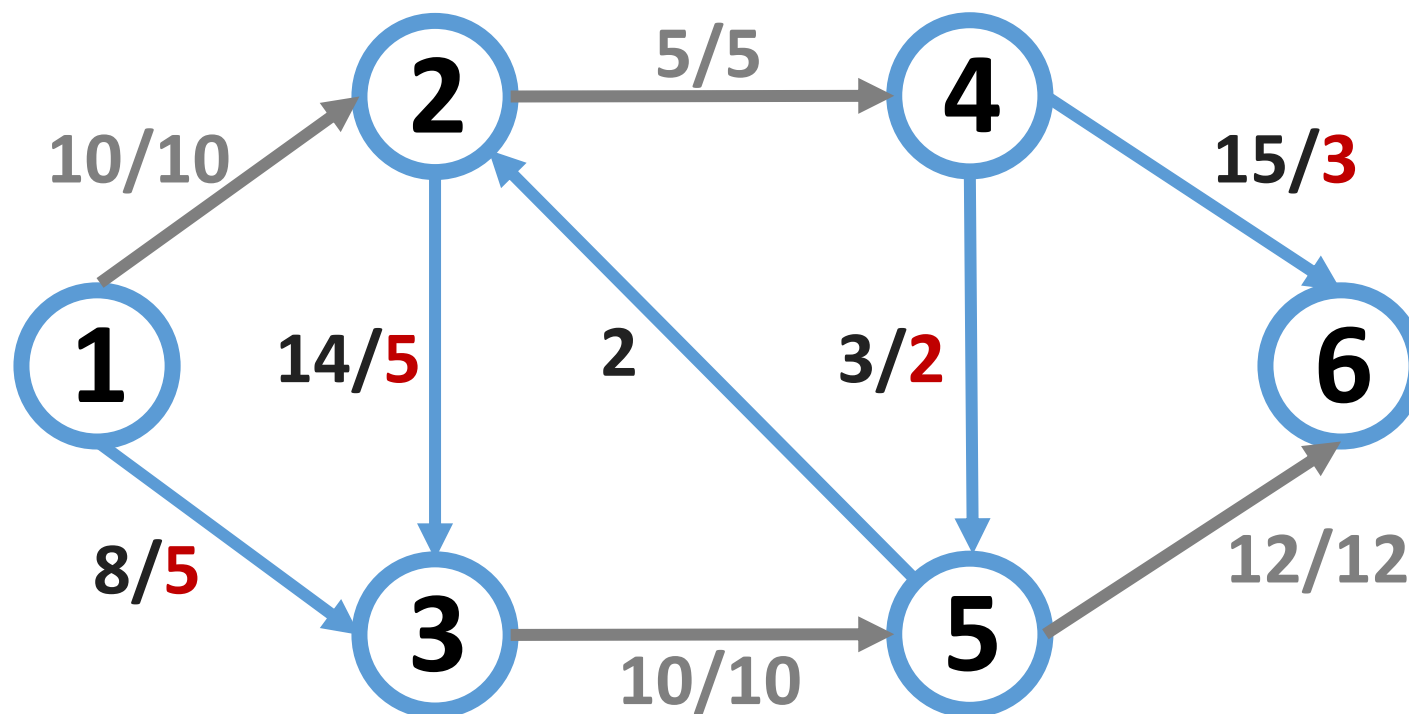
1-3-2-4-5-6



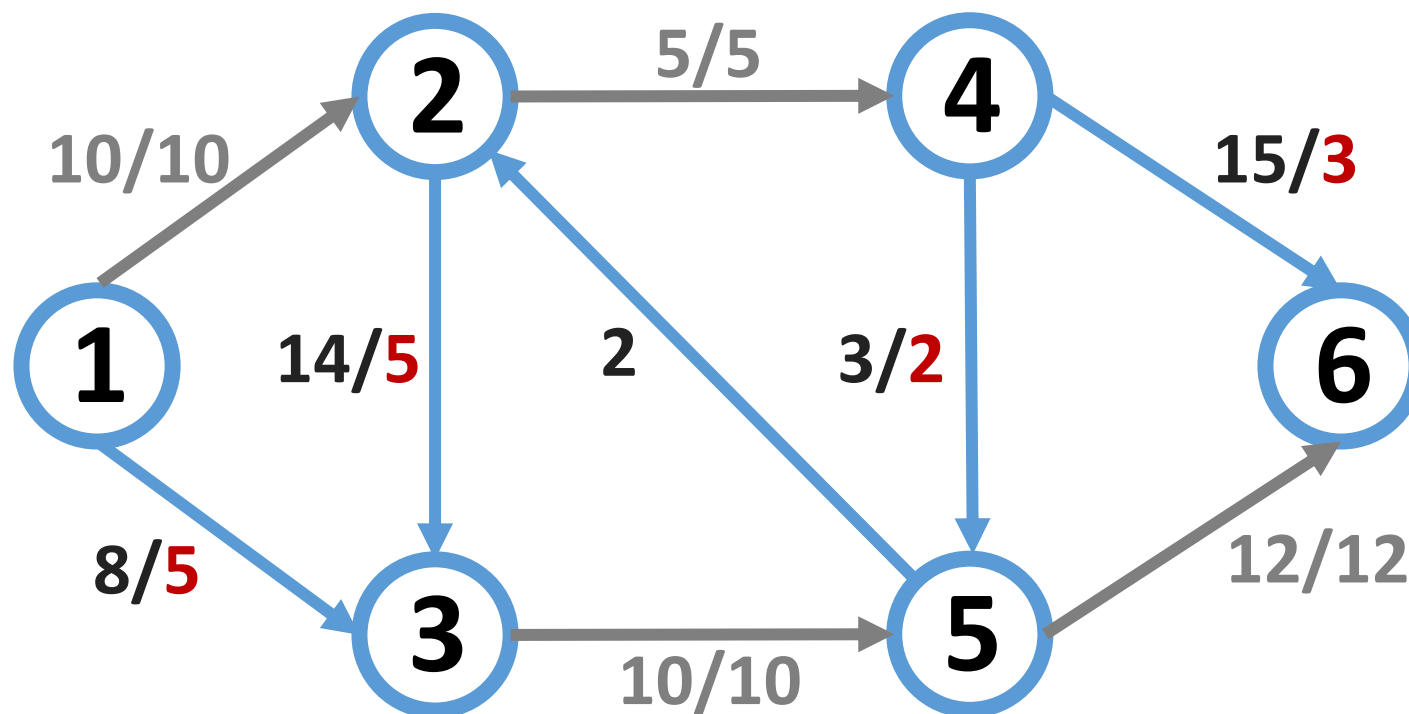
1-3-2-4-5-6



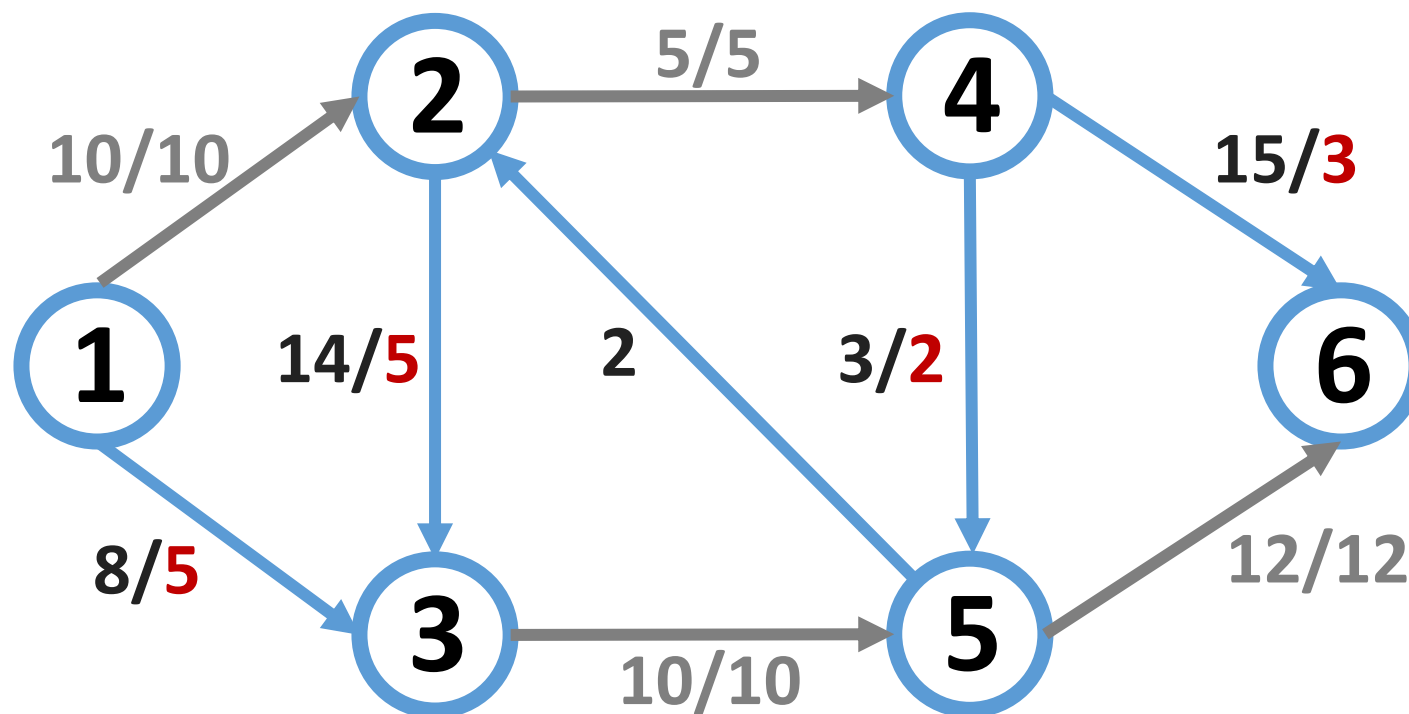
Путей нет!



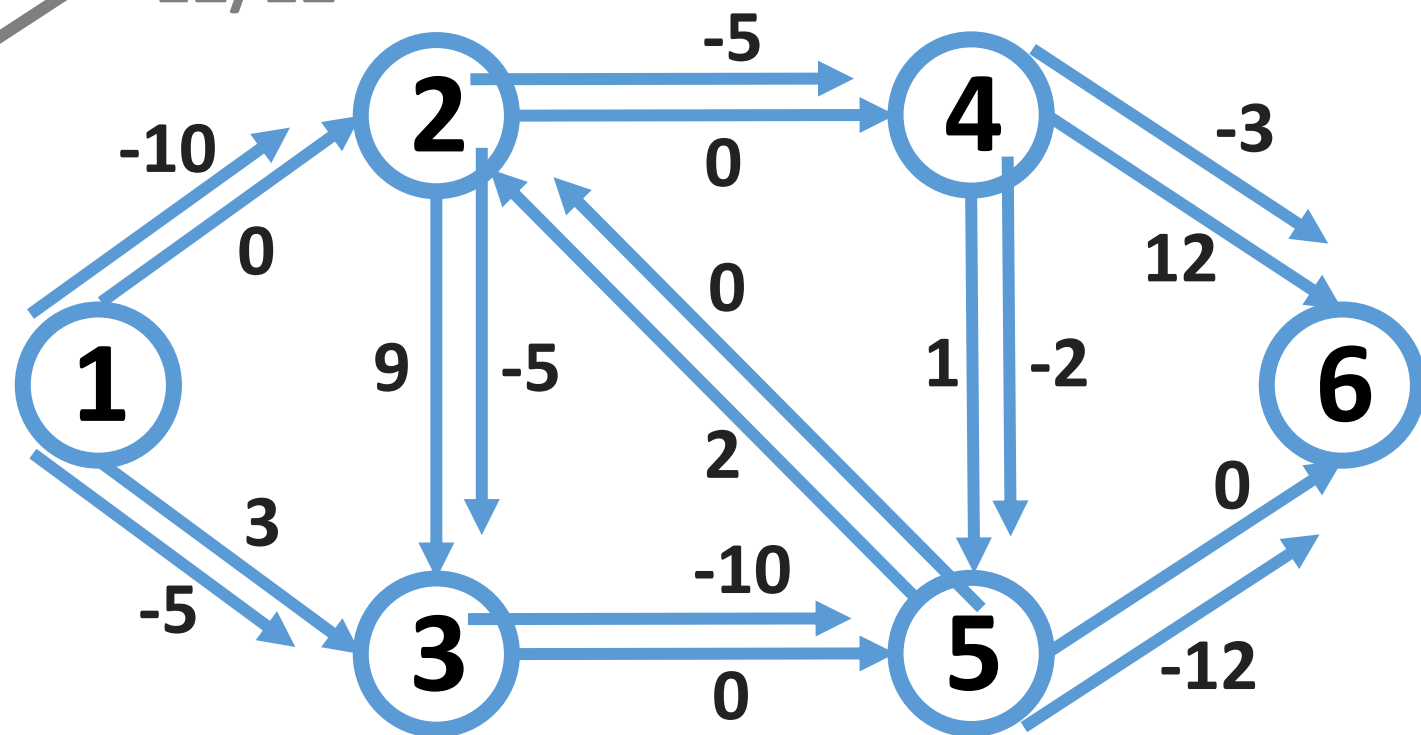
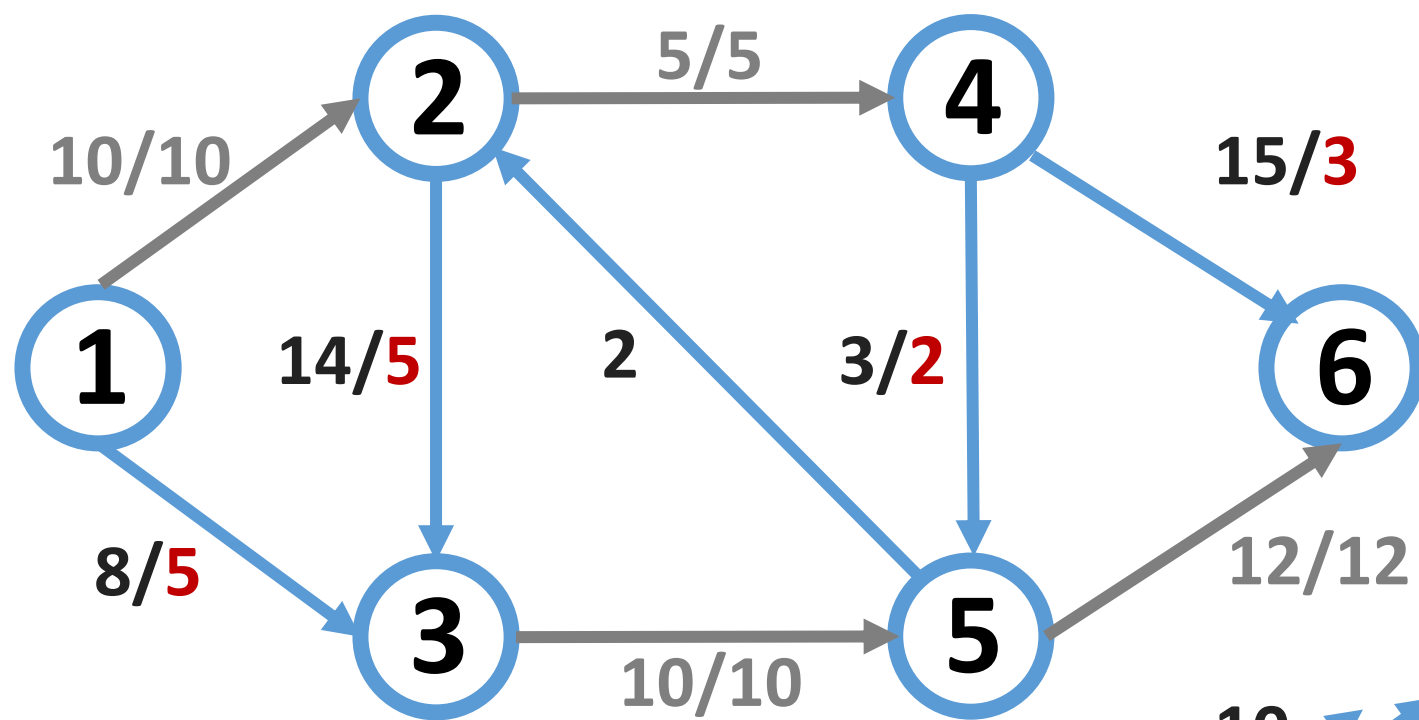
Суммарный поток: $10+5=15$

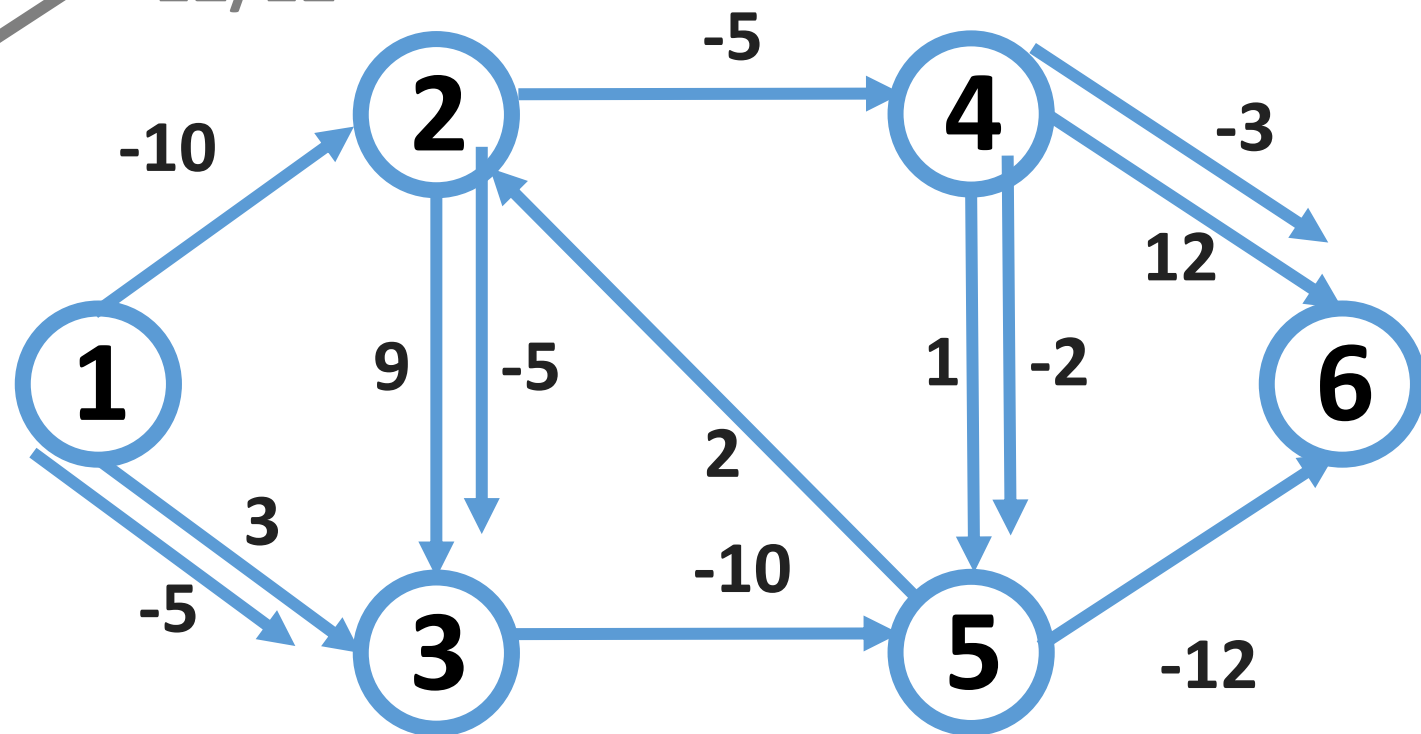
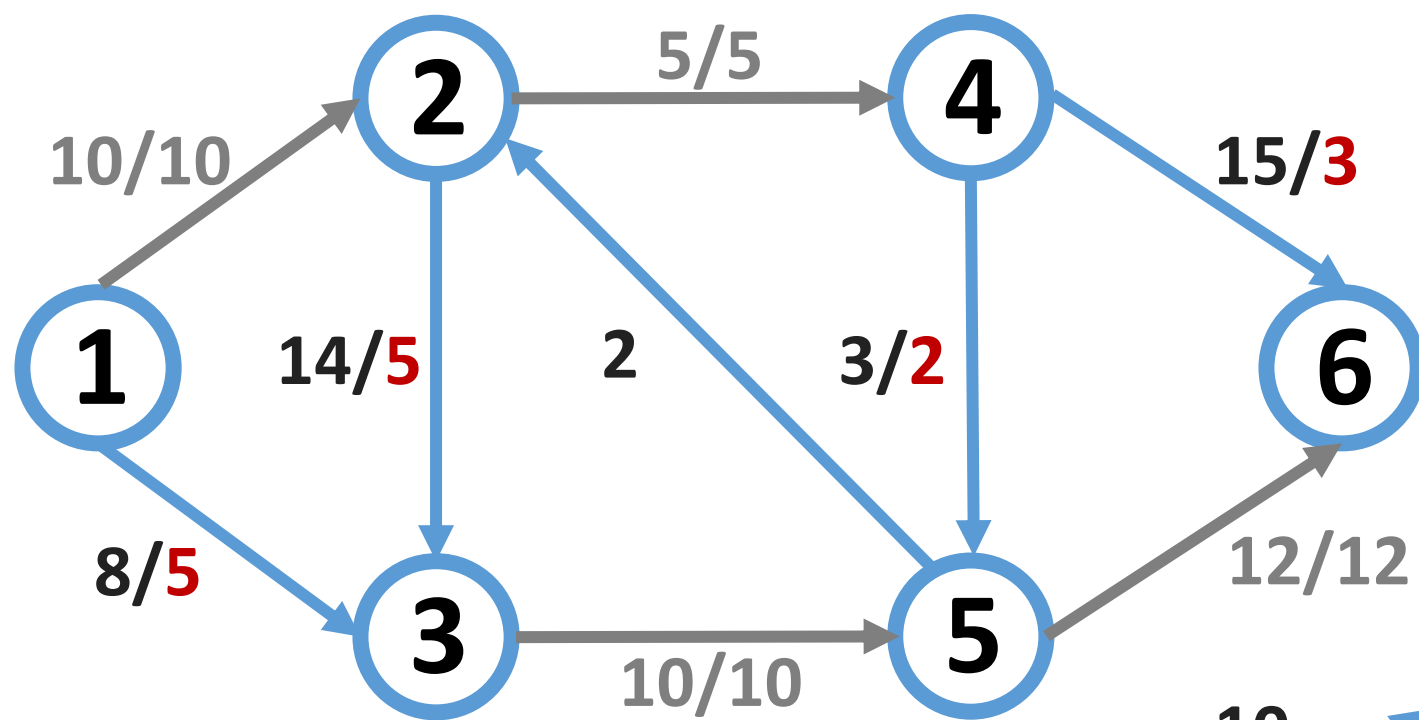


Суммарный поток: $10+5=15$
Суммарный поток: $3+12=15$



Суммарный поток: $10+5=15$
 Суммарный поток: $3+12=15$
 Суммарный поток: $3+3+7+2=15$





Максимальный поток. Алгоритм Форда-Фалкерсона за $O(V^2 * \text{Поток})$.

```
public class MaxFlowFordFulkerson {
    public static int maxFlow(int[][] cap, int s, int t) {
        for (int flow = 0;;) {
            int df = findPath(cap, new boolean[cap.length], s, t, Integer.MAX_VALUE);
            if (df == 0)
                return flow;
            flow += df;
        }
    }

    static int findPath(int[][] cap, boolean[] vis, int u, int t, int f) {
        if (u == t)
            return f;
        vis[u] = true;
        for (int v = 0; v < vis.length; v++)
            if (!vis[v] && cap[u][v] > 0) {
                int df = findPath(cap, vis, v, t, Math.min(f, cap[u][v]));
                if (df > 0) {
                    cap[u][v] -= df;
                    cap[v][u] += df;
                    return df;
                }
            }
        return 0;
    }

    // Usage example
    public static void main(String[] args) {
        int[][] capacity = { { 0, 3, 2 }, { 0, 0, 2 }, { 0, 0, 0 } };
        System.out.println(4 == maxFlow(capacity, 0, 2));
    }
}
```

Алгоритмы нахождения максимального потока

- Алгоритм Форда-Фалкерсона
- Алгоритм Эдмондса-Карпа
- Алгоритм Диница <https://e-maxx.ru/algo/dinic>
- Алгоритм Карзанова
- Алгоритм Малхотри-Кумара-Махешвари
- Алгоритм Галила-Наамада
- Алгоритм Слейтора-Тарьяна
- Алгоритм Голдберга-Тарьяна
- Алгоритм CHM
- Алгоритм Кинга
- Алгоритм Голдберга-Рао

Посмотреть асимптотику!

<http://algotlist.manual.ru/maths/graphs/maxflows/>

Определения

Разрез (s-t cut) - разбиение множества всех вершин V на два подмножества A и B таких, что $s \in A$, $t \in B$, причём пересечение A и B равно пустому множеству.

Поток через разрез (A, B) - сумма всех потоков из A в B
 $\sum_{u \in A} \sum_{v \in B} f(u, v)$. Он не превышает пропускную способность разреза, поскольку $f(u, v) \leq c(u, v)$.

Минимальный разрез - разрез с минимальной пропускной способностью.

Теорема Форда — Фалкерсона

Теорема о максимальном потоке в графе звучит так: величина максимального потока в графе путей равна величине пропускной способности его **минимального разреза**.

Разрезом сети (G, a) с источником s и стоком t называется множество $\{e_1, \dots, e_k\}$ дуг сети, обладающее следующими свойствами:

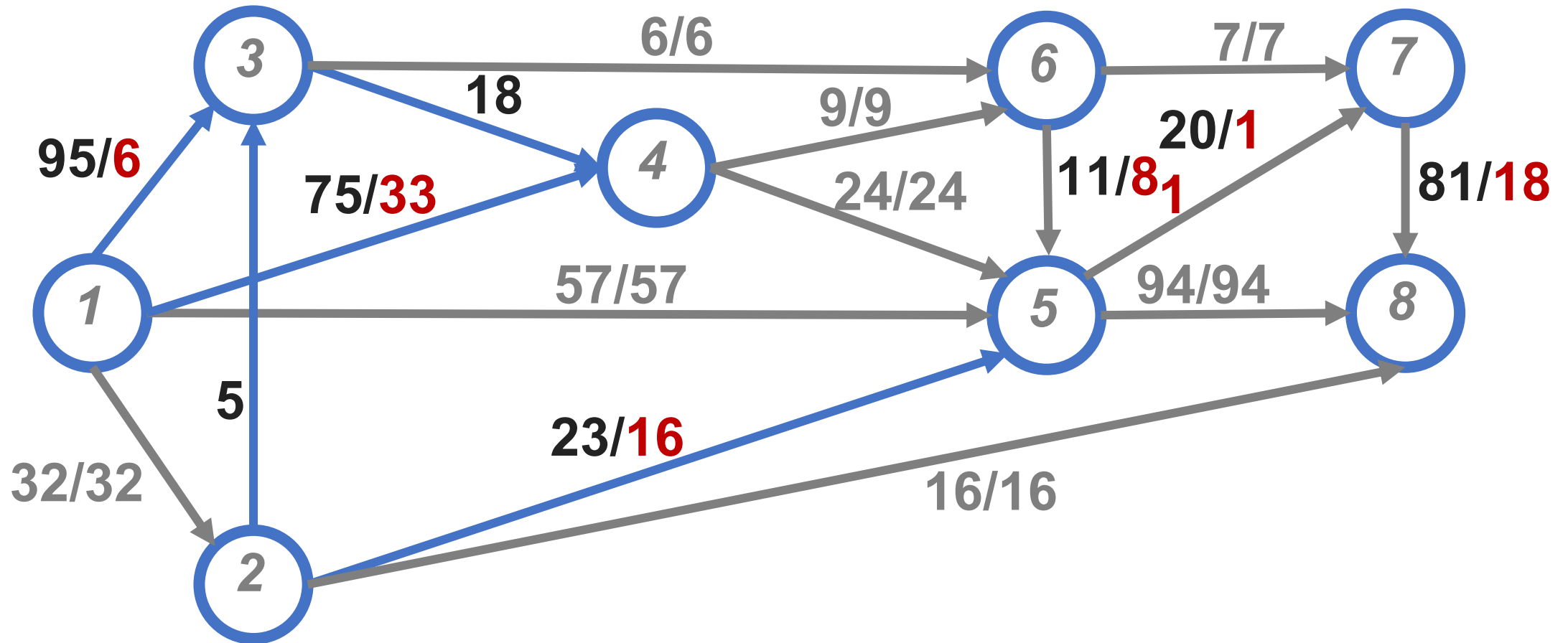
- в орграфе $G - \{e_1, \dots, e_k\}$ нет (s, t) -цепей;
- если любую из дуг $e_1, \dots, e_k$ добавить к орграфу $G - \{e_1, \dots, e_k\}$, то в полученном орграфе найдётся (s, t) -цепь.

Пропускной способностью разреза называется сумма пропускных способностей дуг, входящих в разрез. Разрез называется **минимальным**, если его пропускная способность минимально среди пропускных способностей всех разрезов сети.

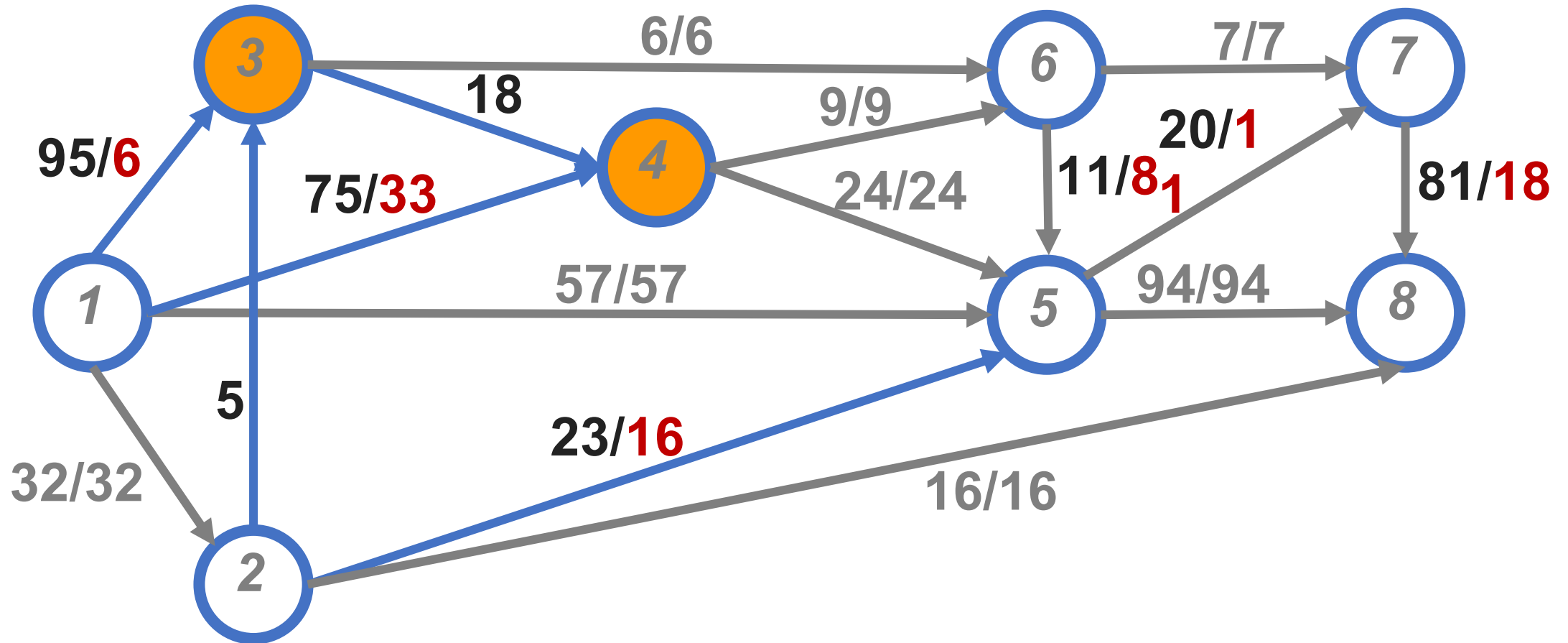
Как найти минимальный разрез?

Если для сети найден максимальный поток, то минимальный разрез можно найти следующим образом: включить в множество V_s вершину s и все вершины $v \in V$, для каждой из которых существует цепь P из s в v с остаточной пропускной способностью $h(P) > 0$; положить $V_t = V \setminus V_s$. Полученный таким образом разрез (V_s, V_t) будет минимальным.

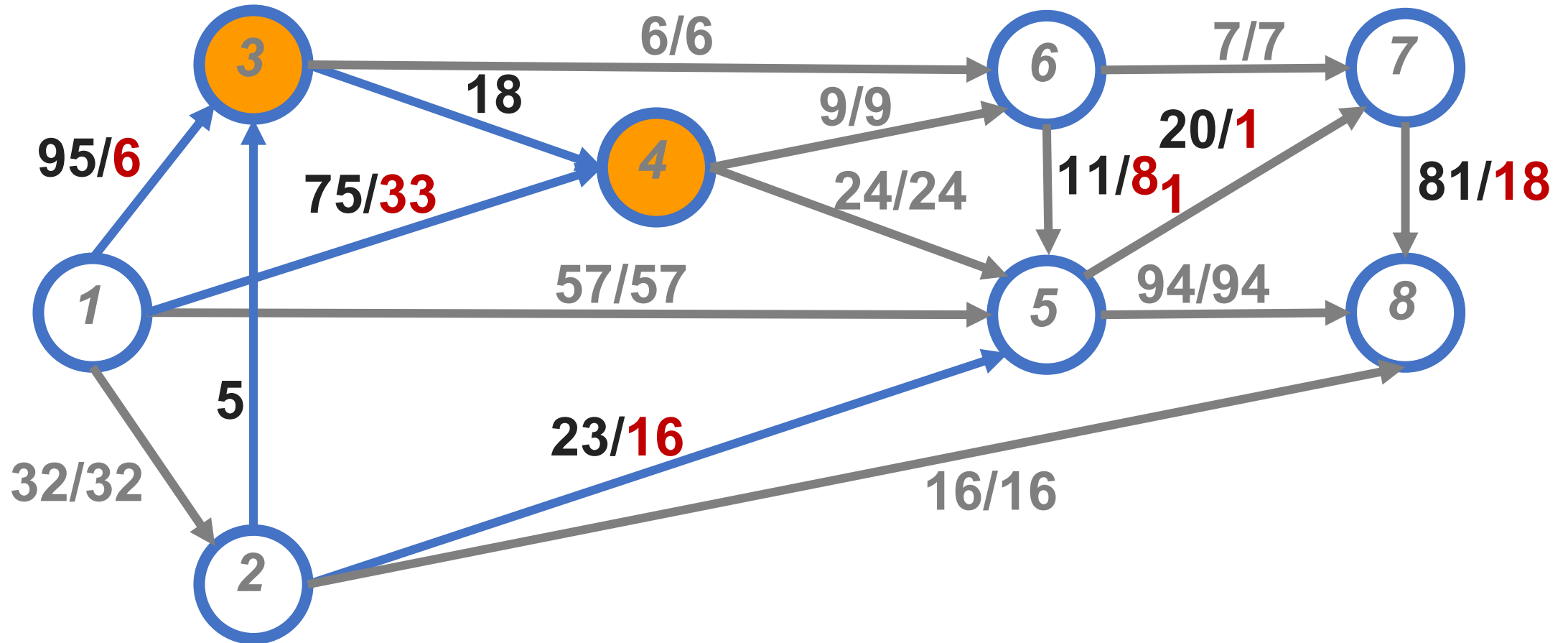
Найти минимальный разрез



Найти минимальный разрез

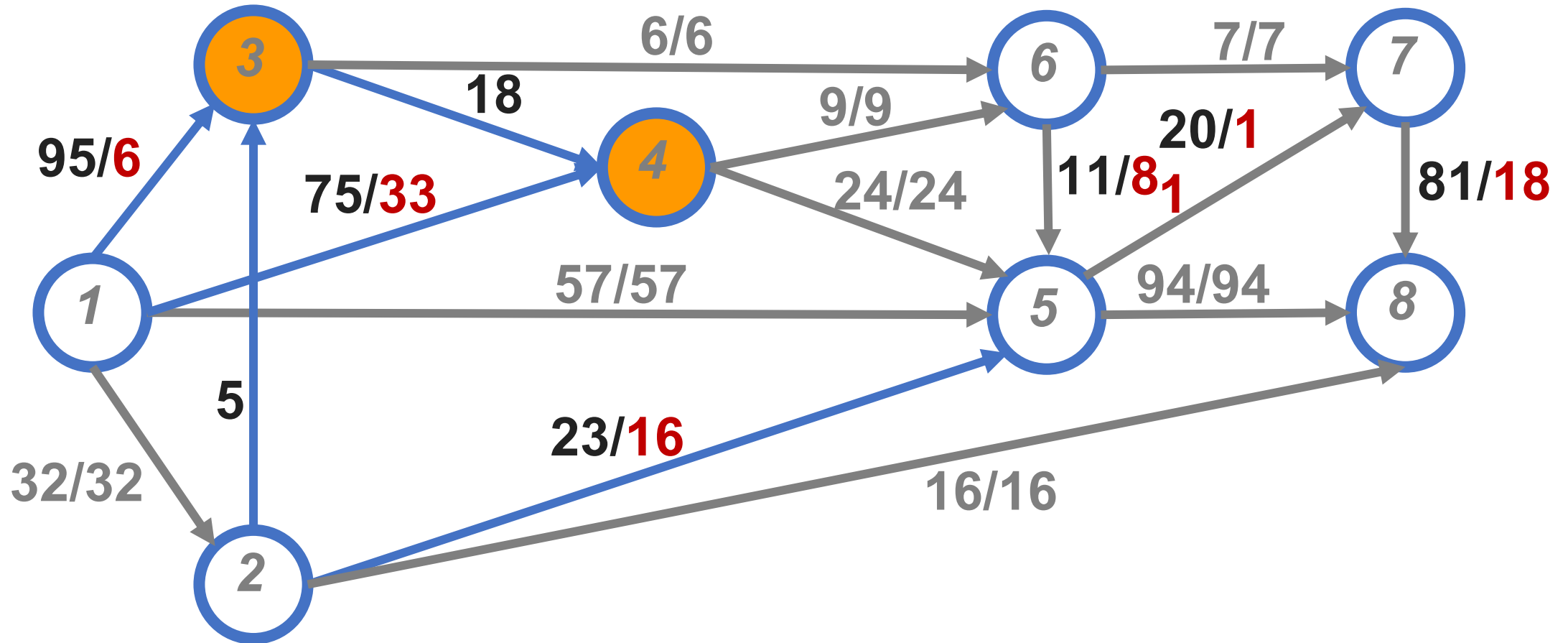


Найти минимальный разрез



$V_S = \{1, 3, 4\}$

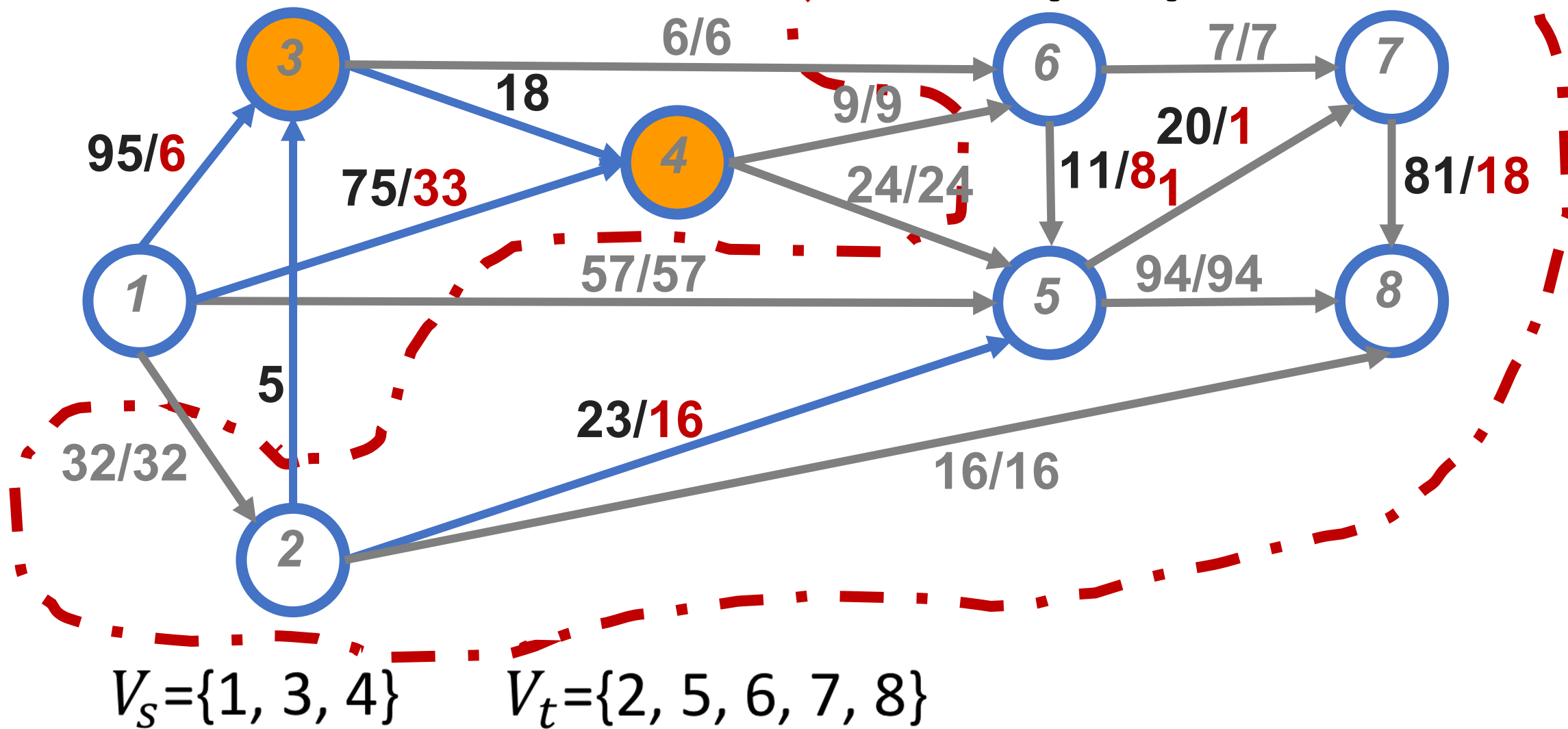
Найти минимальный разрез



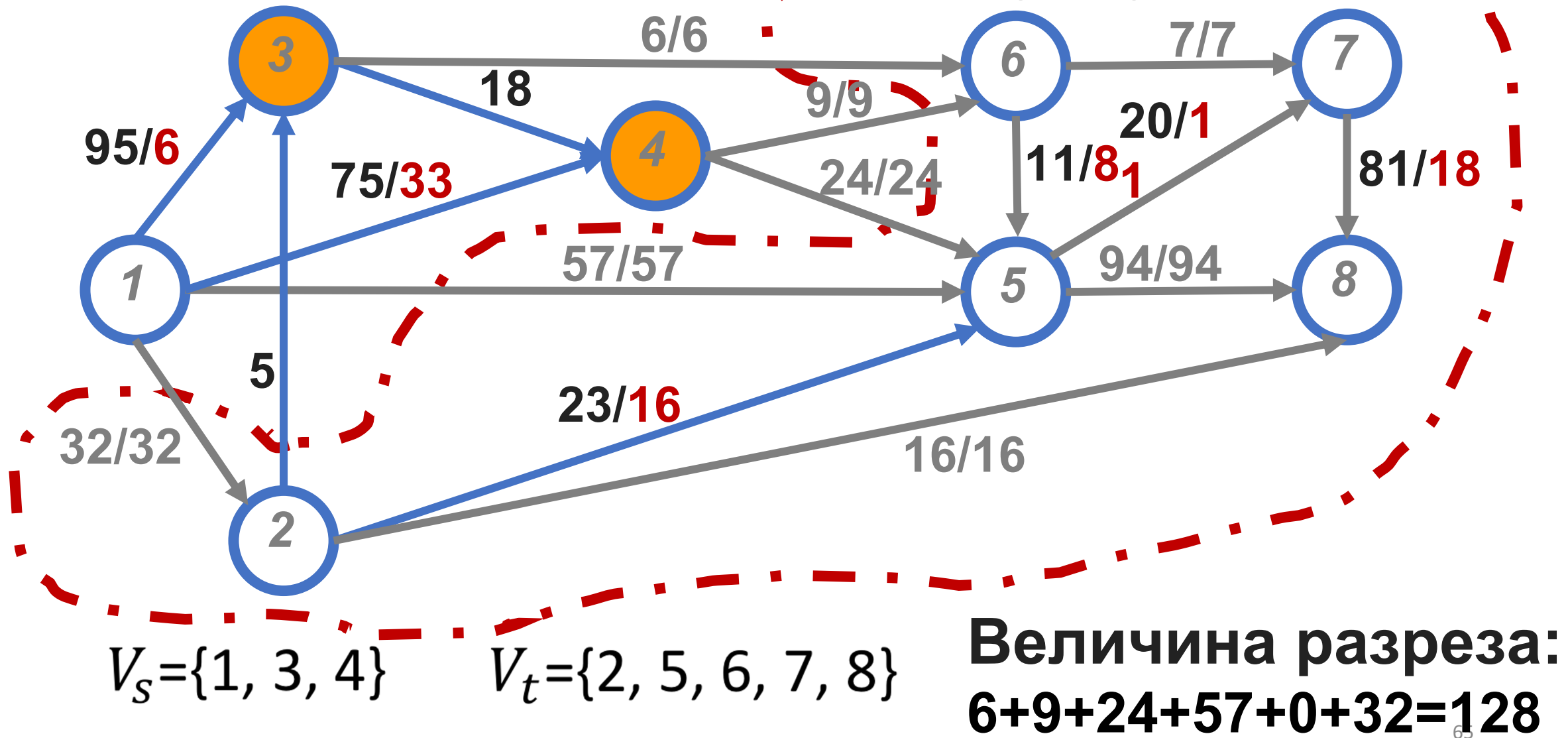
$$V_s = \{1, 3, 4\}$$

$$V_t = \{2, 5, 6, 7, 8\}$$

Найти минимальный разрез

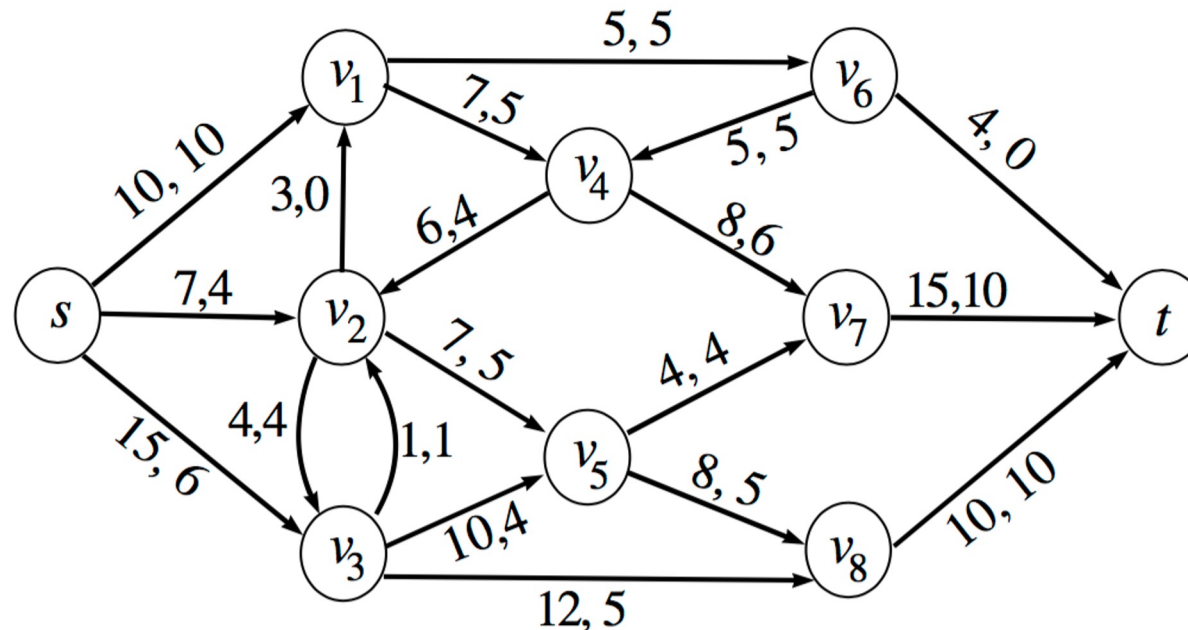


Найти минимальный разрез



Максимальный поток минимальной стоимости

- **Стоимость потока (flow cost)** через ребро в транспортной сети со стоимостями ребер равна произведению потока в этом ребре на стоимость. **Стоимость (cost) потока** есть сумма стоимостей потоков в ребрах этого потока.



	пр. сп.	стои- мость	ПОТОК
0-1	3	3	1
0-2	3	1	3
1-3	2	1	1
1-4	2	1	0
2-3	1	4	1
2-4	2	2	2
3-5	2	2	2
4-5	2	1	2



СТОИМОСТЬ: 21

ПОТОК

2
2
2
0
0
2
2
2



СТОИМОСТЬ: 20

ПОТОК

2
2
1
1
1
1
2
2



СТОИМОСТЬ: 22

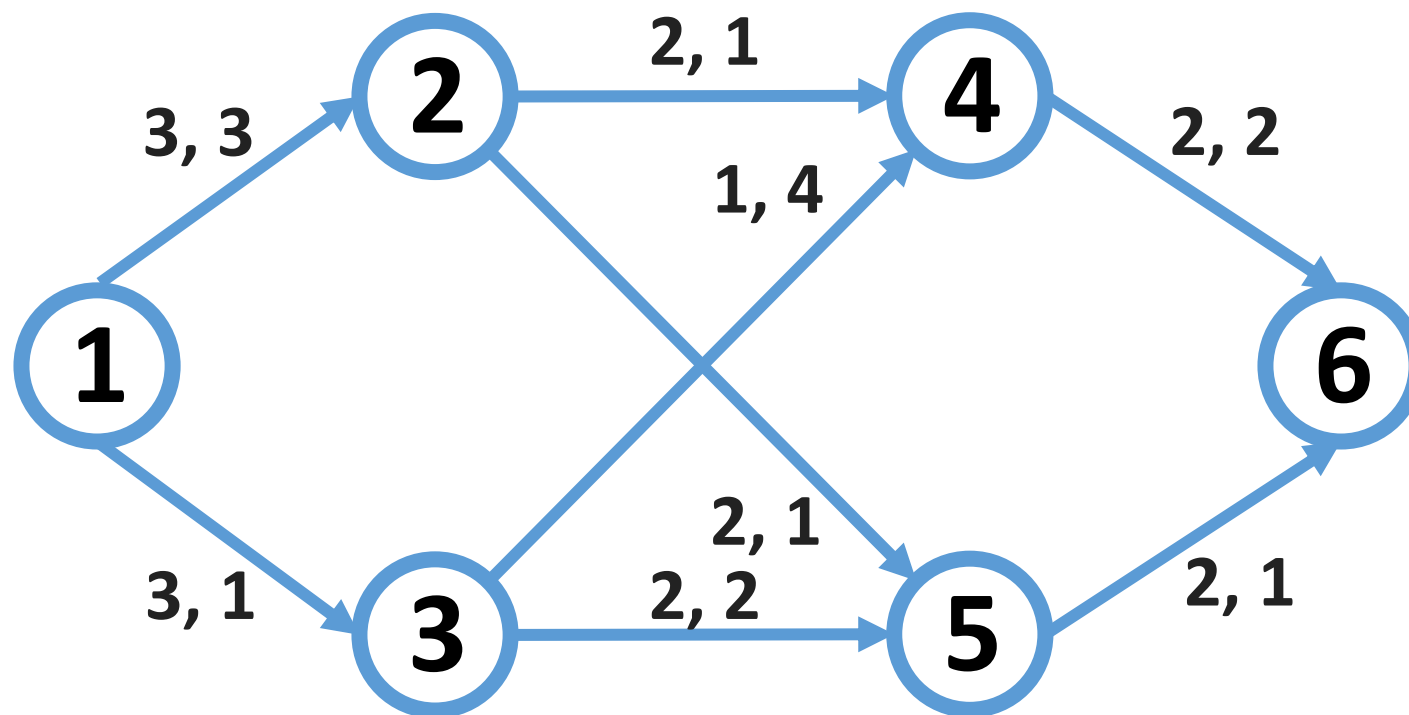
Теорема

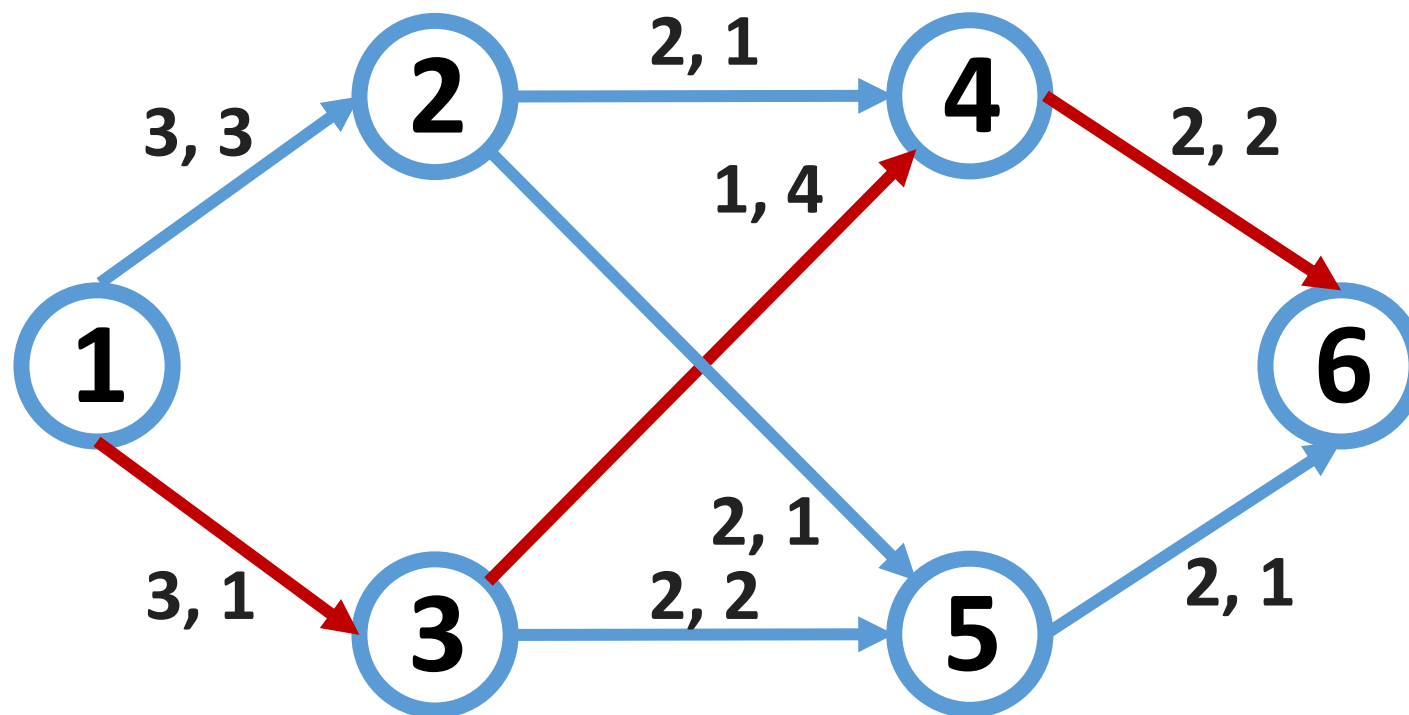
- Максимальный поток является максимальным потоком минимальной стоимости тогда и только тогда, когда его остаточная сеть **не содержит** (ориентированный) цикл с отрицательной стоимостью.

Максимальный поток минимальной стоимости

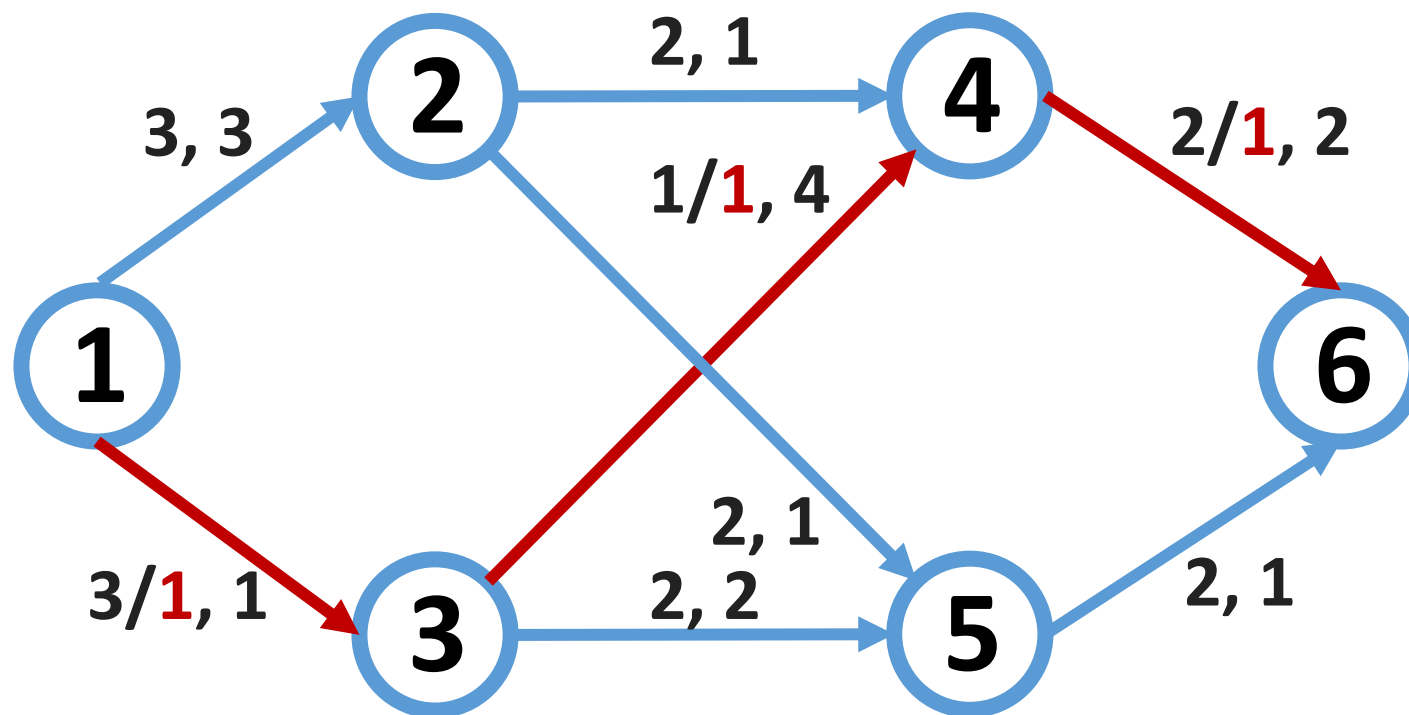
- **Шаг 1.** Требуется найти максимальный поток минимальной стоимости.
- **Шаг 2.** Для каждого ребра зададим поток равный 0.
- **Шаг 3.** Построим остаточную сеть
- **Шаг 4.** При помощи алгоритма Белмана-Форда найдем отрицательные циклы в остаточной сети. Если нет — перейдем к **шагу 7**.
- **Шаг 5.** Выберем один из отрицательных циклов.
- **Шаг 6.** Избавимся от отрицательного цикла, для этого пустим по нему максимально возможный поток. Перейдем к **шагу 4**.
- **Шаг 7.** Отрицательных циклов в остаточной сети нет, значит, максимальный поток минимальной стоимости найден.

Найти максимальный поток минимальной стоимости

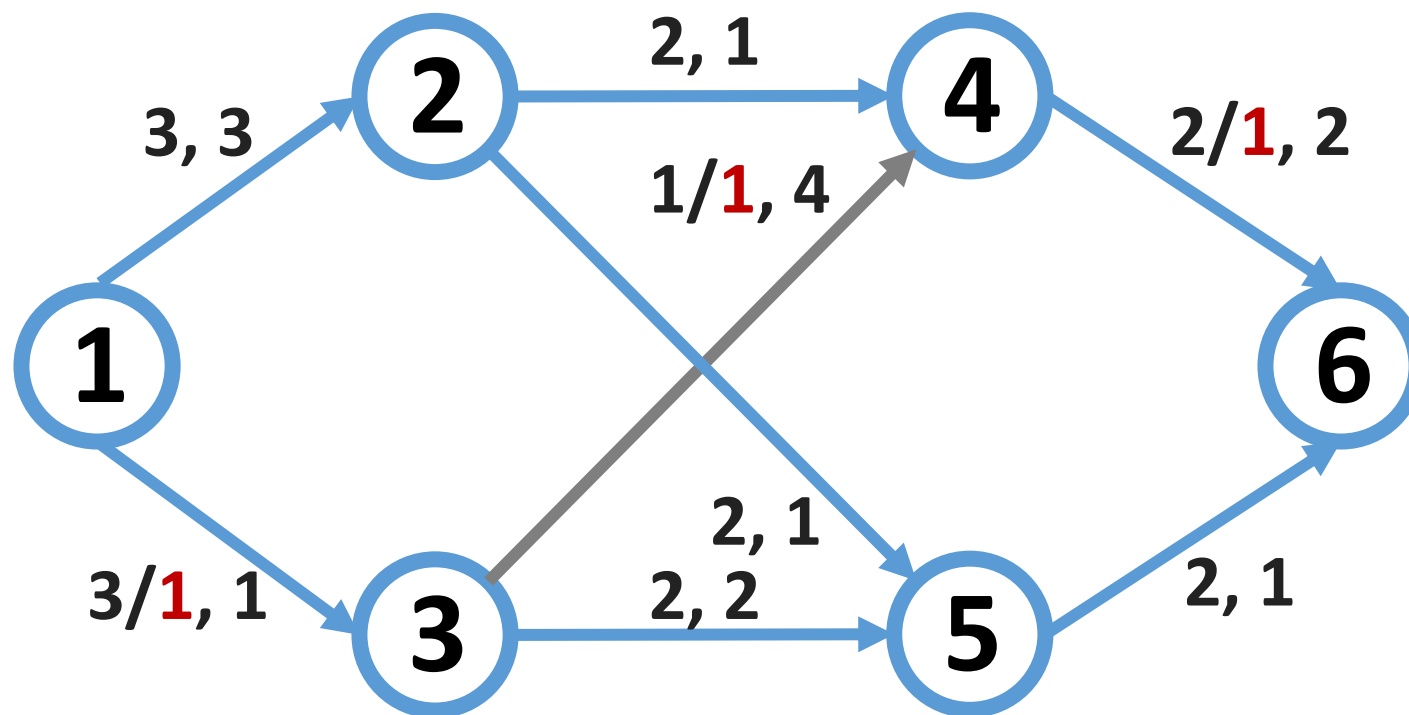




1-3-4-6

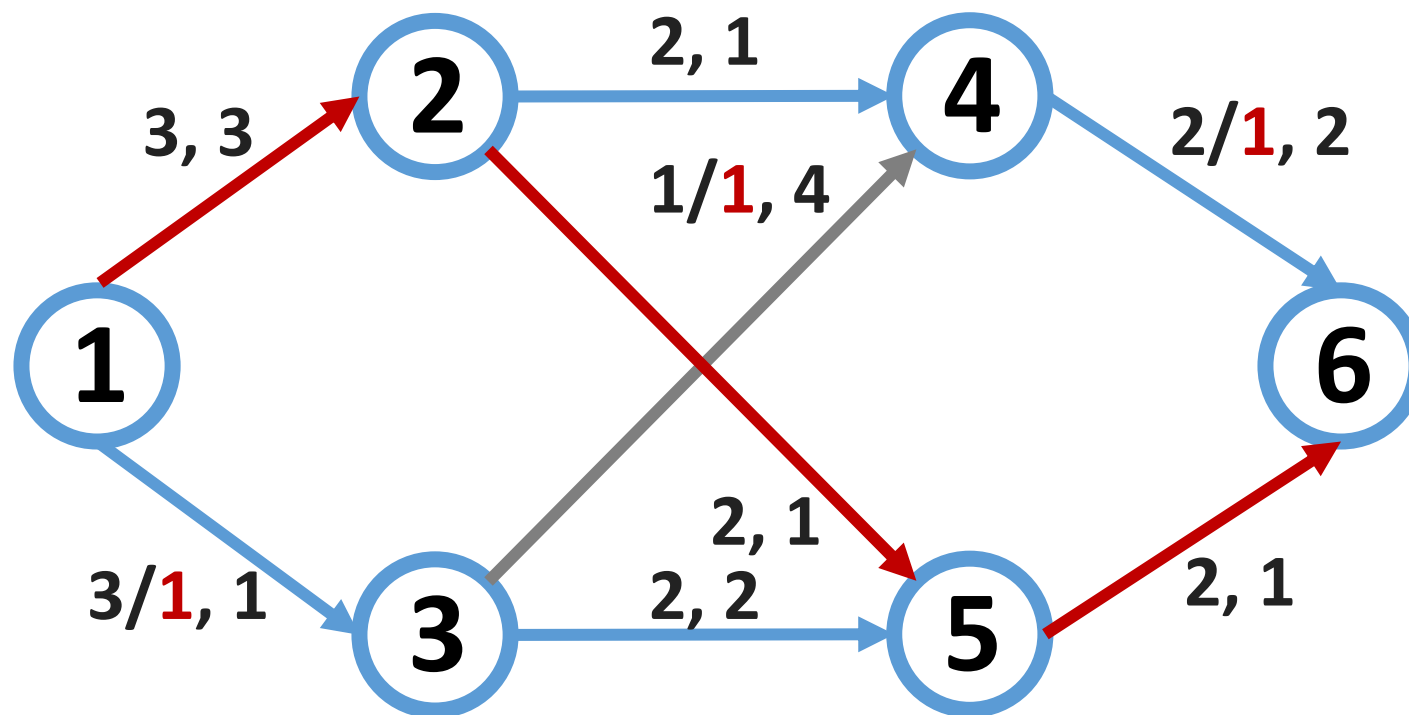


1-3-4-6



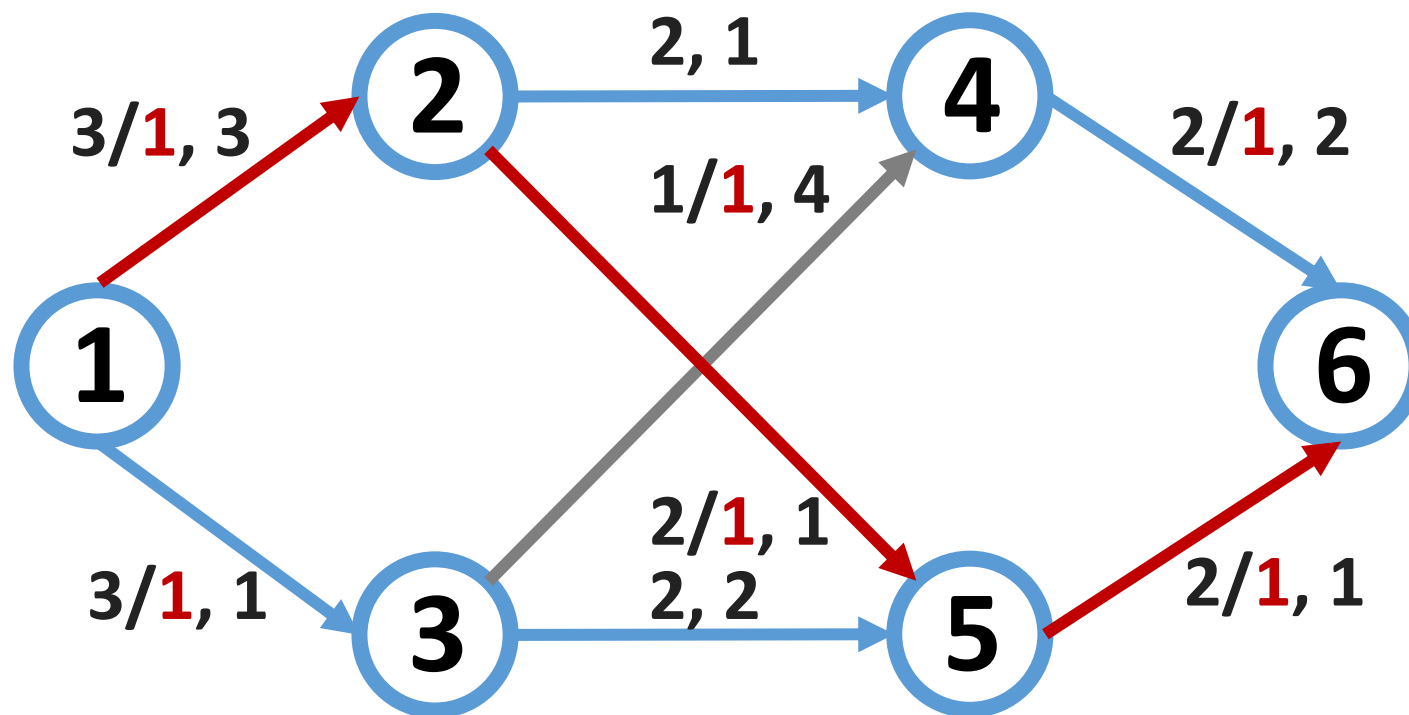
1-3-4-6

+1

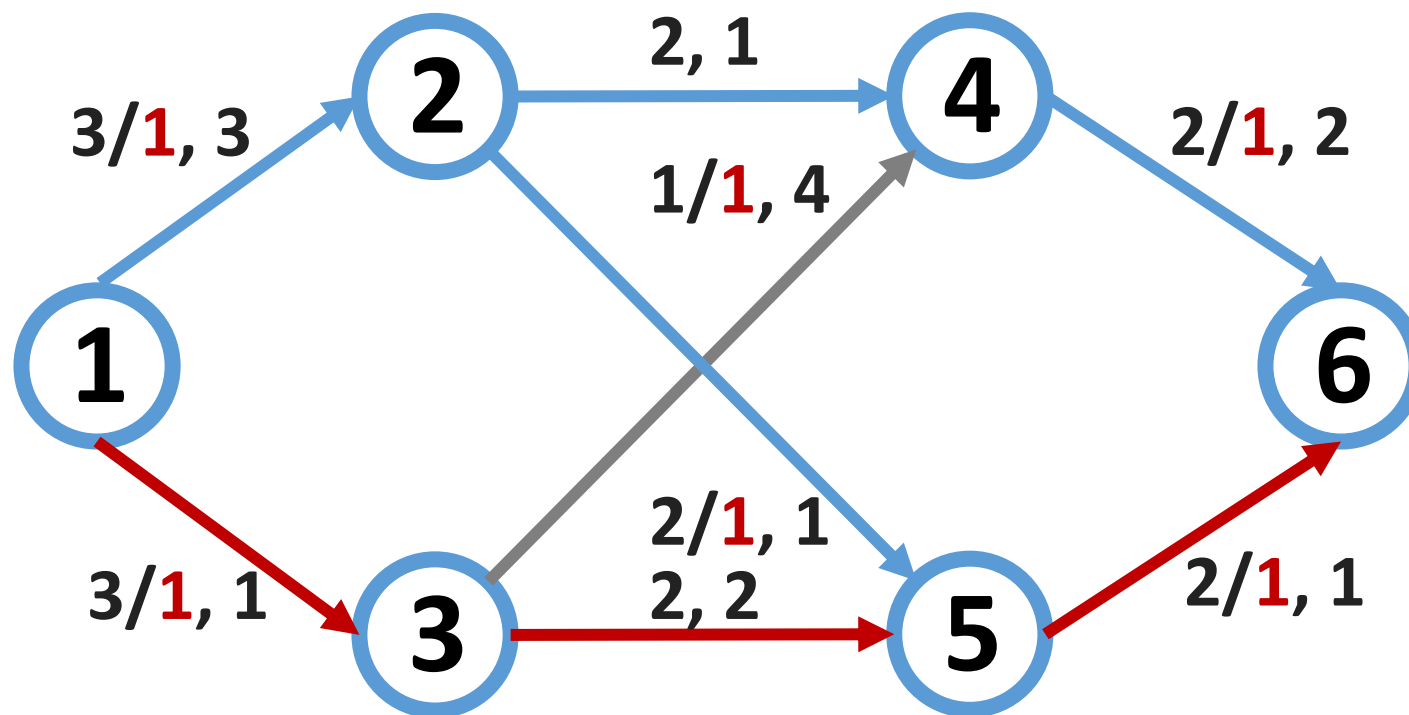


1-2-5-6

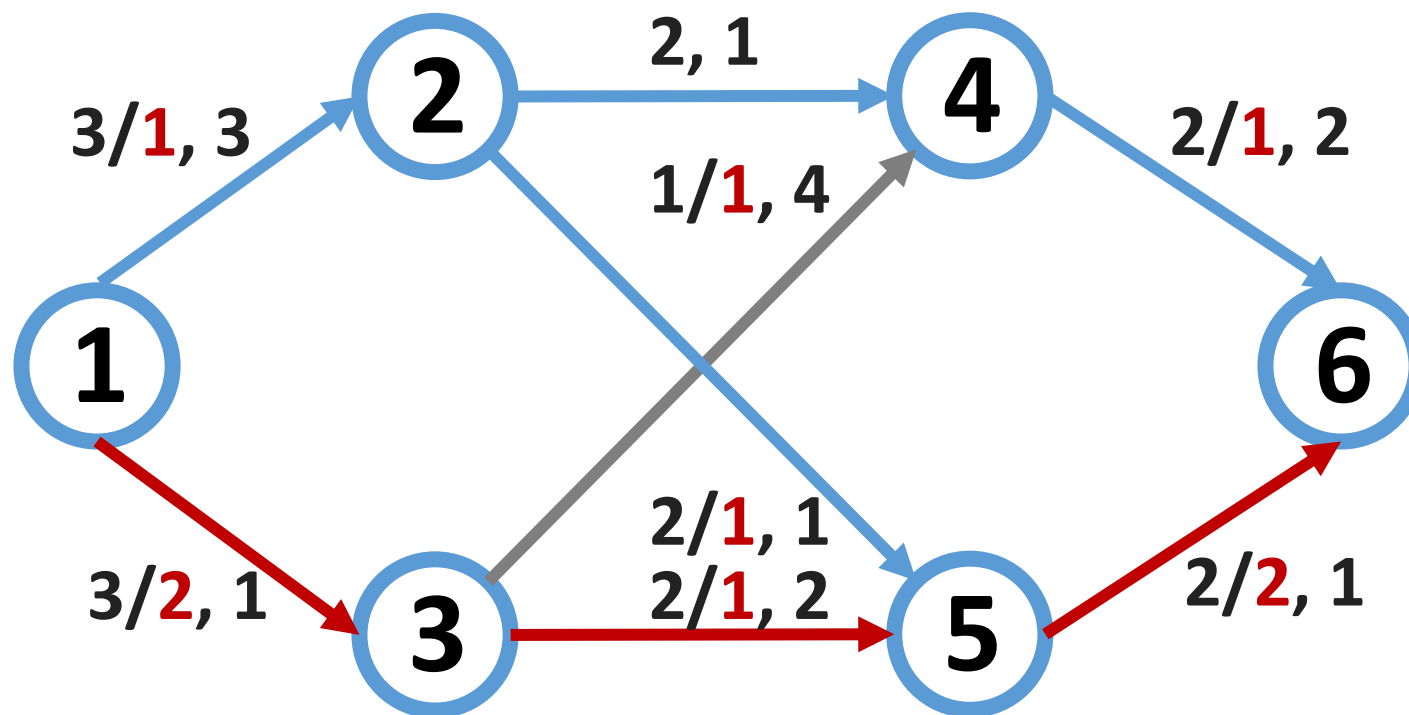
+1



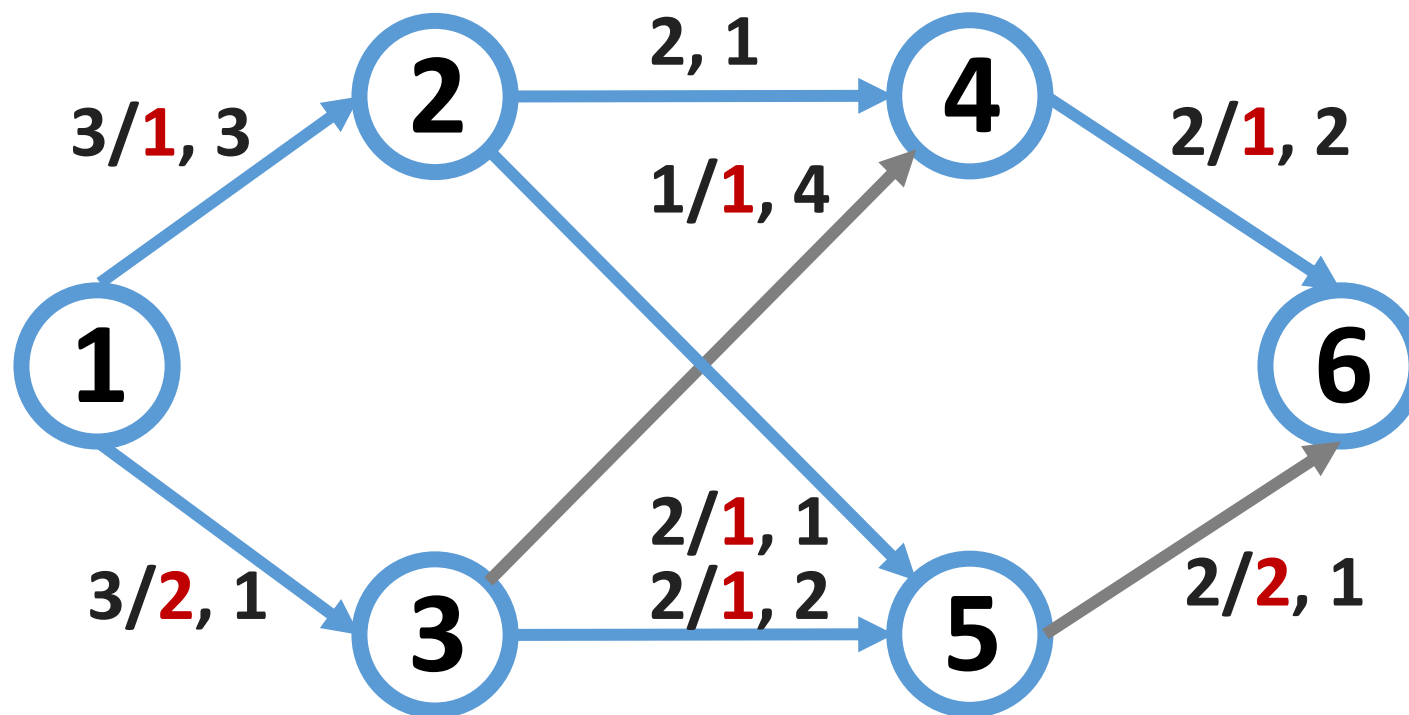
1-2-5-6



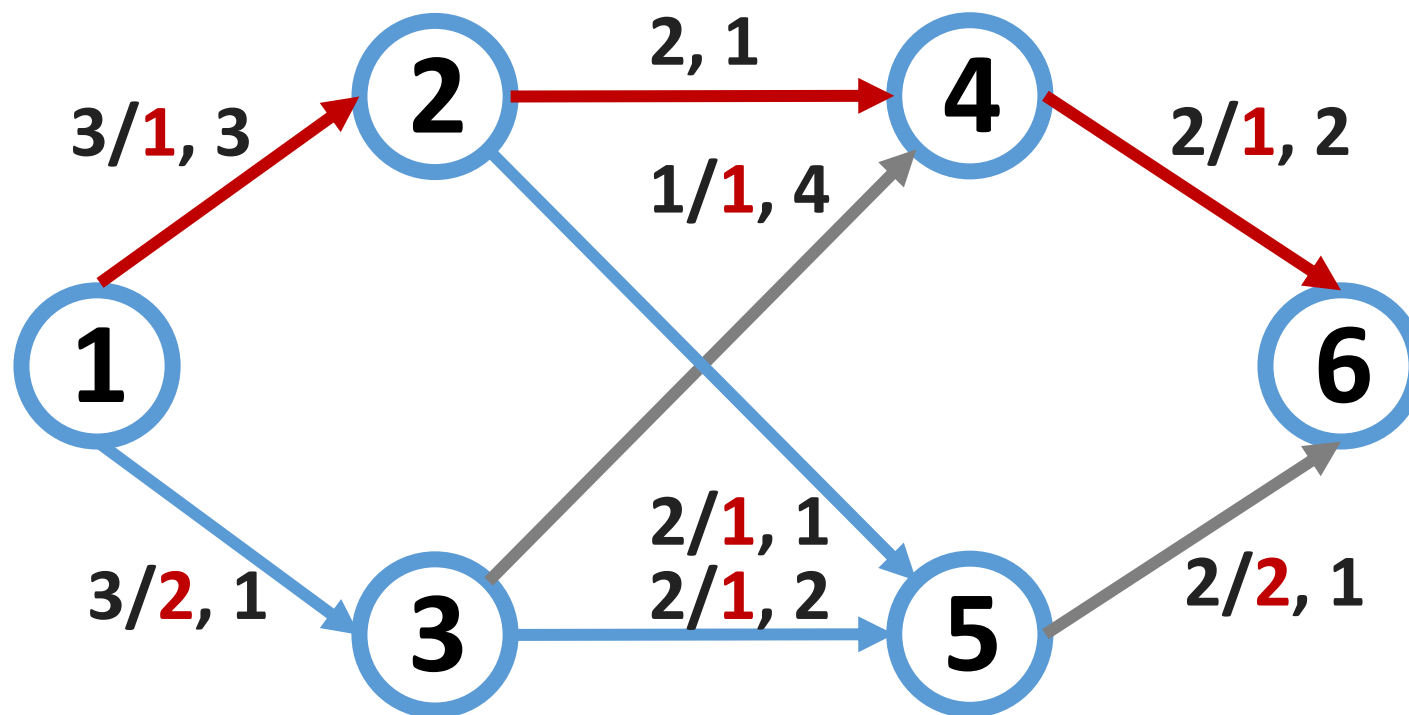
1-3-5-6



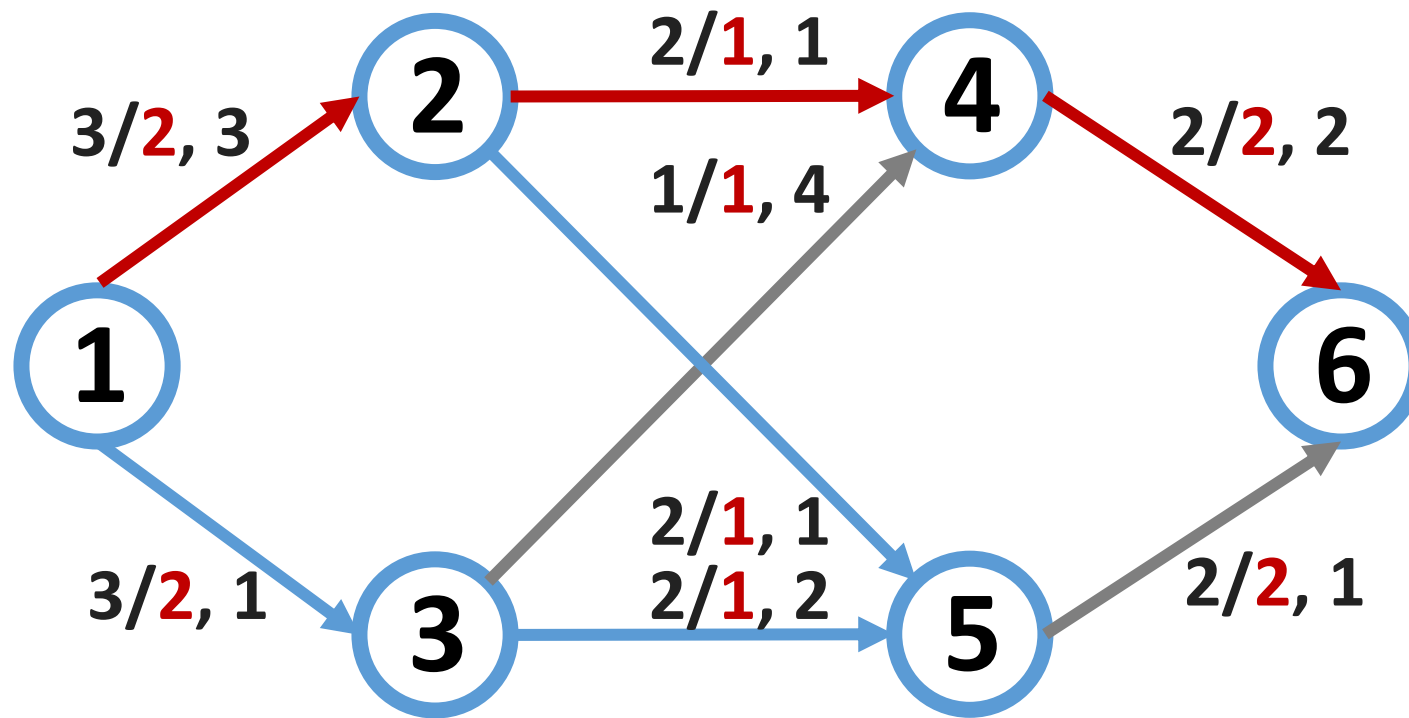
1-3-5-6



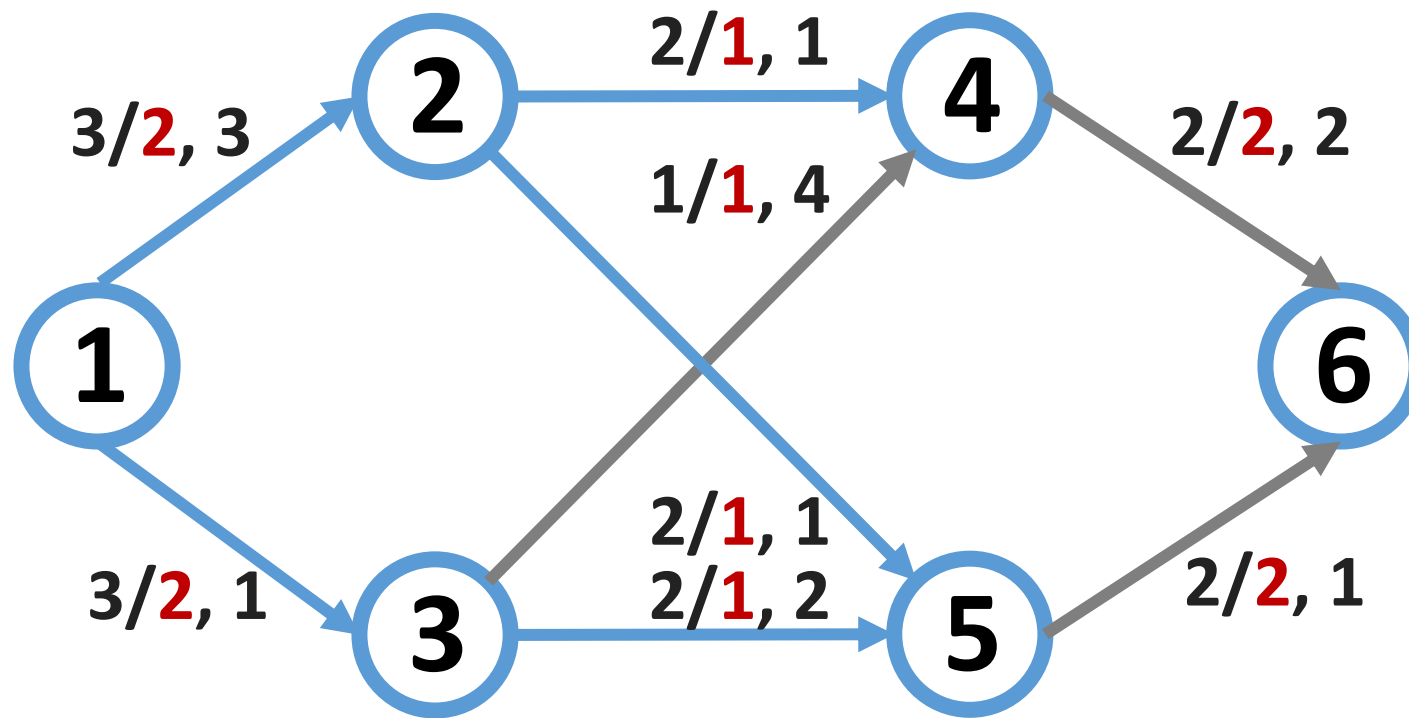
1-3-5-6



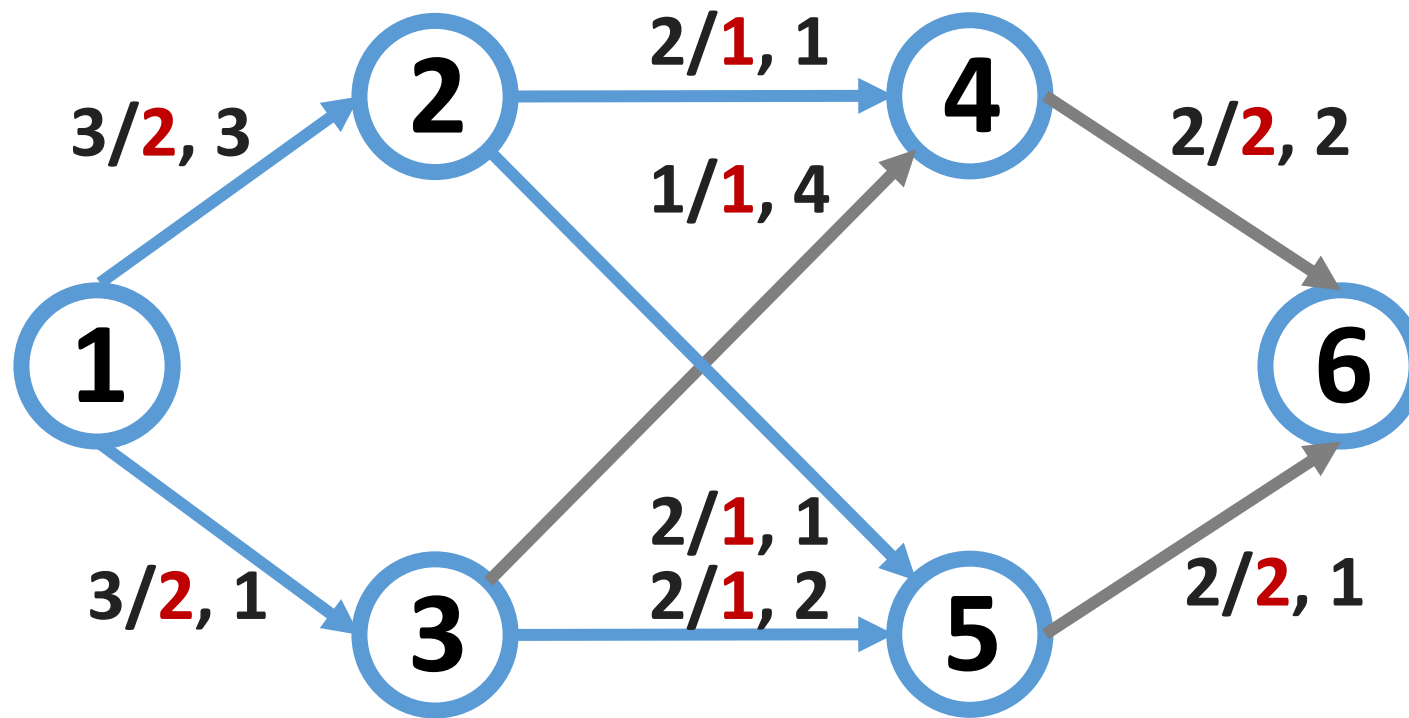
1-2-4-6



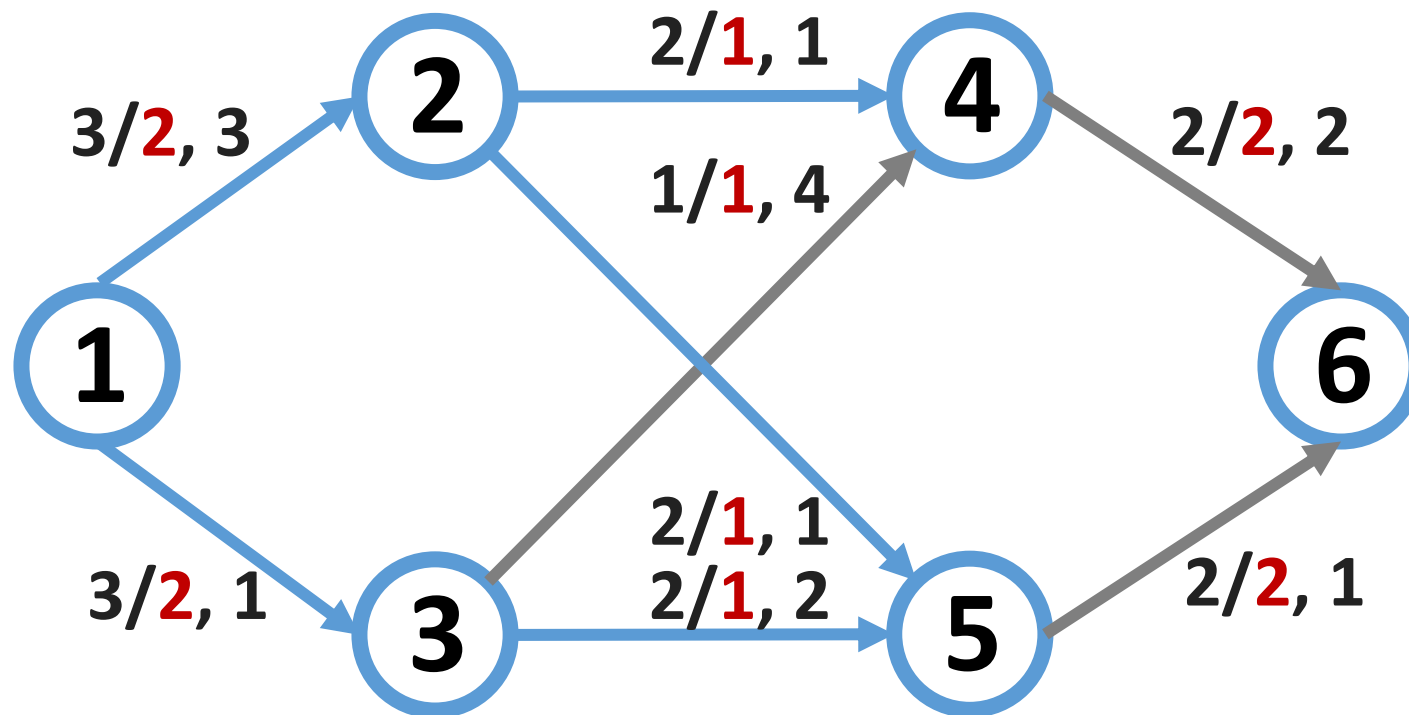
1-2-4-6



1-2-4-6

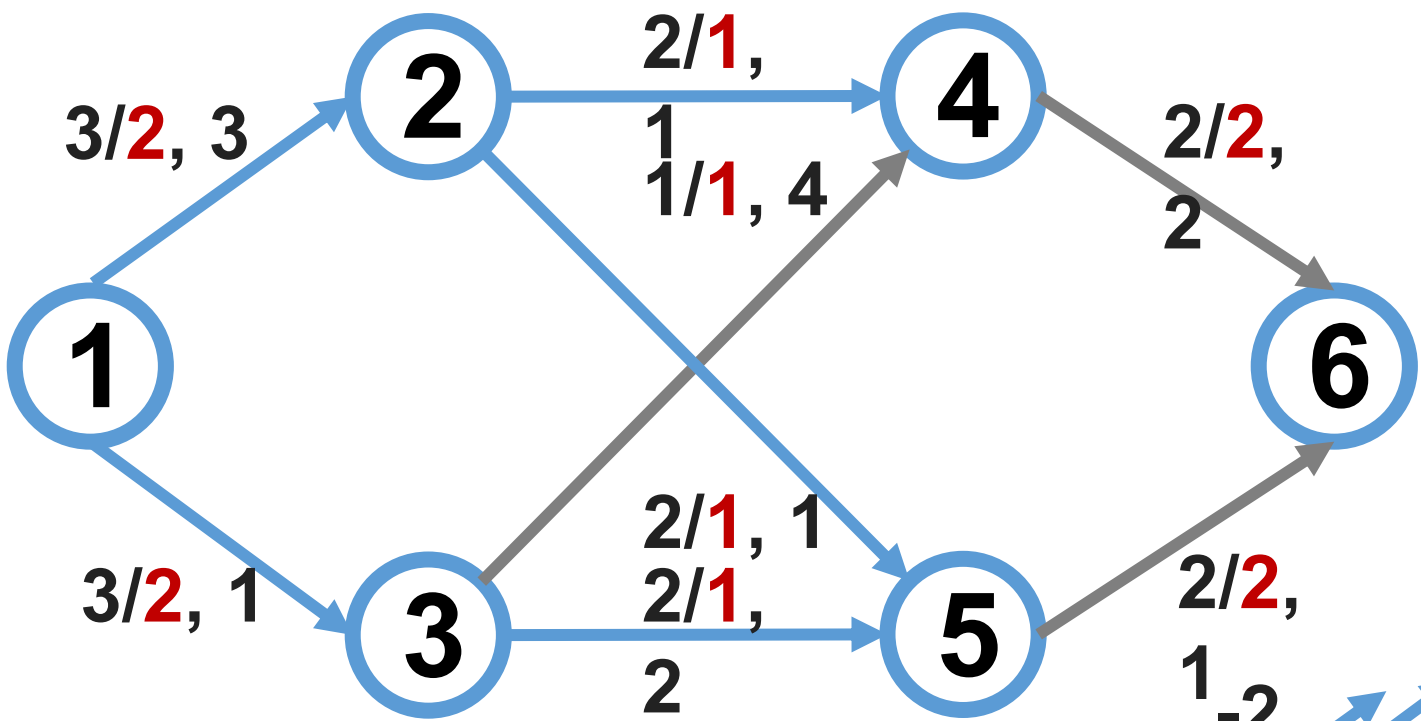


Путей нет!

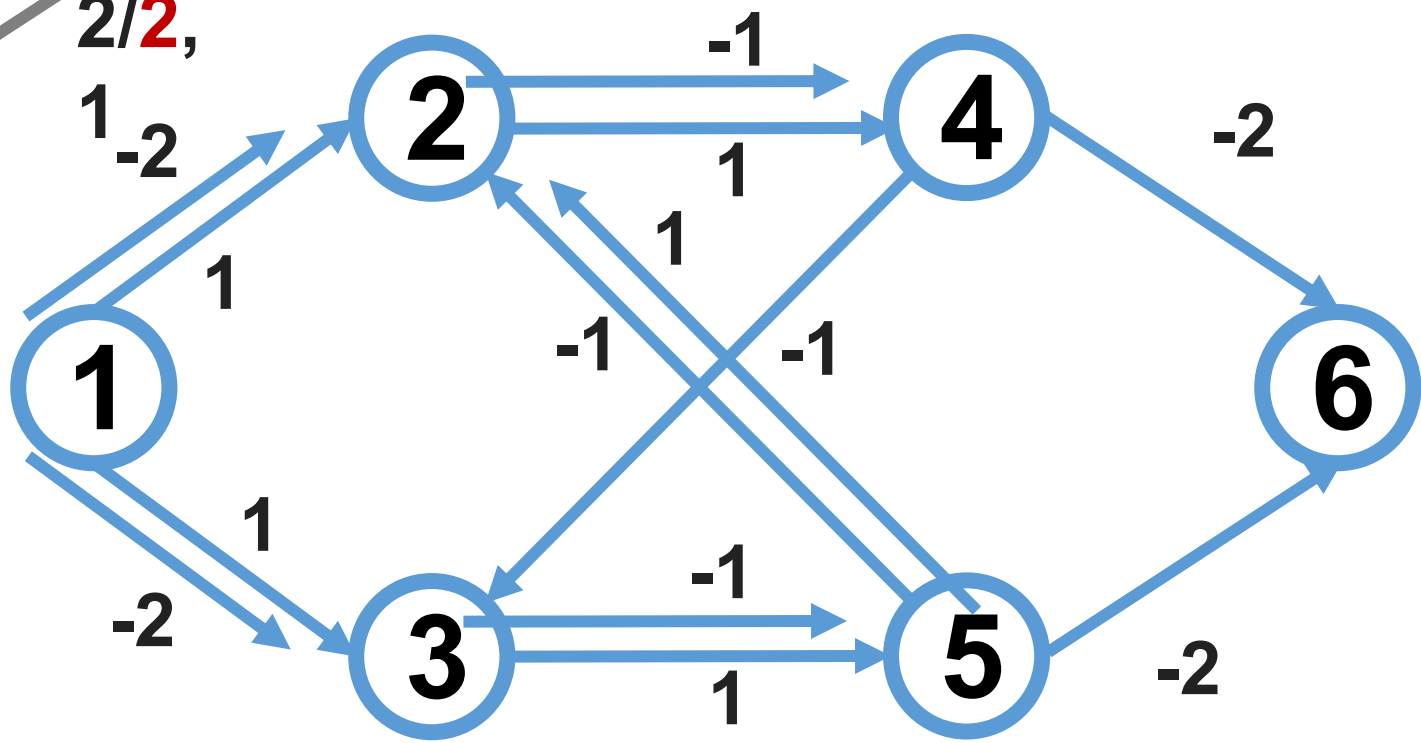


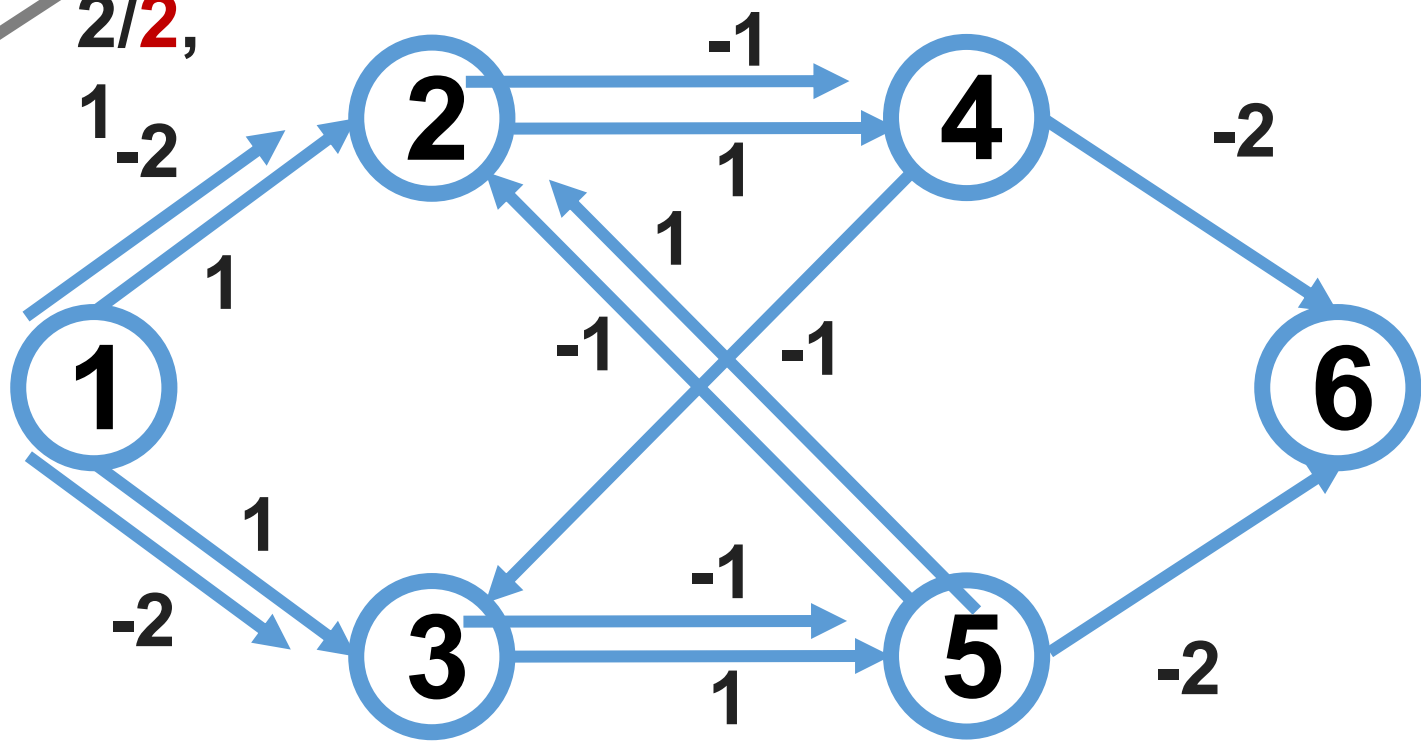
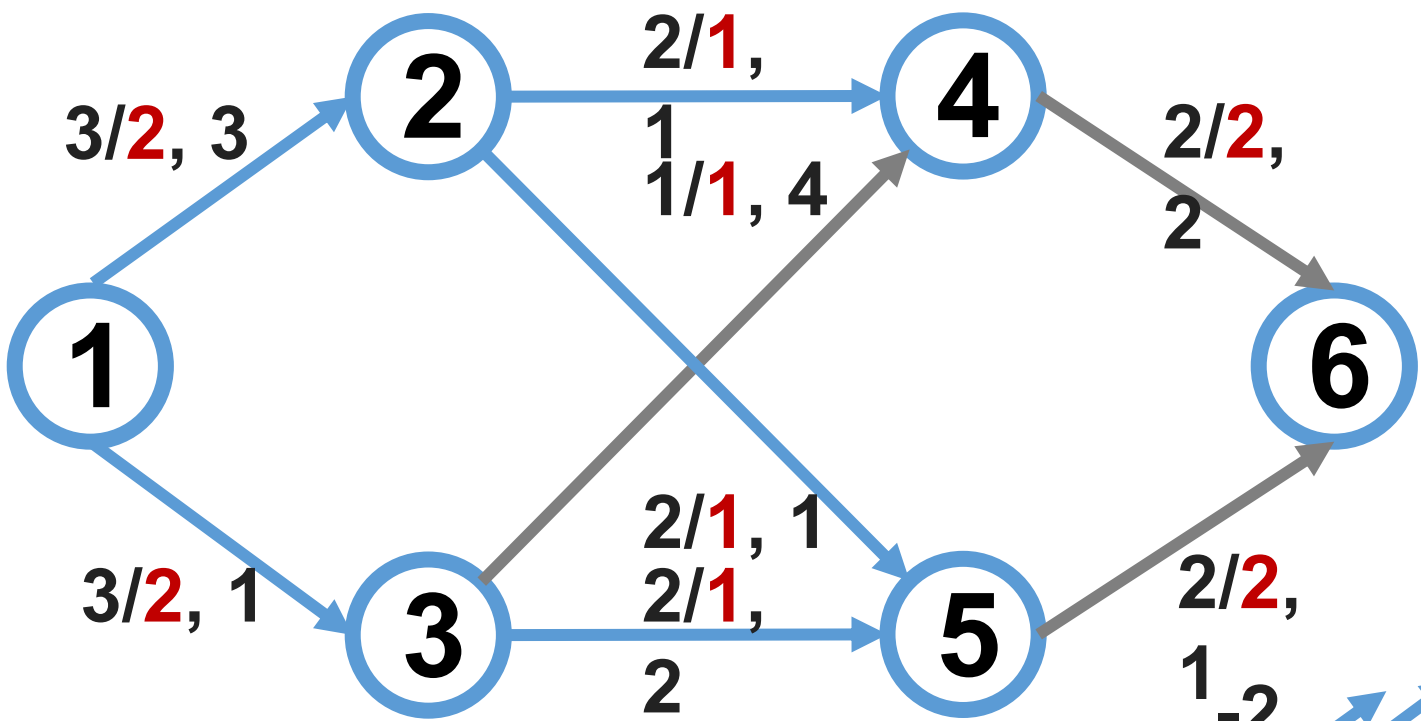
Максимальный поток: 4

Стоимость: $6+2+1+4+1+2+4+2=22$

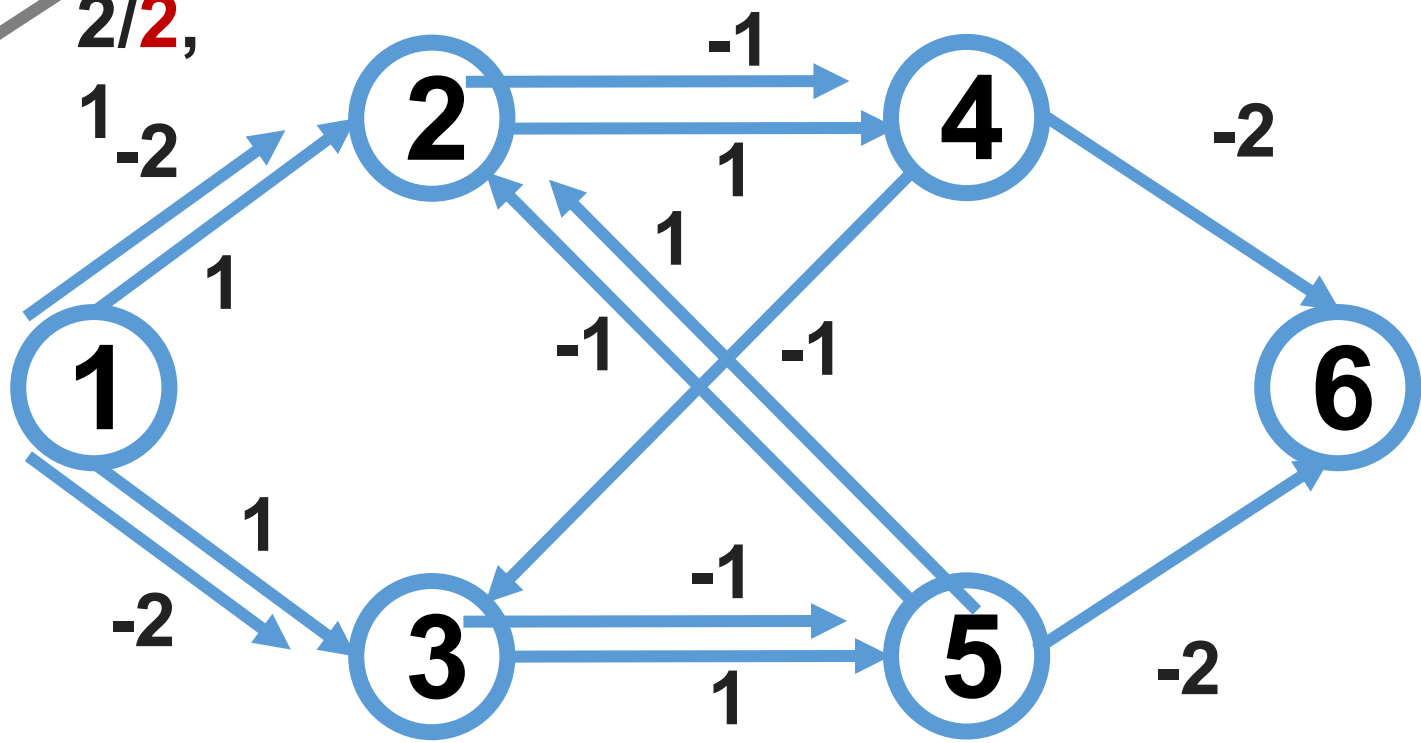
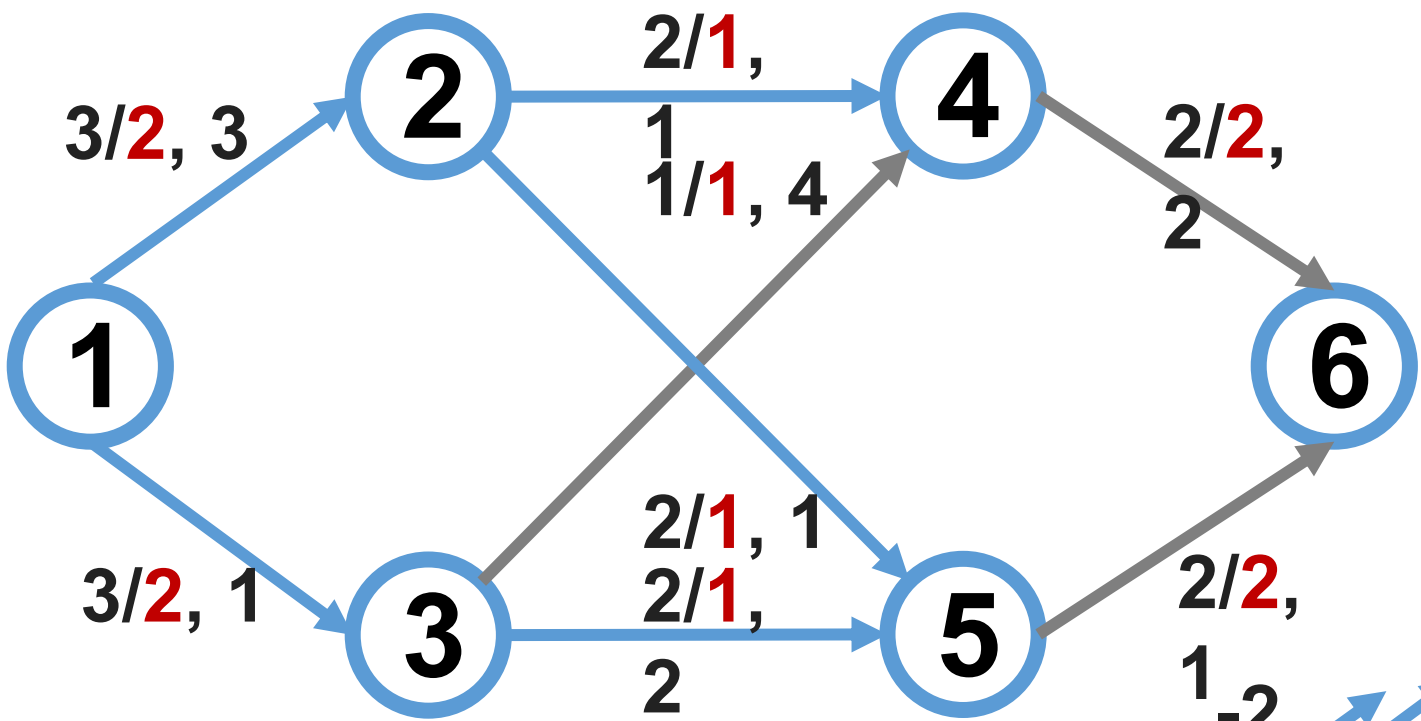


**Отрицательные
циклы
5-2-1-3-5**





**Отрицательные
ЦИКЛЫ**
5-2-1-3-5
4-3-1-2-4

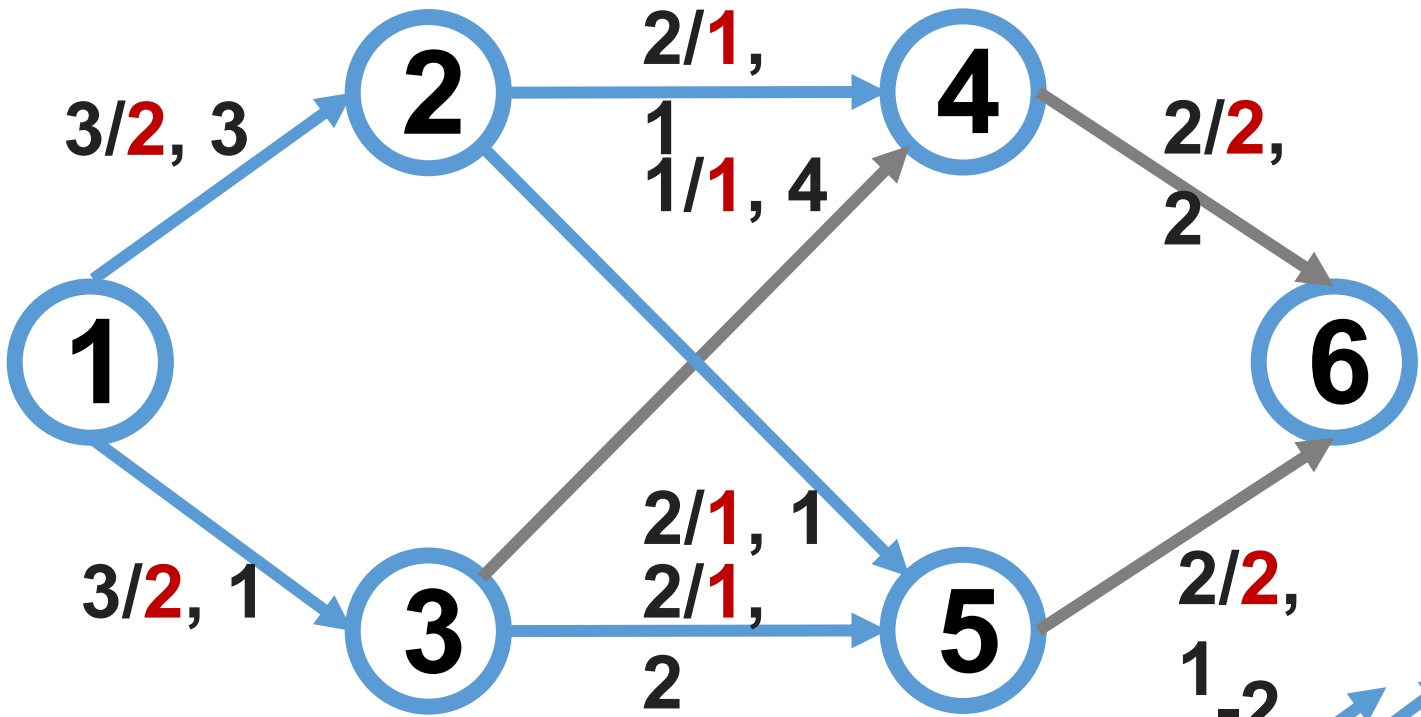


**Отрицательные
циклы**

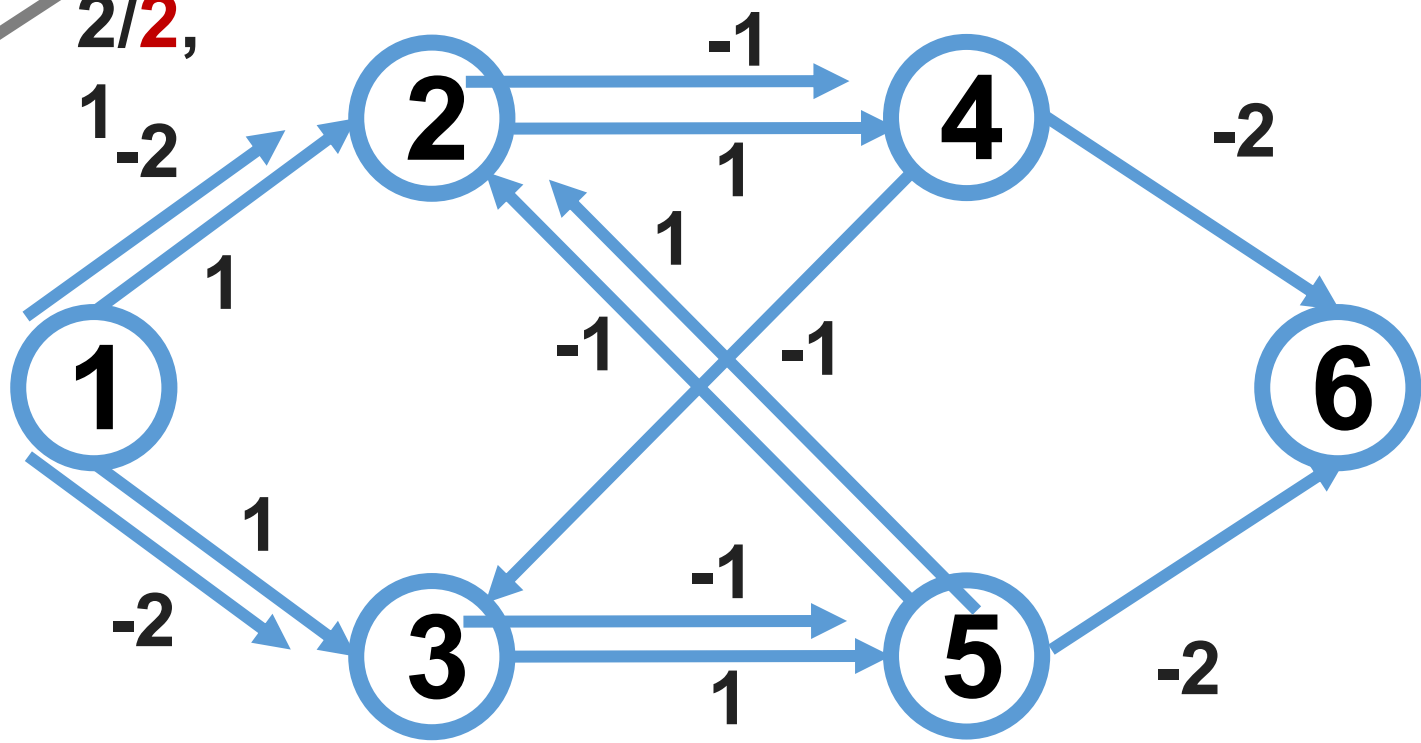
5-2-1-3-5

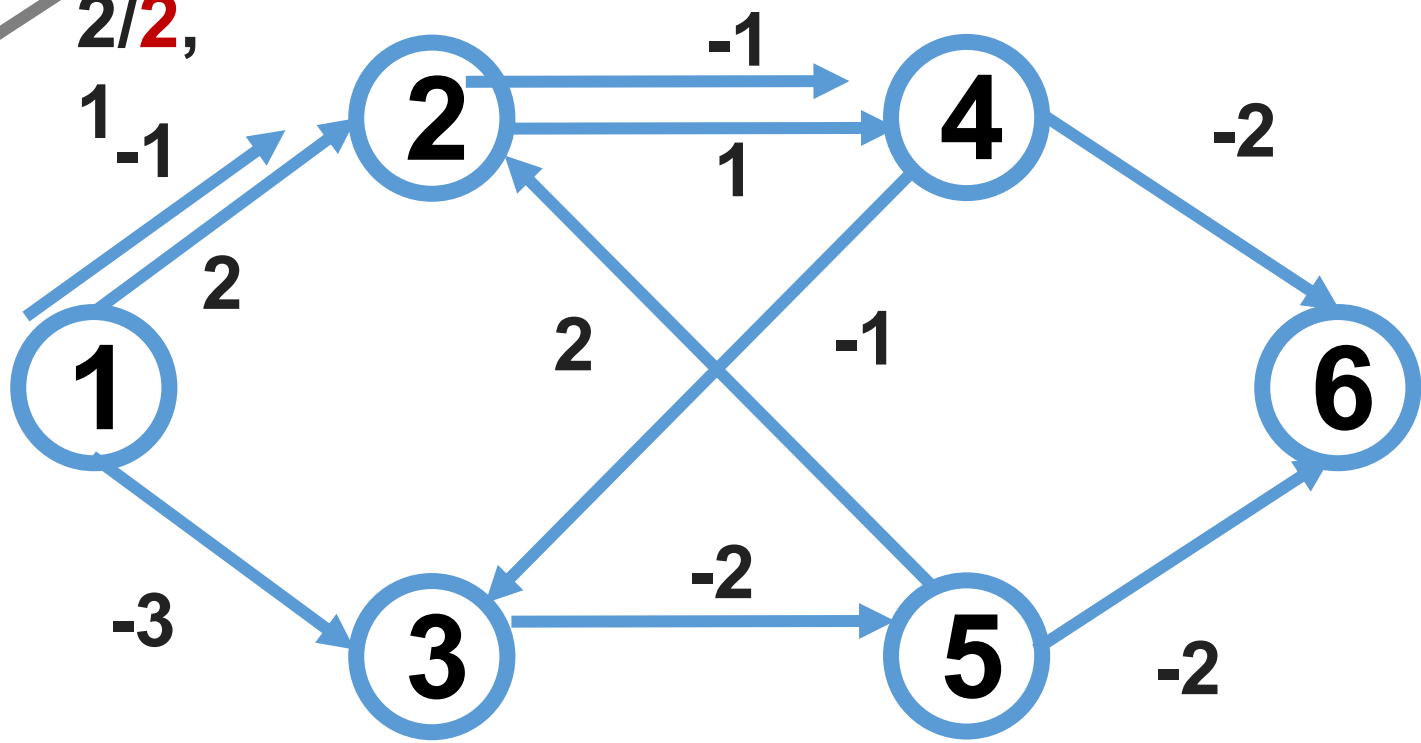
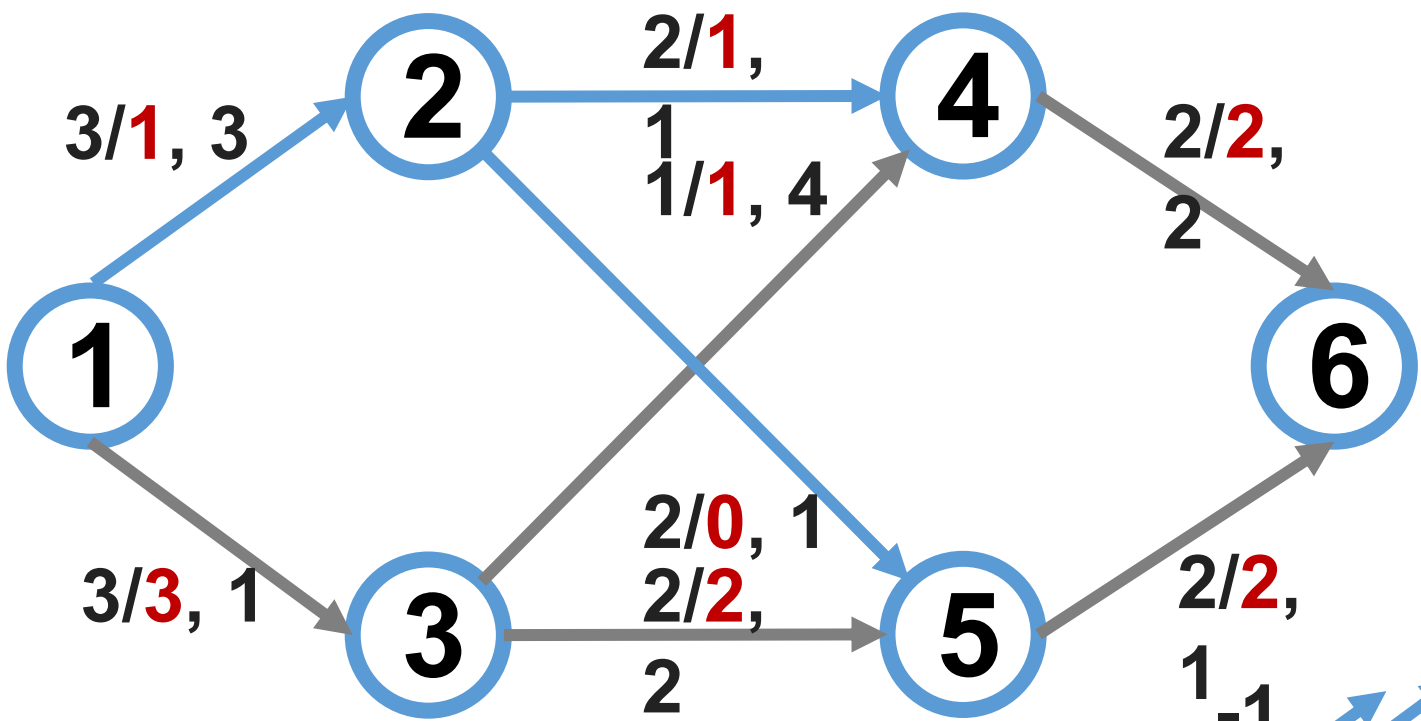
4-3-1-2-4

4-3-5-2-4

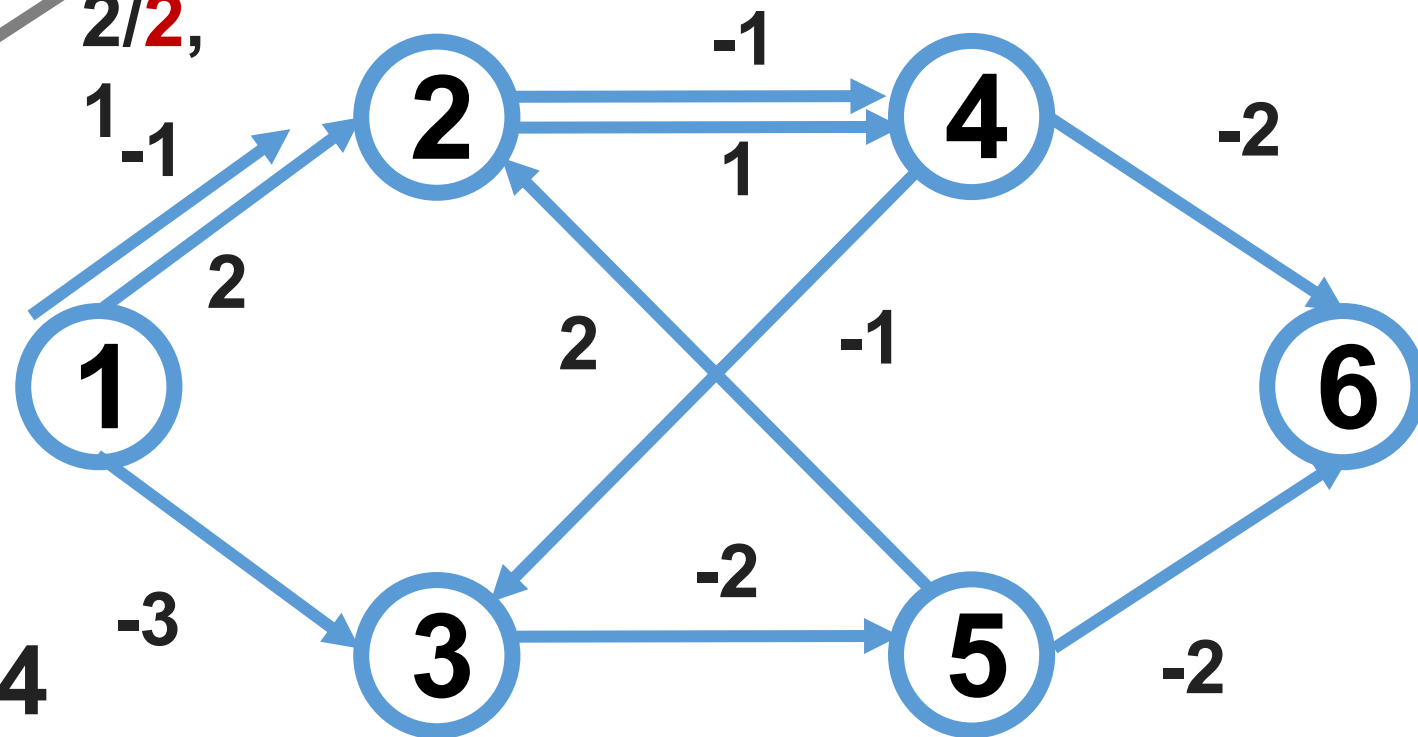
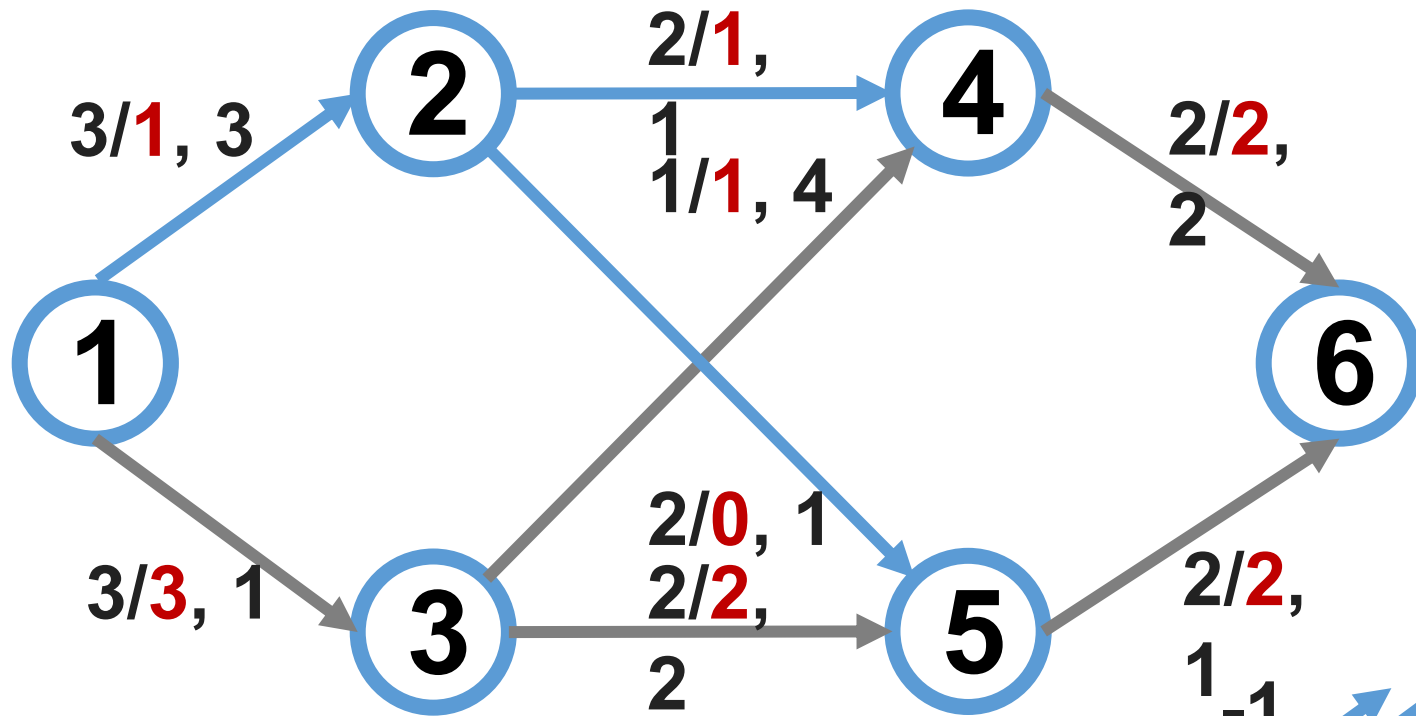


**Отрицательные
циклы
5-2-1-3-5**





Отрицательные
ЦИКЛЫ
5-2-1-3-5

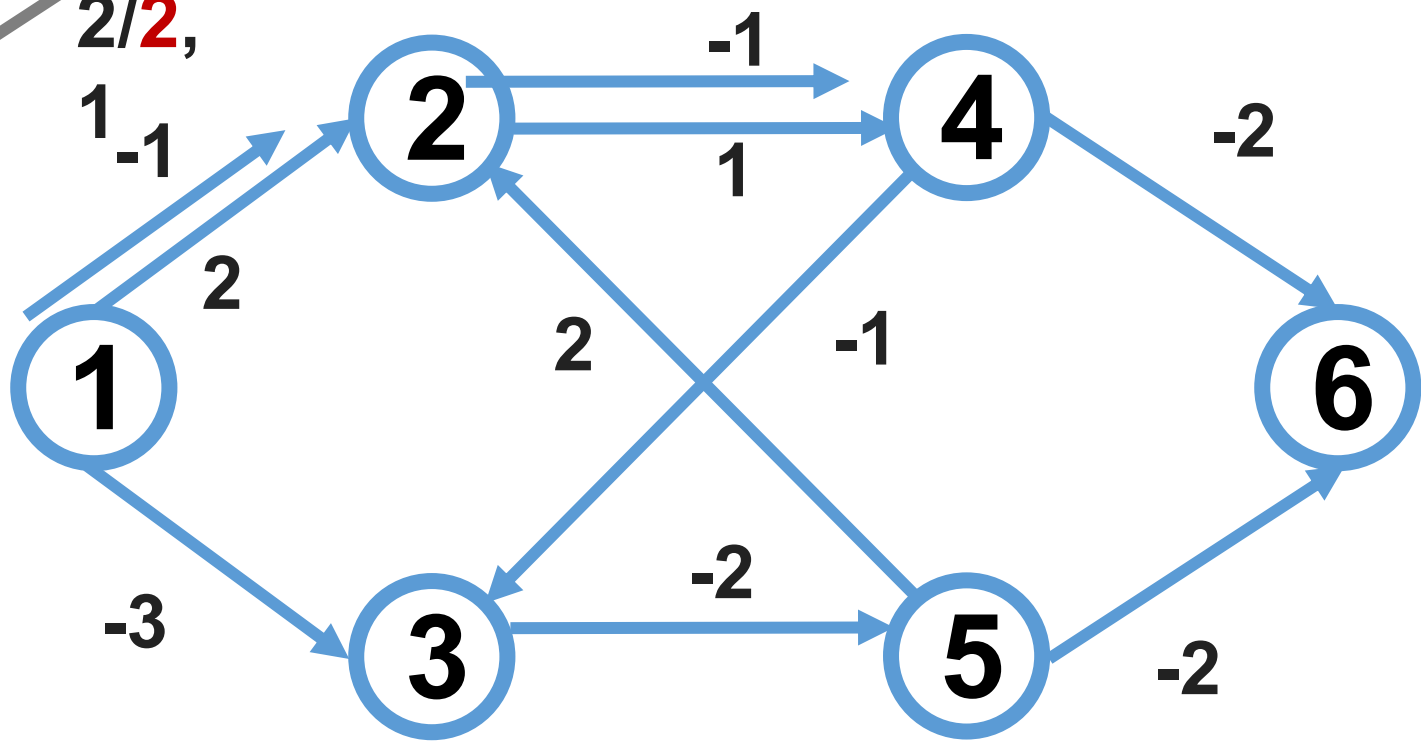
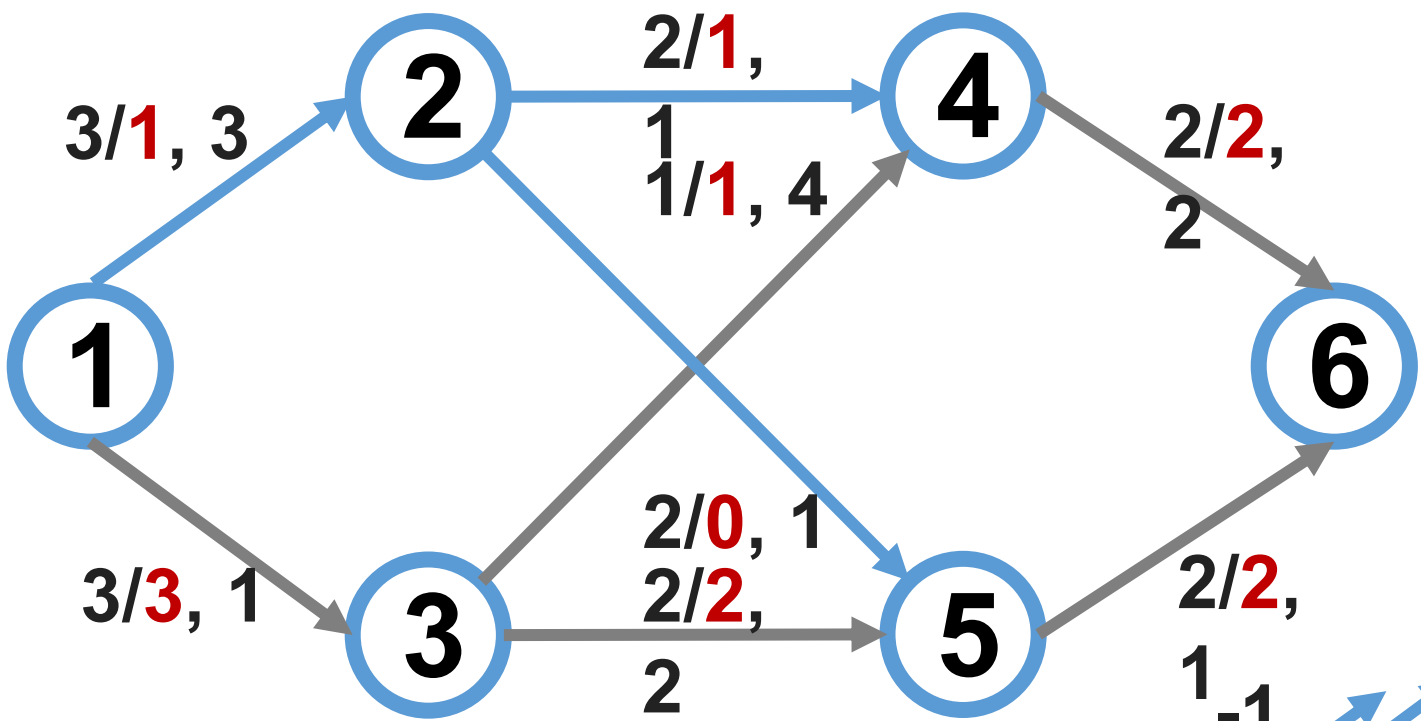


Отрицательные
циклы

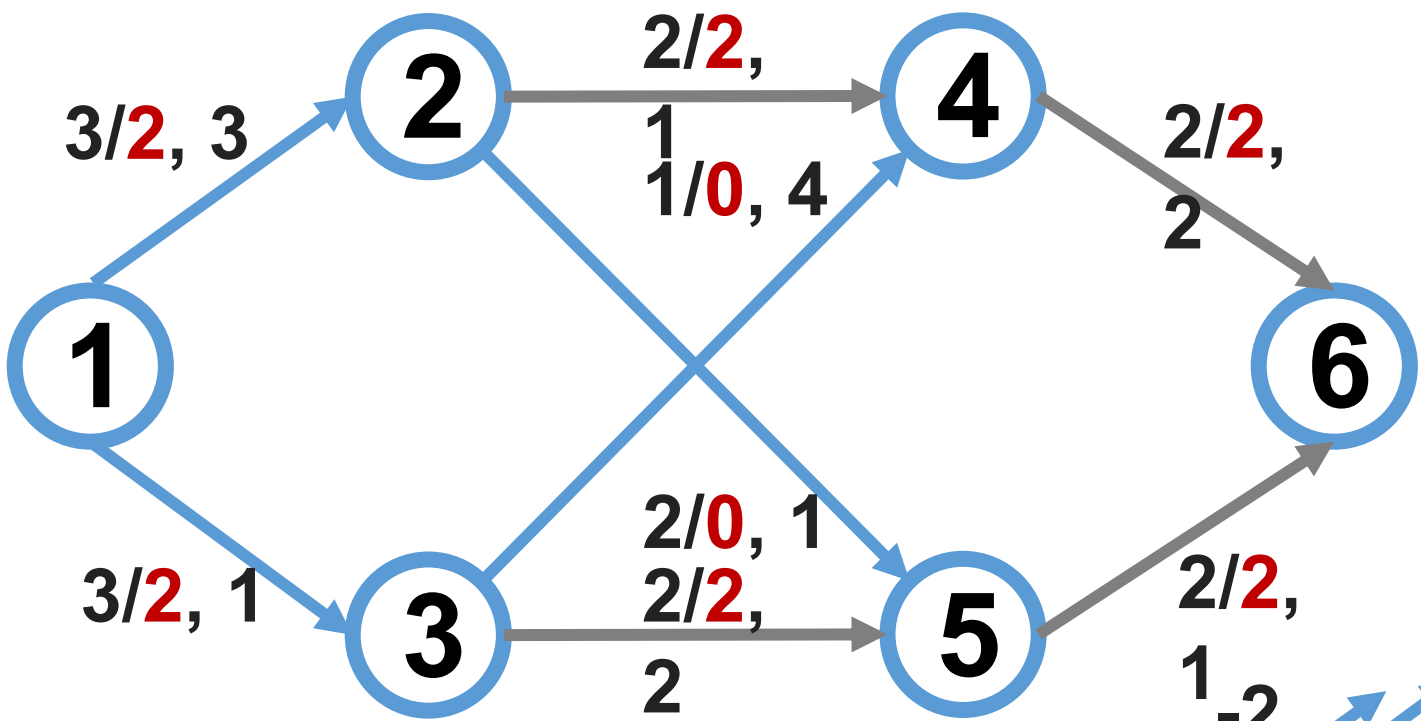
5-2-1-3-5

Максимальный поток: 4

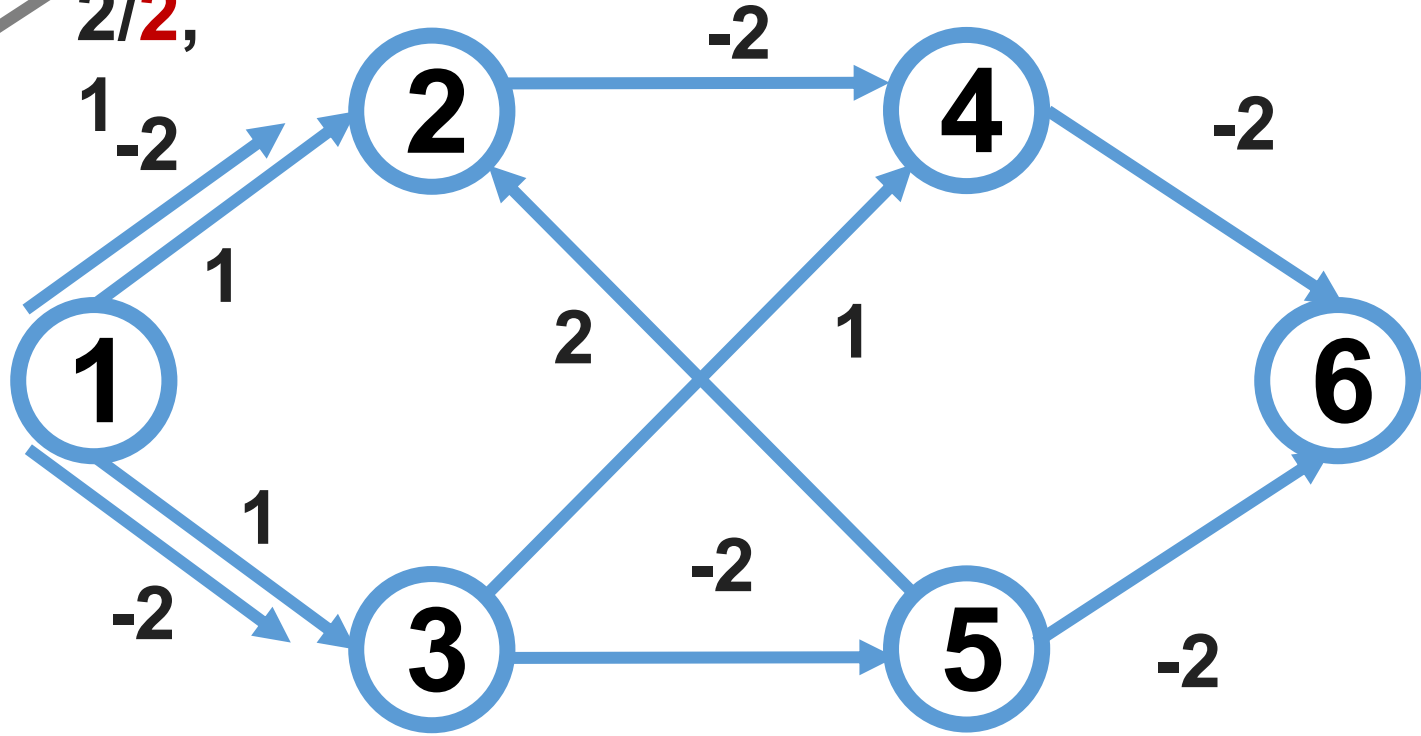
Стоимость: $3+3+1+4+4+4+2=21$

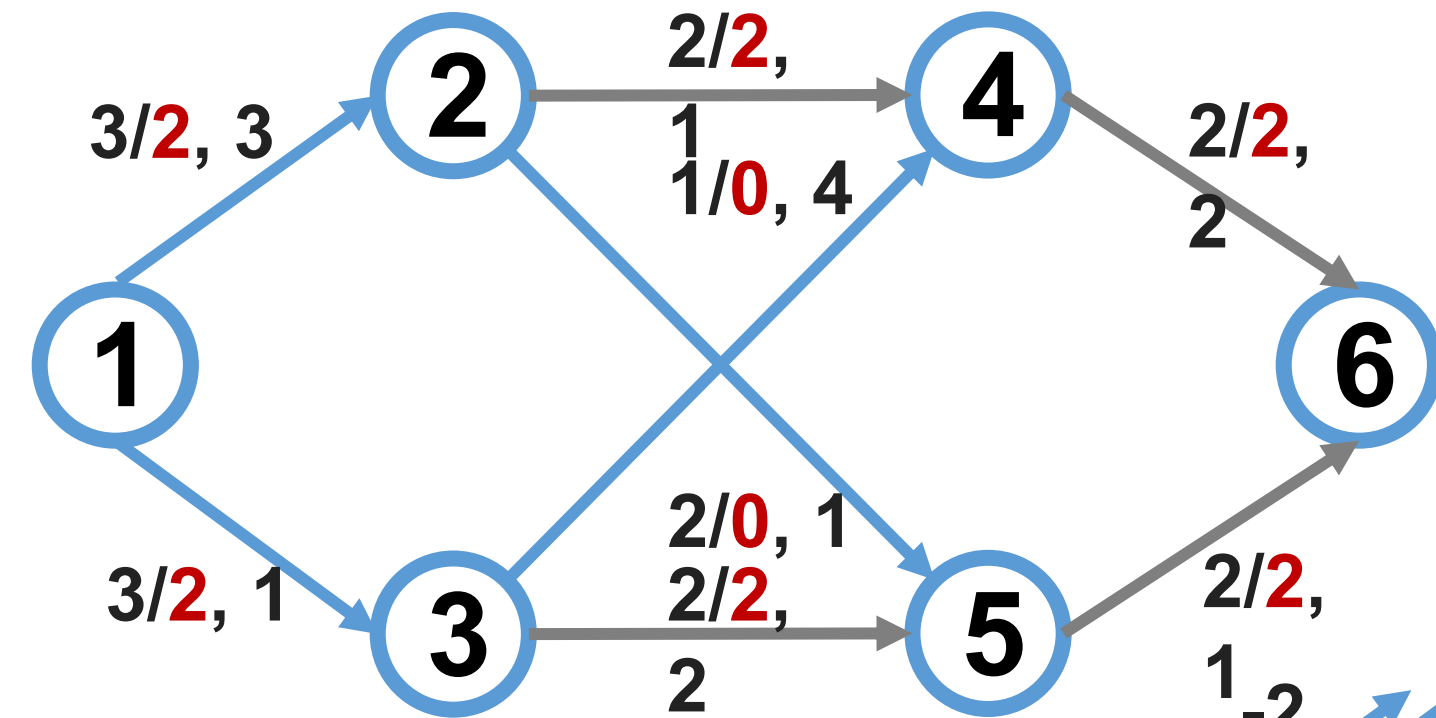


Отрицательные
циклы
4-3-1-2-4

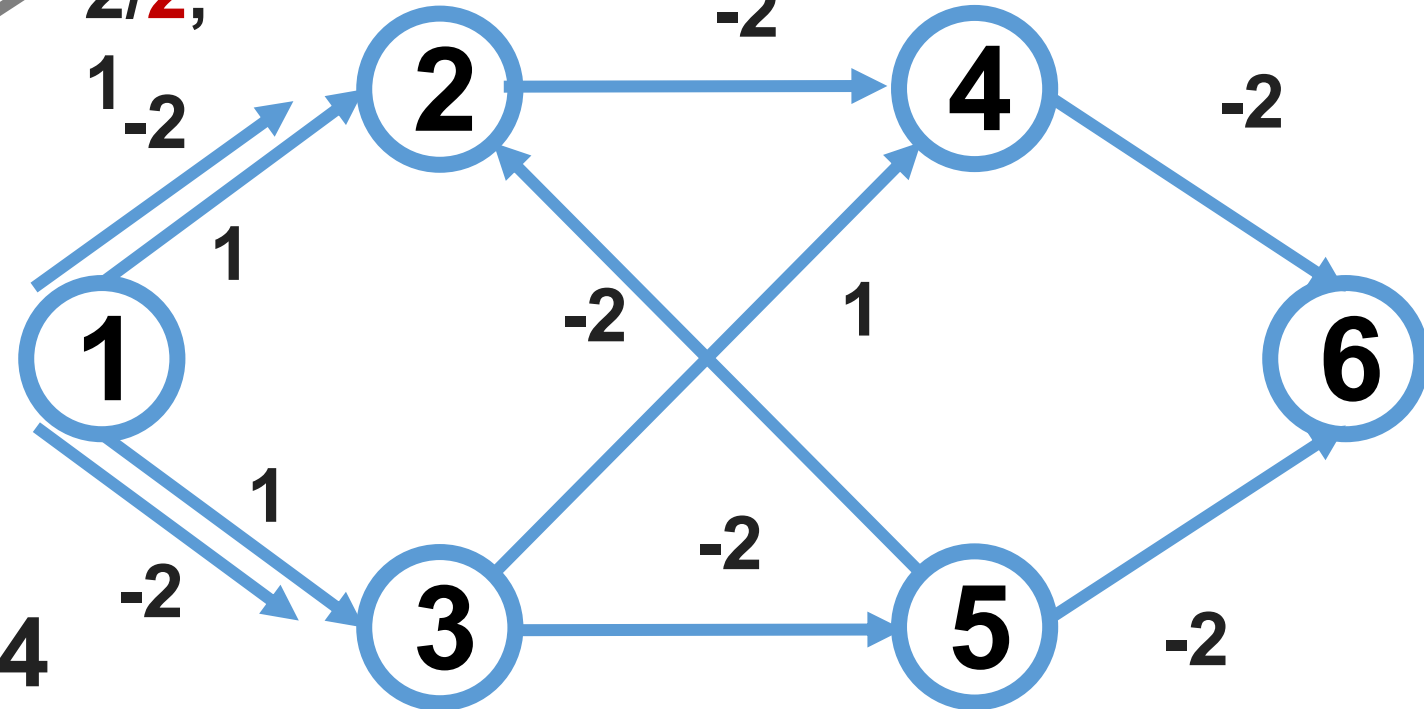


Отрицательные
циклы
4-3-1-2-4





Избавляемся от всех отрицательных циклов аналогичным образом.



Отрицательные циклы

4-3-1-2-4

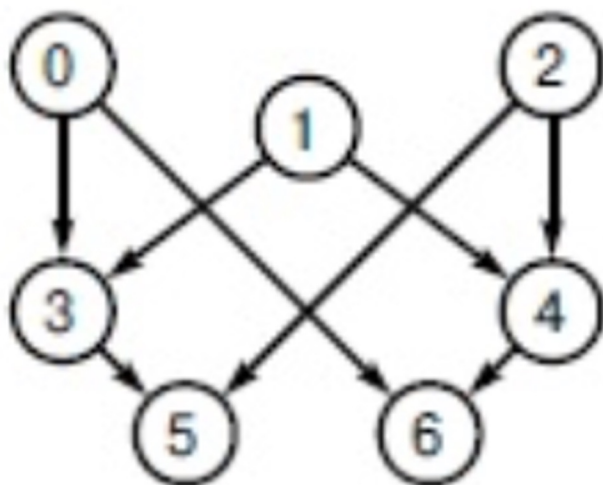
Максимальный поток: 4

Стоимость: $6+2+4+2+4+2=20$

Многополюсная сеть

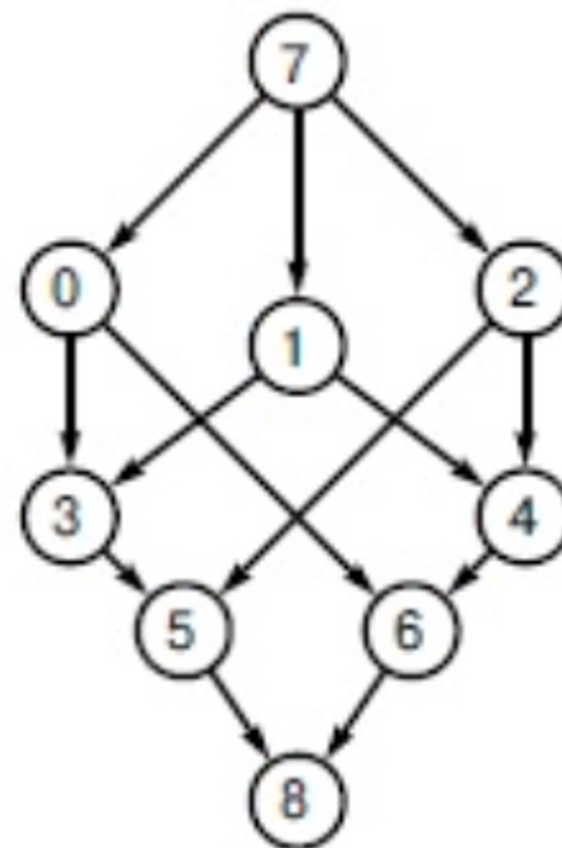
пропускная
способность

0-3	2
0-6	3
1-3	3
1-4	1
2-4	1
2-5	1
3-5	2
4-6	3



пропускная
способность

0-3	2
0-6	3
1-3	3
1-4	1
2-4	1
2-5	1
3-5	2
4-6	3
7-0	5
7-1	4
7-2	2
5-8	3
6-8	6



Многополюсная сеть

- Задача о максимальном потоке в сетях общего вида эквивалентна задаче о максимальном потоке в st -сетях.
- Добавим фиктивную вершину-исток s и ребра из нее во все истоки сети (пропускная способность каждого такого ребра равна оттоку из его конечной вершины), а также фиктивную вершину-сток t и ребра из каждого стока сети в нее (пропускная способность каждого такого ребра равна оттоку его начальной вершины). Любой максимальный поток в st -сети непосредственно соответствует максимальному потоку в исходной сети.

Остаточной сетью G^R по отношению к сети G и некоторому потоку f в ней называется сеть, в которой каждому ребру $(u, v) \in G$ с пропускной способностью c_{uv} и потоком f_{uv} соответствуют два ребра:

- (u, v) с пропускной способностью $c_{uv}^R = c_{uv} - f_{uv}$
- (u, v) с пропускной способностью $c_{uv}^R = f_{uv}$

Стоит отметить, что при таком определении в остаточной сети могут появляться кратные рёбра: если в исходной сети было как ребро (u, v) , так и (v, u) .

Остаточное ребро можно интуитивно понимать как меру того, насколько ещё можно увеличить поток вдоль какого-то ребра. В самом деле, если по ребру (u, v) с пропускной способностью c_{uv} протекает поток f_{uv} , то потенциально по нему можно пропустить ещё $c_{uv} - f_{uv}$ единиц потока, а в обратную сторону можно пропустить до f_{uv} единиц потока, что будет означать отмену потока в первоначальном направлении.

Блокирующим потоком в данной сети называется такой поток, что любой путь из истока s в сток t содержит насыщенное этим потоком ребро. Иными словами, в данной сети не найдётся такого пути из истока в сток, вдоль которого можно беспрепятственно увеличить поток.

Блокирующий поток не обязательно максимален. Теорема Форда-Фалкерсона говорит о том, что поток будет максимальный тогда и только тогда, когда в остаточной сети не найдётся $s - t$ пути; в блокирующем же потоке ничего не утверждается о существовании пути по рёбрам, появляющимся в остаточной сети.

Слоистая сеть для данной сети строится следующим образом: сначала определяются длины кратчайших путей из истока s до всех остальных вершин; назовём уровнем $\text{level}[u]$ вершины её расстояние от истока. Тогда в слоистую сеть включают все те рёбра (u, v) исходной сети, которые ведут с одного уровня на какой-либо другой, более поздний, уровень, т. е. $\text{level}[u] + 1 = \text{level}[v]$ (почему в этом случае разница расстояний не может превосходить единицы, следует из свойства кратчайших расстояний). Таким образом, удаляются все рёбра, расположенные целиком внутри уровней, а также рёбра, ведущие назад, к предыдущим уровням.

Очевидно, слоистая сеть ациклична. Кроме того, любой $s - t$ путь в слоистой сети является кратчайшим путём в исходной сети.

Построить слоистую сеть по данной сети очень легко: для этого надо запустить обход в ширину по рёбрам этой сети, посчитав тем самым для каждой вершины величину $\text{level}[]$, и затем внести в слоистую сеть все подходящие рёбра.

Примечание: термин “слоистая сеть” в русскоязычной литературе не употребляется; обычно эта конструкция называется просто “вспомогательным графом”. Впрочем, на английском языке обычно используется термин “layered network”.

Алгоритм представляет собой несколько **фаз**. На каждой фазе сначала строится остаточная сеть, затем по отношению к ней строится слоистая сеть (обходом в ширину), а в ней ищется произвольный блокирующий поток. Найденный блокирующий поток прибавляется к текущему потоку, и на этом очередная итерация заканчивается.

Этот алгоритм схож с алгоритмом Эдмондса-Карпа, но основное можно понимать так: на каждой итерации поток увеличивается не вдоль одного кратчайшего $s - t$ пути, а вдоль целого набора таких путей (ведь именно такими путями и являются пути в блокирующем потоке слоистой сети).

Чтобы завершить построение алгоритма Диница, надо описать алгоритм нахождения блокирующего потока в слоистой сети - ключевое место алгоритма.

Мы рассмотрим три возможных варианта реализации поиска блокирующего потока:

- Искать $s - t$ пути по одному, пока такие пути находятся. Путь можно найти за $O(m)$ обходов в глубину, а всего таких путей будет $O(m)$ (поскольку каждый путь насыщает как минимум одно ребро). Итоговая асимптотика поиска одного блокирующего потока составит $O(m^2)$.
- Аналогично предыдущей идее, однако удалять в процессе обхода в глубину из графа все “лишние” рёбра, т. е. рёбра, вдоль которых не получится дойти до стока.

Это очень легко реализовать: достаточно удалять ребро после того, как мы просмотрели его в обходе в глубину (кроме того случая, когда мы прошли вдоль ребра и нашли путь до стока). С точки зрения реализации, надо просто поддерживать в списке смежности каждой вершины указатель на первое неудалённое ребро, и увеличивать этот указатель в цикле внутри обхода в глубину.

Оценим асимптотику этого решения. Каждый обход в глубину завершается либо насыщением как минимум одного ребра (если этот обход достиг стока), либо продвижением вперёд как минимум одного указателя (в противном случае). Можно понять, что один запуск обхода в глубину из основной программы работает за $O(k + n)$, где k - число продвижений указателей. Учитывая, что всего запусков обхода в глубину в рамках поиска одного блокирующего потока будет $O(p)$, где p - число рёбер, насыщенных этим блокирующим потоком, то весь алгоритм поиска блокирующего потока отработает за $O(pk + pn)$, что, учитывая, что все указатели в сумме прошли расстояние $O(m)$, даёт асимптотику $O(m + pn)$. В худшем случае, когда блокирующий поток насыщает все рёбра, асимптотика получается $O(nm)$; эта асимптотика и будет использоваться далее.

```

const int MAXN = ...; // число вершин
const int INF = 1000000000; // константа-бесконечность

int n, c[MAXN][MAXN], f[MAXN][MAXN], s, t, d[MAXN],
ptr[MAXN], q[MAXN];

bool bfs() {
    int qh=0, qt=0;
    q[qt++] = s;
    memset (d, -1, n * sizeof d[0]);
    d[s] = 0;
    while (qh < qt) {
        int v = q[qh++];
        for (int to=0; to<n; ++to)
            if (d[to] == -1 && f[v][to] <
c[v][to]) {
                q[qt++] = to;
                d[to] = d[v] + 1;
            }
    }
    return d[t] != -1;
}

```

```

int dfs (int v, int flow) {
    if (!flow) return 0;
    if (v == t) return flow;
    for (int & to=ptr[v]; to<n; ++to) {
        if (d[to] != d[v] + 1) continue;
        int pushed = dfs (to, min (flow,
c[v][to] - f[v][to]));
        if (pushed) {
            f[v][to] += pushed;
            f[to][v] -= pushed;
            return pushed;
        }
    }
    return 0;
}

int dinic() {
    int flow = 0;
    for (;;) {
        if (!bfs()) break;
        memset (ptr, 0, n * sizeof
ptr[0]);

        while (int pushed = dfs (s, INF))
            flow += pushed;
    }
    return flow;
}

```

Первая теорема Карзанова:

Число итераций алгоритма Диница в сети N (s - исток, t - сток) с целочисленными пропускными способностями - $O(\sqrt{N})$.

Вторая теорема Карзанова:

Число итераций алгоритма Диница с целочисленными пропускными способностями - $O(C^{1/3} |V|^{2/3})$.

Алгоритм Карзанова состоит из нескольких этапов, на каждом из которых ищется тупиковый поток в многослойной сети.

При поиске тупикового потока каждый раз насыщается один узел. Каждый этап начинается с проталкивания предпотока из V_0 , а затем из слоя в слой до тех пор, пока не будет достигнут узел V_n . Затем происходит балансировка предпотока в каждом узле. Таким образом, каждый этап состоит из двух шагов. Первый шаг называется продвижением предпотока, а второй - балансировкой предпотока. Эти шаги повторяются до получения тупикового потока в остаточной подсети.

Трудоемкость алгоритма Карзанова построения тупиковых потоков в многослойной сети ограничена величиной $O(n^2)$. Поскольку число этапов равно n , суммарное время работы алгоритма Карзанова составляет $O(n^3)$.